ADG Education

Campus Paris · 15 rue du Louvre, 75001 Paris

TITRE PROFESSIONNEL

Concepteur développeur d'applications

CDA · RNCP 37873 · niveau 6 (bachelor)

MÉMOIRE DE FIN DE FORMATION

Conception et développement de Briefly

Application web et mobile de suivi de l'avancement des équipes

Présenté par Chiraz Lamraoui

Entreprise d'accueil : Drif Consult

Tuteur entreprise : M. Yasser El Habib Drif

Enseignant responsable : M. Anis BENAMMAR

Soutenance : 27 octobre 2026

Année académique 2025-2026

Remerciements

Je tiens à remercier **ADG Education – Campus Paris**, où j'ai eu l'opportunité d'effectuer mon Bachelor et de construire mon parcours académique et professionnel.

Je remercie également **Anis Benammar**, mon encadrant, pour son accompagnement dans le cadre de ce projet de fin d'études, ainsi que **Yasser El Habib Drif**, mon tuteur d'alternance, pour sa confiance, ses conseils et son accompagnement tout au long de cette expérience.

Une pensée toute particulière va à **ma famille, mes sœurs et ma nièce**, pour leur présence, leurs encouragements et leur soutien tout au long de mon parcours.

Enfin, merci à **la personne qui partage mon quotidien**, pour sa présence, sa patience et son soutien durant cette aventure.

Merci à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de cette belle étape.

Table des matières

Remerciements **2**
Table des matières **7**
Table des figures **9**
Liste des tableaux **12**
Résumé **13**
Abstract **14**
Introduction générale **15**
Plan du rapport 15
Chapitre 1 : Étude préalable **16**
Introduction 16
1.1 Présentation générale du projet 16
1.1.1 Problématique 16
1.1.2 Travail demandé 17
1.1.3 Méthodologie adoptée 18
1.1.3.1 Choix de la méthodologie 18
1.2 Étude de l'existant 19
1.2.1 Solutions existantes 19
1.2.1.1 Description des solutions existantes 19
1.2.1.2 Critique de l'existant 23
1.2.2 Solution proposée 24
1.3 Compétences du titre CDA mobilisées 25
Conclusion 26
Chapitre 2 : Analyse et spécification des besoins **27**
Introduction 27
2.1 Spécification informelle des besoins 27
2.1.1 Définition des acteurs 27
2.1.2 Analyse des besoins fonctionnels 28
Commun (authentification et profil) 28
Membre 28
Chef d'équipe 29
Administrateur 29
2.1.3 Analyse des besoins non fonctionnels 30
2.2 Spécification semi-formelle 31
2.2.1 Diagrammes de cas d'utilisation 31
2.2.2 Diagrammes de séquences (analyse, boîte noire : utilisateur ↔ système) 34
2.2.3 Maquettage des interfaces 36
2.2.4 Planification des sprints 41
Conclusion 42

Chapitre 3 : Conception.....43
Introduction.....43
3.1 Conception globale.....43
3.1.1 Architecture physique.....43
Communication unifiée : un catalogue d'opérations, deux formes de réponse.....45
3.1.2 Architecture logique.....46
3.2 Conception détaillée.....47
3.2.1 Partie métier.....48
Création d'un utilisateur par l'administrateur.....48
Création d'une tâche par le chef d'équipe.....49
Mise à jour de l'avancement par le membre assigné.....50
Diagramme d'activité : mettre à jour une tâche.....52
Diagramme de classes.....52
Schéma relationnel.....54
3.2.2 Client mobile dans l'architecture.....55
Conclusion.....56
Chapitre 4 : Réalisation.....57
Introduction.....57
4.1 Environnement du travail.....57
4.1.1 Configuration matérielle.....57
4.1.2 Configuration logicielle.....57
Outils.....57
Technologies.....58
4.2 Réalisation des sprints.....65
Sprint 1 : Authentification, profil, préférences.....65
Sprint 2 : Espace membre.....67
Sprint 3 : Espace chef d'équipe.....70
Sprint 4 : Espace administrateur.....74
Sprint 5 : Tests, sécurité et intégration continue.....77
Sprint 6 : Application mobile.....77
4.3 Sécurité de l'application.....80
4.3.1 Mots de passe.....80
4.3.2 Session et jeton.....81
4.3.3 Protection CSRF.....82
4.3.4 Contrôle d'accès.....83
4.3.5 Formulaires et données.....87
4.3.6 Limites.....88
4.4 Intégration continue.....88
Conclusion.....89

Chapitre 5 : Déploiement.....90
Introduction.....90
5.1 Choix d'hébergement.....90
5.2 Installation sur le serveur.....90
5.3 Vérification côté web.....93
5.4 Application mobile sur iPhone.....94
5.5 Exploitation et limites.....95
Conclusion du chapitre.....95
Conclusion générale.....96
Perspectives.....96
Netographie.....98

Table des figures

Figure 1.1 – Cycle de vie Scrum 14
Figure 1.2 – Interface de Jira Software 16
Figure 1.3 – Interface de Trello 17
Figure 1.4 – Interface de Monday.com 18
Figure 1.5 – Interface de Linear 18
Figure 1.6 – Canal Slack, support habituel du stand-up écrit 19
Figure 2.1 – Vue d'ensemble des cas d'utilisation de Briefly 30
Figure 2.2 – Cas d'utilisation de l'administrateur 30
Figure 2.3 – Cas d'utilisation du chef d'équipe 31
Figure 2.4 – Cas d'utilisation du membre 31
Figure 2.5 – Authentification 33
Figure 2.6 – Mot de passe oublié 34
Figure 2.7 – Maquette fil de fer de l'écran de connexion (web) 35
Figure 2.8 – Maquette fil de fer du tableau de bord du membre (web) 36
Figure 2.9 – Maquette fil de fer du détail de tâche et de la mise à jour d'avancement (web) 36
Figure 2.10 – Maquette fil de fer de la vue des tâches d'équipe du chef (web) 37
Figure 2.11 – Maquette fil de fer de la vue d'ensemble de l'administrateur (web) 38
Figure 2.12 – Maquettes fil de fer mobiles : connexion, tableau de bord et détail de tâche 38
Figure 3.1 – Architecture physique de Briefly (déploiement 3-tiers) 42
Figure 3.2 – Communication unifiée entre clients et serveur 43
Figure 3.3 – Architecture logique en couches (dépendances descendantes) 46
Figure 3.4 – Diagramme de séquence : création d'un utilisateur par l'administrateur 48
Figure 3.5 – Diagramme de séquence : le chef d'équipe crée une tâche 49
Figure 3.6 – Diagramme de séquence : le membre met à jour l'avancement 50
Figure 3.7 – Diagramme d'activité : mise à jour d'une tâche 51
Figure 3.8 – Diagramme de classes du domaine Briefly 52
Figure 3.9 – Chaîne de communication du client mobile 56
Figure 4.1 – Méthode respond() (app/Http/Controllers/Controller.php) 61
Figure 4.2 – Méthode recordProgress() (app/Services/TaskService.php) 62
Figure 4.3 – Politique updateStatus (app/Policies/TaskPolicy.php) 62
Figure 4.4 – Connexion JSON du client Flutter (mobile/lib/core/api_client.dart) 63
Figure 4.5 – Pourcentage d'avancement dérivé du statut (app/Enums/TaskStatus.php) 64
Figure 4.6 – Validation de la création d'un utilisateur (app/Http/Requests/StoreUserRequest.php) 64

Figure 4.7 – Note de blocage obligatoire si le statut est BLOCKED (app/Http/Requests/UpdateTaskProgressRequest.php) 65
Figure 4.8 – Page de connexion web (/login) 68
Figure 4.9 – Demande de réinitialisation du mot de passe (/forgot-password) 68
Figure 4.10 – Saisie du nouveau mot de passe (/reset-password/{token}) 68
Figure 4.11 – Profil et changement de mot de passe (/profile) 69
Figure 4.12 – Sélecteur de langue et bascule de thème (briefly-theme) 69
Figure 4.13 – Tableau de bord membre : anneau d'avancement et tâches personnelles (/dashboard) 72
Figure 4.14 – Liste des tâches personnelles (/tasks) 72
Figure 4.15 – Détail d'une tâche et formulaire d'avancement (/tasks/{id}) 73
Figure 4.16 – Historique des points d'avancement (/tasks/history) 73
Figure 4.17 – Tableau de bord chef d'équipe : cartes des équipes dirigées (/dashboard) 76
Figure 4.18 – Liste des projets du chef d'équipe (/projects) 77
Figure 4.19 – Détail d'un projet et tâches de l'équipe (/projects/{id}) 77
Figure 4.20 – Création ou édition d'une tâche par le chef d'équipe 78
Figure 4.21 – Suivi des tâches d'équipe avec filtre de statut (/team/tasks) 78
Figure 4.22 – Fiche d'un membre d'équipe (/team/members/{user}) 79
Figure 4.23 – Tableau de bord administrateur : statistiques, achèvement et graphiques (/admin) 82
Figure 4.24 – Gestion des utilisateurs (/admin/users) 82
Figure 4.25 – Gestion des équipes (/admin/teams) 83
Figure 4.26 – Gestion des projets (/admin/projects) 83
Figure 4.27 – Connexion mobile (LoginScreen) 87
Figure 4.28 – Mot de passe oublié et réinitialisation (ForgotPasswordScreen, ResetPasswordScreen) 87
Figure 4.29 – Tableau de bord mobile : anneau d'achèvement et tâches (DashboardScreen) 88
Figure 4.30 – Liste des tâches et historique (TaskListScreen, TaskHistoryScreen) 88
Figure 4.31 – Détail de tâche et saisie d'avancement (TaskDetailScreen) 88
Figure 4.32 – Projets, tâches d'équipe et fiche membre (ProjectListScreen, TeamTasksScreen, TeamMemberScreen) 88
Figure 4.33 – Profil, thème et langue FR/EN (ProfileScreen) 89
Figure 4.34 – Écrans d'administration mobile (AdminDashboardScreen et listes CRUD) 89
Figure 4.35 – Cast hashed et attributs masqués (app/Models/User.php) 92
Figure 4.36 – Émission du jeton Sanctum à la connexion mobile (app/Http/Controllers/LoginController.php) 93
Figure 4.37 – Exemptions CSRF pour le client mobile (app/Http/Middleware/VerifyCsrfToken.php) 94

Figure 4.38 – Middleware EnsureAdmin (app/Http/Middleware/EnsureAdmin.php) 95
Figure 4.39 – Politique view d'un projet (app/Policies/ProjectPolicy.php) 95
Figure 4.40 – Méthodes isAdmin() et isTeamLead() (app/Models/User.php) 96
Figure 4.41 – Vérification que l'assigné appartient à l'équipe (app/Services/TaskService.php) 97
Figure 4.42 – Contrôle local de la note de blocage (mobile/lib/models/models.dart) 97
Figure 5.1 – Site PHP : racine www/briefly/public/ 104
Figure 5.2 – Base lamraoui_briefly 104
Figure 5.3 – Réponse de GET /up 105
Figure 5.4 – Page de connexion 105
Figure 5.5 – Tableau de bord administrateur 106
Figure 5.6 – Connexion mobile (LoginScreen) 107
Figure 5.7 – Tableau de bord mobile (DashboardScreen) 107

Liste des tableaux

Tableau 1.1 – Comparaison des solutions existantes au regard du besoin Briefly 20
Tableau 1.2 – Compétences du titre CDA mises en œuvre dans Briefly 23
Tableau 2.1 – Backlog du produit 39
Tableau 3.1 – Filtres d'accès (conception) 44
Tableau 3.2 – Détails des classes du domaine 53
Tableau 3.3 – Schéma relationnel logique 55
Tableau 4.1 – Configuration matérielle 58
Tableau 4.2 – Outils de développement 59
Tableau 4.3 – Principales technologies 65
Tableau 4.11 – Tables physiques 66
Tableau 4.4 – Tests fonctionnels du sprint 1 70
Tableau 4.5 – Tests fonctionnels du sprint 2 74
Tableau 4.6 – Tests fonctionnels du sprint 3 80
Tableau 4.7 – Tests fonctionnels du sprint 4 84
Tableau 4.8 – Tests fonctionnels du sprint 6 (API JSON consommée par le mobile) 90
Tableau 4.9 – Mesures de sécurité de Briefly 98
Tableau 4.10 – Jobs du workflow d'intégration continue 100
Tableau 5.1 – Environnement de production 102
Tableau 5.2 – Variables d'environnement principales en production 103

Résumé

Ce mémoire présente Briefly, une application web et mobile de suivi de l'avancement des équipes, réalisée dans le cadre du titre professionnel Concepteur développeur d'applications, en alternance chez Drif Consult. Le projet part d'un constat. Le point quotidien (*stand-up*) se tient le plus souvent dans une messagerie, sans lien avec les tâches, et les chefs d'équipe n'ont pas de tableau de bord à jour.

Briefly s'adresse à une organisation fermée, sans inscription publique. L'administrateur crée les comptes, les équipes et les projets. Le chef d'équipe crée et suit les tâches de ses équipes. Le membre rend compte de chaque tâche en trois champs : ce qui a été réalisé, ce qui vient ensuite et ce qui bloque. Une tâche au statut bloqué exige une note de blocage. Les tableaux de bord calculent un taux d'avancement par membre, par équipe et par projet.

Le développement a suivi la méthode Scrum, en six sprints. Le serveur repose sur Laravel 13 (PHP 8.5), Blade, Bootstrap et MySQL. Le client mobile est écrit en Flutter. Un seul ensemble de routes Laravel sert le navigateur en HTML et l'application mobile en JSON, authentifiée par un jeton Sanctum. Les règles d'autorisation et de validation ne sont donc écrites qu'une fois. Une suite de tests PHPUnit vérifie ces règles, notamment les refus d'accès entre rôles. Le serveur a été mis en ligne chez alwaysdata ; l'application mobile installée sur iPhone s'y connecte en HTTPS.

Mots-clés : application web, application mobile, Laravel, Flutter, Sanctum, MySQL, suivi d'avancement, Scrum, UML.

Abstract

This report presents Briefly, a web and mobile application for tracking team progress. It was built for the French professional title *Concepteur développeur d'applications* (application designer and developer) during a work-study placement at Drif Consult. The project starts from a common situation. The daily stand-up usually happens in a chat tool, with no link to the tasks, and team leads have no up-to-date dashboard.

Briefly targets a closed organisation with no public sign-up. The administrator creates accounts, teams and projects. The team lead creates and follows the tasks of their teams. Members report on each task in three fields: what was done, what comes next and what is blocking. A task marked as blocked requires a blocker note. Dashboards compute a completion rate per member, per team and per project.

Development followed Scrum over six sprints. The server is built with Laravel 13 (PHP 8.5), Blade, Bootstrap and MySQL. The mobile client is written in Flutter. A single set of Laravel routes serves HTML to the browser and JSON to the mobile app, which authenticates with a Sanctum token. Authorisation and validation rules are therefore written only once. A PHPUnit test suite checks these rules, including access refusals between roles. The Laravel application is hosted on alwaysdata; the Flutter app on a physical iPhone connects to that HTTPS server.

Keywords: web application, mobile application, Laravel, Flutter, Sanctum, MySQL, progress tracking, Scrum, UML.

Introduction générale

Les équipes internes doivent savoir qui travaille sur quoi, où en sont les tâches et ce qui bloque. Cette information se disperse souvent entre messageries, courriels et réunions. Il devient difficile d'en tirer un état des lieux fiable.

Le point quotidien oral ou écrit ne tient pas longtemps. Les blocages arrivent tard, parfois seulement en réunion. Les chefs d'équipe n'ont pas de tableau de bord à jour. Les outils du marché couvrent mal ce besoin. Certains privilégient la conversation. D'autres imposent une gestion de projet trop lourde pour un usage interne. Le chapitre 1 développe ce constat et compare les solutions existantes.

Nous avons conçu et réalisé Briefly, une application interne de suivi d'avancement. Elle existe en client web (Laravel) et en client mobile (Flutter). Les comptes sont créés par l'administrateur. Il n'y a pas d'inscription publique. Les utilisateurs sont administrateur, chef d'équipe ou membre. Les chapitres suivants détaillent les besoins, l'architecture et les écrans.

Plan du rapport

Le premier chapitre présente le projet, la problématique et une critique de l'existant. Il justifie le besoin d'un outil interne dédié et situe le projet par rapport aux compétences du titre CDA.

Le deuxième chapitre analyse et spécifie les besoins. À partir des acteurs, nous formalisons les exigences fonctionnelles et non fonctionnelles, puis les cas d'utilisation et les maquettes fil de fer des écrans principaux.

Le troisième chapitre décrit la conception : architectures côté serveur et côté clients, diagrammes utilisés avant le développement et schéma relationnel de la base.

Le quatrième chapitre décrit la réalisation : environnement de travail, sprints, interfaces web et mobile, puis sécurité de l'application et intégration continue.

Le cinquième chapitre présente la mise en ligne chez alwaysdata, les contrôles sur le web et le branchement de l'application mobile sur ce serveur.

La conclusion générale dresse le bilan, les apports du projet et les pistes d'évolution.

Chapitre 1 : Étude préalable

Introduction

Ce chapitre présente Briefly, la problématique, le travail demandé et la démarche de réalisation. Il examine ensuite les outils déjà utilisés par les équipes. L'étude de l'existant débouche sur une solution de principe, décrite au niveau fonctionnel. Le chapitre se termine par les compétences du titre CDA que le projet mobilise.

1.1 Présentation générale du projet

Briefly est un projet de titre professionnel *Concepteur développeur d'applications* (CDA, RNCP 37873, niveau 6). Il s'adresse à des équipes qui ont besoin d'une visibilité quotidienne sur le travail en cours, sans plateforme de gestion de projet trop lourde. L'application vise des organisations fermées : agences, petites structures, équipes internes. Un administrateur y crée les comptes.

Briefly est une application web et mobile légère, centrée sur les tâches d'un projet. Elle distingue trois rôles (administrateur, chef d'équipe, membre) et un compte rendu d'avancement structuré. Le besoin qui la justifie est celui du *stand-up* quotidien : savoir ce qui a été réalisé, ce qui vient ensuite et ce qui bloque, puis en tirer un tableau de bord d'avancement. Le chapitre 2 précise les prérogatives de chaque rôle.

1.1.1 Problématique

Un écart persiste entre le rituel agile du point quotidien et les outils que les équipes emploient pour le tenir. Dans de nombreuses organisations, le *daily stand-up* se tient en présentiel, en visio ou, plus souvent, sous la forme d'un message Slack ou Microsoft Teams. C'est le principal moment où chacun dit ce qu'il a accompli, ce qu'il compte faire et ce qui l'empêche d'avancer. Le rituel est oral ou textuel, peu structuré et rapidement oublié. Les informations se dispersent dans un fil de conversation. Les blocages ne sont pas distingués des simples commentaires. Le chef d'équipe n'a pas de tableau de bord stable pour mesurer l'avancement d'une équipe, d'un membre ou d'un projet.

Les forges et les tableaux Kanban du marché répondent à d'autres besoins : tracer des tickets, paramétrer des workflows, visualiser un flux de cartes. Le *stand-up* n'y est généralement pas

le cœur du produit. Il reste un usage parallèle, souvent improvisé dans la messagerie. La comparaison nominative de ces outils fait l'objet de la section 1.2.

Les équipes ont besoin d'un signal quotidien simple, comparable d'un jour à l'autre, consultable depuis un navigateur comme depuis un téléphone. Les outils du marché surchargent ce besoin, ou le laissent à l'état de message libre. Un blocage signalé oralement n'est pas relié à une tâche. Un taux d'avancement n'est pas calculé. L'administrateur d'une petite organisation n'a pas de vue d'ensemble fiable.

Comment offrir à une organisation fermée un suivi d'avancement quotidien, structuré autour des tâches, accessible sur le web et sur mobile, avec des rôles nets ? Briefly répond à cette question. Il n'existe pas d'entité « brief » quotidienne distincte des tâches. Le *stand-up* est le problème métier à résoudre. La tâche, enrichie de champs d'avancement, en est le support.

1.1.2 Travail demandé

Pour ce titre CDA, nous concevons et réalisons Briefly, application de suivi d'avancement pour des équipes internes. Elle existe en client web (Laravel) et en client mobile (Flutter), sur un même backend. Les choix d'architecture et de technologies sont détaillés aux chapitres 3 et 4.

L'organisation cible est fermée : il n'y a pas d'inscription publique, l'administrateur crée les comptes. Trois rôles structurent l'usage : administrateur, chef d'équipe et membre. Le chapitre 2 spécifie leurs prérogatives, y compris le fait que le chef d'équipe ne crée pas les projets. Un utilisateur peut appartenir à plusieurs équipes, un chef peut en diriger plusieurs, un projet peut en impliquer plusieurs.

Les tâches sont le cœur opérationnel. Elles appartiennent à un projet, sont assignées à un utilisateur et portent un statut (à faire, en cours, bloqué, terminé). Elles portent aussi un compte rendu en trois champs (réalisé, à venir, note de blocage) qui reprend les trois questions du *stand-up* en les rattachant à une tâche. Lorsque le statut est « bloqué », la note de blocage est obligatoire. Il n'existe pas d'entité « brief » quotidienne distincte.

Le livrable est une application authentifiée, rôle par rôle, dans laquelle le web et le mobile parlent le même langage métier. Une maquette isolée n'est pas le livrable demandé. L'interface est prévue en français et en anglais, en thème clair et en thème sombre.

1.1.3 Méthodologie adoptée

Un projet de titre CDA mêle analyse, conception et réalisation itérative. Les besoins se précisent au contact des incréments. Nous produisons des incréments vérifiables et nous restons ouverts à l'ajustement.

1.1.3.1 Choix de la méthodologie

Nous avons retenu une démarche agile, formalisée par Scrum. Le cycle en V, encore enseigné comme référence de planification, suppose un recueil de besoins relativement stable, une conception détaillée préalable et une réalisation linéaire. Briefly est un produit d'usage quotidien. Les écrans de tableau de bord, la formulation des champs d'avancement, le cloisonnement des rôles et le confort mobile ne se valident vraiment qu'au contact d'un incrément utilisable. Une spécification figée trop tôt aurait figé aussi des hypothèses fragiles, notamment sur le périmètre du *stand-up*. Nous avons traité ce périmètre comme un problème d'usage, pas comme une table métier autonome.

Scrum organise le travail. Dans le contexte d'un mémoire de titre professionnel, les rôles de *Product Owner*, de *Scrum Master* et d'équipe de développement sont nécessairement concentrés. Nous les distinguons toutefois conceptuellement, afin de ne pas confondre priorisation du besoin, animation de la méthode et production technique.

Nous organisons le travail en sprints de deux à quatre semaines. Chacun débouche sur un incrément potentiellement livrable. Le carnet de produit, le carnet de sprint et l'incrément évitent l'accumulation de fonctionnalités sans priorisation, autant que la livraison unique, trop tardive. La figure 1.1 résume ce cycle. Le tableau 2.1 (chapitre 2) présente le backlog détaillé des six sprints. La mêlée du processus de développement n'est pas le *stand-up* que l'application outille : l'une synchronise l'équipe projet, l'autre est le besoin métier.

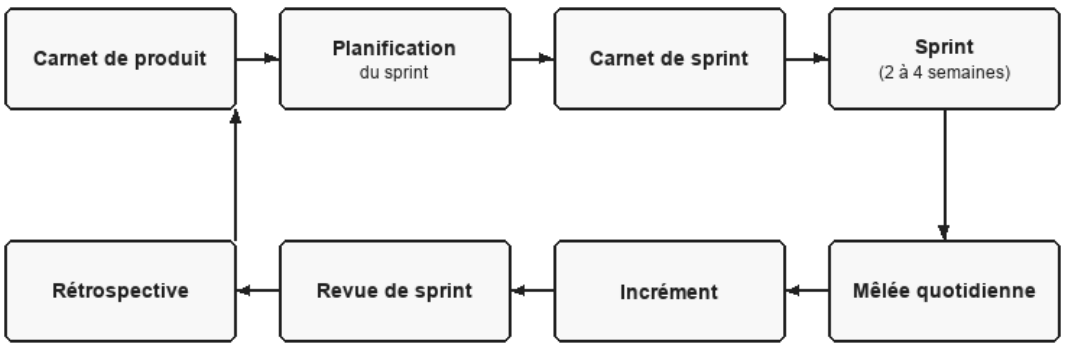


Figure 1.1 – Cycle de vie Scrum

Chaque incrément de Briefly a été conçu comme une boucle : planifier, réaliser, inspecter, adapter.

1.2 Étude de l'existant

Nous examinons d'abord les solutions déjà utilisées pour suivre le travail et animer le point quotidien, puis nous formulons la solution proposée. Cette section établit pourquoi l'existant ne suffit pas et dans quelle direction Briefly s'oriente.

1.2.1 Solutions existantes

Nous avons retenu cinq références représentatives parmi les outils et les pratiques qui occupent aujourd'hui le terrain du suivi d'équipe : une forge logicielle (Jira), un couple d'outils visuels de gestion du travail (Trello, et à titre de comparaison Asana), une plateforme de type « Work OS » (Monday.com), un suivi de tickets moderne (Linear), et enfin la pratique dominante du *stand-up* dans Slack ou Microsoft Teams. Ces références couvrent à la fois les produits spécialisés et le rituel informel qui, dans les faits, les complète ou les remplace.

1.2.1.1 Description des solutions existantes

Pour chaque solution, nous indiquons la fonction, le public visé et les limites au regard du besoin de Briefly.

Jira

Jira, édité par Atlassian, est une plateforme de suivi de tickets et de projets, très répandue dans les équipes logicielles. Elle permet de créer des issues, de les organiser en sprints, de définir des workflows, des tableaux, des filtres et des rapports. Les rôles y sont riches : administrateurs de site, administrateurs de projet, développeurs, observateurs. Le paramétrage peut coller à des organisations de grande taille. Une application mobile existe, de même qu'une intégration poussée avec Confluence, Bitbucket ou d'autres outils Atlassian.

Le public visé est d'abord celui des équipes de développement et des directions techniques qui ont besoin d'une traçabilité fine : historique d'une issue, liens entre tickets, estimation, jalons. Jira peut être la source de vérité pour un département entier. En contrepartie, la prise en main est longue, l'interface est dense, et le coût, calculé par utilisateur, devient rapidement dissuasif pour une petite structure. Le *stand-up* n'y est pas un objet de premier plan : on peut filtrer les tickets « bloqués » ou ajouter un commentaire, mais rien n'oblige un membre à renseigner, sur chaque tâche, ce qui a été fait, ce qui suit et ce qui bloque. Le tableau de bord

d’avancement existe sous forme de gadgets et de rapports, au prix d’une configuration que peu de petites équipes maîtrisent. Jira résout le suivi de tickets. Il n’outille pas, nativement, le point quotidien léger.

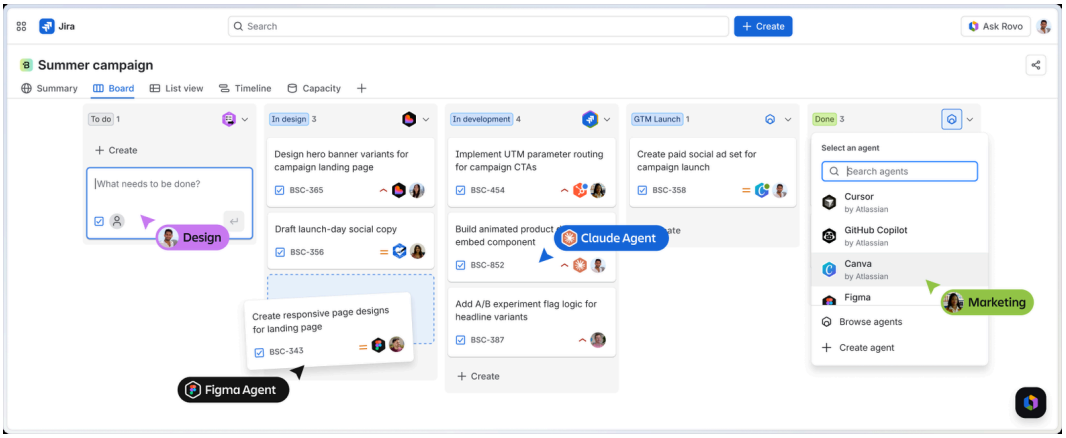


Figure 1.2 – Interface de Jira Software

Trello et Asana

Trello, également dans l’écosystème Atlassian, propose un tableau Kanban simple : listes, cartes, étiquettes, dates et pièces jointes. Asana, souvent cité comme alternative, offre une vue un peu plus structurée (listes, chronologie, charges) tout en restant dans la famille des outils visuels de gestion du travail. L’un et l’autre s’adressent à des équipes qui veulent voir le travail avancer de colonne en colonne, sans investir dans une forge. Le mobile est de bonne qualité, le coût d’entrée est bas, voire nul sur les offres limitées, et l’adoption est rapide.

Trello est visuellement clair. Il manque de structure organisationnelle et d’avancement. Il ne distingue pas nativement un administrateur d’organisation, un chef d’équipe et un membre avec des droits métier différents : on invite des collaborateurs à un tableau. Les blocages se réduisent souvent à une étiquette rouge ou à un commentaire. Trello n’impose pas les champs « réalisé / à venir / note de blocage » et n’offre pas de tableau de bord de taux d’avancement comparable d’une équipe à l’autre. Asana va plus loin sur les responsabilités et les échéances. L’outil coordonne des tâches sans outiller le *stand-up*. Dans les deux cas, l’information quotidienne continue souvent de vivre dans la messagerie.

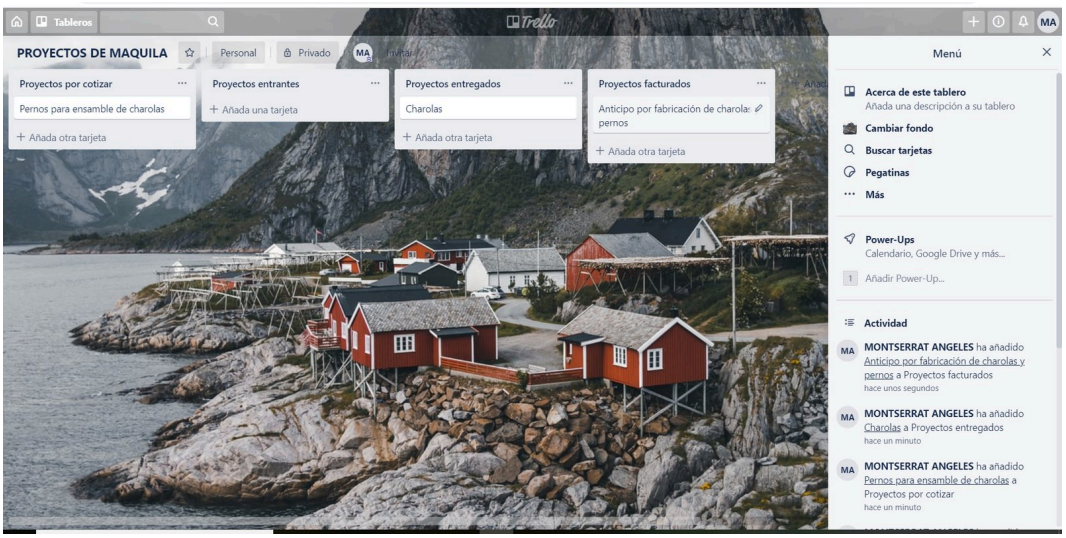


Figure 1.3 – Interface de Trello

Monday.com

Monday.com se présente comme un « Work OS ». Il propose des tableaux hautement paramétrables, des vues multiples (kanban, chronologie, charge), des automatisations et des tableaux de bord visuels. L'outil s'adresse à des équipes opérationnelles variées (marketing, ressources humaines, production) autant, sinon plus, qu'aux développeurs. La personnalisation et les synthèses graphiques expliquent une partie de l'adoption. Une application mobile accompagne le produit web.

Cette puissance pose problème pour le besoin qui nous occupe. Monday.com demande un travail de conception d'espace de travail avant de produire de la valeur : colonnes, formules, automatisations, tableaux de bord. Le coût par siège est élevé. Les rôles existent, mais ils relèvent davantage de l'administration de la plateforme que d'un modèle métier simple à trois profils. On peut, à force de paramétrage, approximer un suivi de blocages et un taux d'avancement. Rien n'en fait toutefois le cœur du produit. Pour une organisation qui veut seulement que chaque membre tienne à jour trois informations sur sa tâche, Monday.com est disproportionné.

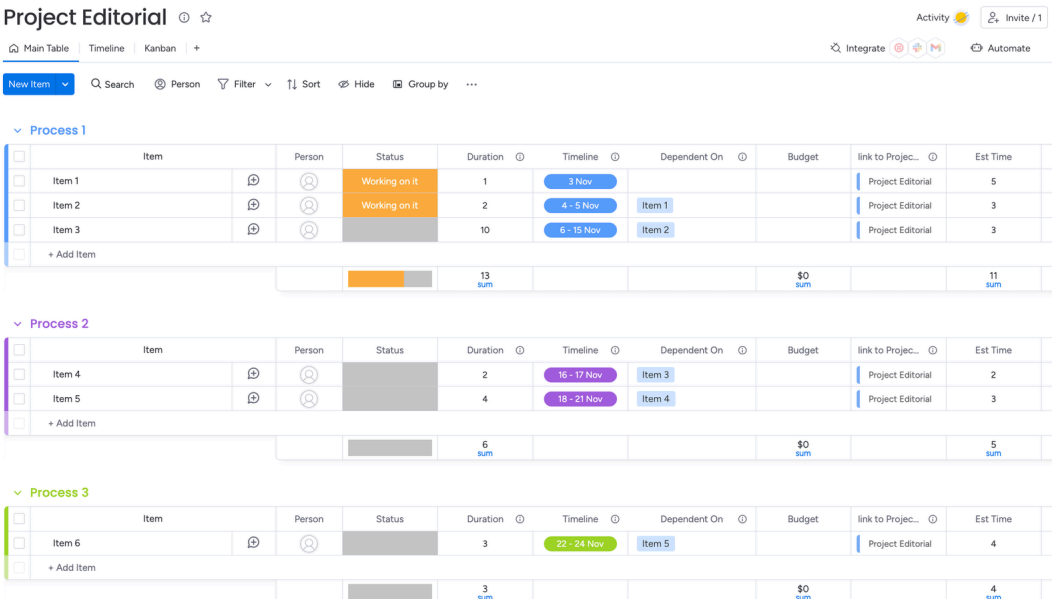


Figure 1.4 – Interface de Monday.com

Linear

Linear est un suivi de tickets moderne, rapide, au design soigné, pensé pour les équipes produit et ingénierie des start-up. Il gère les issues, les cycles, les projets, les vues clavier-first et une application mobile. Le public visé est clairement logiciel. Linear assume d'être un outil de *issue tracking*, à la différence d'un « Work OS » généraliste. La vitesse, la clarté et les opinions assumées en font une référence souvent opposée à Jira.

Linear reste centré sur le ticket et le cycle. Les statuts sont flexibles, les vues sont excellentes, mais les trois questions du *stand-up* n'y sont pas des champs de premier plan, et encore moins une obligation lorsque l'on signale un blocage. Le modèle de rôles et d'équipes est celui d'un workspace d'ingénierie, pas celui d'une petite organisation fermée où seul l'administrateur crée les comptes et les projets. L'offre est payante au-delà d'un usage restreint.

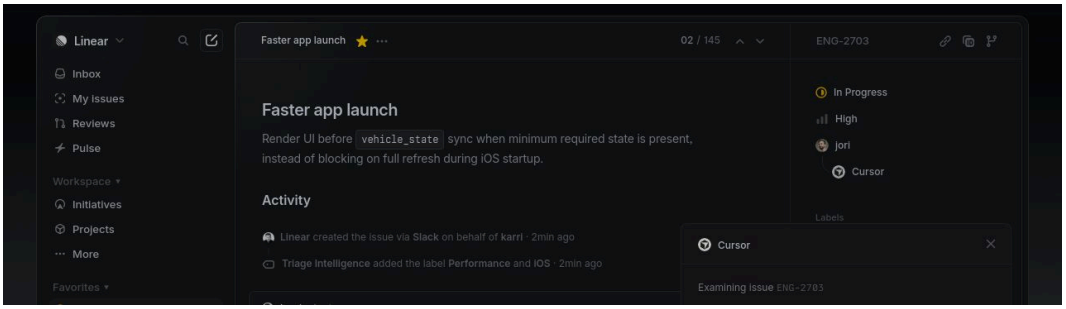


Figure 1.5 – Interface de Linear

Le stand-up dans Slack ou Microsoft Teams

La pratique la plus répandue est un message, parfois automatisé par un robot, dans un canal d'équipe Slack ou Microsoft Teams. Chaque matin, ou à heure fixe, les membres collent un texte du type : « hier… aujourd'hui… bloqué par… ». Des applications du magasin Slack proposent un rappel et un recueil de réponses. Cette solution s'adresse à toute équipe qui a déjà sa messagerie comme place centrale. Elle est immédiate, gratuite au regard du coût de la suite collaborative, et disponible sur mobile de façon native, puisque c'est l'application de chat elle-même.

L'outil est trop pauvre pour le besoin. Le *stand-up* n'est relié à aucune tâche, à aucun statut, à aucun projet. Un blocage écrit dans un canal disparaît sous les messages suivants. Le chef d'équipe ne peut pas ouvrir un tableau de bord de taux d'avancement. Il ne filtre pas les tâches bloquées et ne compare pas les équipes. Il n'y a pas de rôles métier au-delà des permissions du canal. Rien n'empêche un membre d'omettre le blocage, de coller un message vague, ou de cesser de répondre. Briefly remplace cette pratique, utile mais volatile, par un avancement structuré, persistant et rattaché au travail réel.

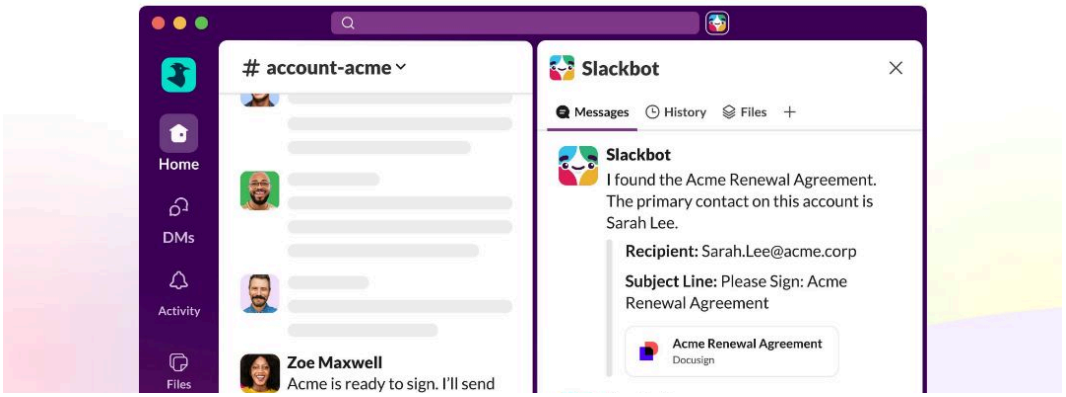


Figure 1.6 – Canal Slack, support habituel du stand-up écrit

1.2.1.2 Critique de l'existant

Nous comparons ces solutions au besoin posé, sous forme de tableau (tableau 1.1). Les critères portent sur la nature de l'outil, le traitement des rôles, l'existence d'un client mobile natif, la structuration des blocages, la présence d'un tableau de bord d'avancement, la complexité de prise en main et le coût.

Tableau 1.1 – Comparaison des solutions existantes au regard du besoin Briefly

Critère	Jira	Trello / Asana	Monday.com	Linear	Slack / Teams
Type	Forge, tickets	Kanban / listes	Work OS	Tickets	Messagerie
Rôles	Nombreux (projet, site)	Tableau, peu de rôles métier	Plateforme, paramétrage lourd	Espace d'ingénierie	Canal, pas de rôle métier
Mobile	Oui, secondaire	Oui	Oui	Oui, limité	Chat seulement
Blocages	Étiquette ou commentaire	Étiquette ou commentaire	Colonne personnalisée	Statut ou commentaire	Texte libre
Avancement	Rapports, config. lourde	Faible en natif	Tableaux après paramétrage	Vues d'équipe, peu de taux	Aucun
Prise en main	Élevée	Faible à moyenne	Élevée	Moyenne	Très faible
Coût	Élevé / utilisateur	Faible / freemium	Élevé	Payant hors usage restreint	Inclus dans la suite

Les outils spécialisés (Jira, Monday.com, Linear) savent tracer le travail, parfois avec une grande sophistication. Ils n'imposent pas le compte rendu quotidien en trois champs et ils pèsent par leur complexité ou leur prix. Les outils légers (Trello, Asana) et surtout la messagerie rendent le rituel facile. Ils n'offrent pas de rôles nets, pas de blocage obligatoire, pas de tableau de bord d'avancement comparable. Aucune de ces solutions ne combine, nativement, une organisation fermée à trois rôles, un avancement « réalisé / à venir / note de blocage » rattaché à la tâche, un statut bloqué exigeant une note, des tableaux de bord de taux d'avancement, et un couple web et mobile s'appuyant sur le même backend.

1.2.2 Solution proposée

Briefly est une application légère de suivi d'avancement, disponible sur le web et sur mobile, destinée à une organisation fermée et à un usage quotidien. Elle se concentre sur un geste : tenir à jour, pour chaque tâche, un avancement lisible par l'équipe, et en dériver des tableaux de bord. Les workflows infinis d'une forge et le remplacement d'une messagerie restent hors périmètre.

Trois rôles (administrateur, chef d'équipe, membre) structurent une organisation fermée ; le détail des droits et des liens équipes / projets est au chapitre 2. L'avancement tient en trois champs sur la tâche (réalisé, à venir, note de blocage) ; les tableaux de bord en dérivent par rôle. Un backend Laravel et un client Flutter partagent les mêmes règles (chapitres 3 et 4).

1.3 Compétences du titre CDA mobilisées

Le référentiel du titre CDA (RNCP 37873) regroupe onze compétences en trois activités types. Le tableau 1.2 indique, pour chacune, ce que Briefly met en œuvre et la partie du mémoire qui le décrit.

Tableau 1.2 – Compétences du titre CDA mises en œuvre dans Briefly

Activité type	Compétence	Mise en œuvre dans Briefly
1. Développer une application sécurisée	Installer et configurer son environnement de travail en fonction du projet	Poste macOS, XAMPP (MySQL), SDK Flutter et Xcode (§4.1)
	Développer des interfaces utilisateur	Vues Blade et Bootstrap 5.3 pour le web, écrans Flutter pour le mobile, en français et en anglais (§4.2)
	Développer des composants métier	Services métier et politiques d'accès (chapitre 4)
	Contribuer à la gestion d'un projet informatique	Démarche Scrum en six sprints, backlog produit (§1.1.3, tableau 2.1)
2. Concevoir et développer une application sécurisée organisée en couches	Analyser les besoins et maquetter une application	Acteurs, besoins fonctionnels et non fonctionnels, cas d'utilisation, séquences d'analyse et maquettes fil de fer Figma (chapitre 2, §2.2.3)
	Définir l'architecture logicielle d'une application	Architecture 3-tiers, communication unifiée et découpage en couches (§3.1). Pages web et réponses pour le mobile (chapitre 4)
	Concevoir et mettre en place une base de données relationnelle	Diagramme de classes et schéma logique (§3.2.1). Tables physiques (tableau 4.11)
	Développer des composants d'accès aux données SQL	Modèles et relations sur MySQL (chapitre 4)
3. Préparer le déploiement d'une application sécurisée	Préparer et exécuter les plans de tests d'une application	Tests par sprint (tableaux 4.4 à 4.8), consolidation au sprint 5 (§4.3, §4.4)
	Préparer et documenter le déploiement d'une application	Migrations, mise en ligne alwaysdata, vérifications web et mobile (chapitre 5)
	Contribuer à la mise en production dans une démarche DevOps	Workflow GitHub Actions (§4.4) ; déploiement manuel au chapitre 5

Conclusion

Le tableau 1.1 montre qu'aucune solution du marché ne relie le *stand-up* quotidien, trois rôles fermés et un couple web / mobile sur le même backend. Briefly cible ce périmètre. Le chapitre 2 en déduit les exigences et le backlog ; les chapitres suivants décrivent la conception, la réalisation et la mise en ligne.

Chapitre 2 : Analyse et spécification des besoins

Introduction

Le chapitre 1 a décrit Briefly comme une application interne de suivi d'avancement, fermée à l'inscription publique, destinée à des équipes déjà constituées. Nous en tirons ici des exigences exploitables, d'abord informelles, puis semi-formelles.

La section 2.1 nomme les acteurs, les exigences fonctionnelles par rôle, puis les exigences non fonctionnelles. La section 2.2 propose les diagrammes UML, les maquettes fil de fer et le backlog produit (tableau 2.1).

2.1 Spécification informelle des besoins

Nous nommons d'abord les utilisateurs du système et relions chaque exigence à un rôle. L'accès est nominatif, les comptes sont créés par l'administration, et les droits découlent de l'appartenance aux équipes autant que du rôle.

2.1.1 Définition des acteurs

Nous retenons trois acteurs. Ils correspondent aux rôles métier distingués par Briefly. Aucun visiteur anonyme n'a de parcours utile au-delà de l'authentification et de la réinitialisation du mot de passe. Il n'existe pas d'auto-inscription.

L'administrateur crée et maintient les utilisateurs, les équipes et les projets. Il consulte un tableau de bord global. Il ne gère pas de tâches personnelles. S'il ouvre la liste des tâches, leur historique, leur mise à jour ou le tableau de bord membre, le système le ramène vers l'administration. Les parcours du chef d'équipe lui sont refusés.

Le chef d'équipe est celui qui dirige une équipe. Un même utilisateur peut diriger plusieurs équipes. Chaque équipe n'a qu'un chef à un instant donné. Le chef d'équipe peut se voir assigner des tâches et en mettre à jour l'avancement, comme un membre. Il ne crée ni les projets, ni les utilisateurs. Ces opérations restent du ressort de l'administrateur.

Le membre appartient à une ou plusieurs équipes. Il consulte son suivi, rend compte de l'avancement des tâches qui lui sont assignées et consulte l'historique de ses points d'avancement. Il ne crée pas de tâches pour autrui et ne pilote pas l'équipe. Il ne met à jour que les tâches dont il est l'assigné.

Le chef d'équipe étend le membre pour le suivi d'équipe. Les prérogatives de l'administrateur restent distinctes. L'administrateur n'emprunte pas le parcours d'exécution des tâches.

2.1.2 Analyse des besoins fonctionnels

Nous commençons par les fonctions d'authentification et de profil, communes aux trois rôles, puis nous détaillons celles du membre, du chef d'équipe et de l'administrateur.

Commun (authentification et profil)

Tous les utilisateurs authentifiables peuvent :

- Se connecter au moyen d'une adresse de courriel et d'un mot de passe.
- Cocher, sur le client web, l'option « se souvenir de moi », pour prolonger la session.
- Demander un lien de réinitialisation en cas de mot de passe oublié, puis définir un nouveau mot de passe.
- Se déconnecter.
- Modifier leur nom et leur adresse de courriel.
- Changer leur mot de passe.
- Choisir la langue de l'interface, entre le français et l'anglais.
- Basculer entre un thème clair et un thème sombre.

Sans compte créé par un administrateur, ces parcours restent inaccessibles. Le système ne propose aucun formulaire d'inscription publique.

Membre

Le membre accède à un tableau de bord et aux tâches qui lui sont assignées. Il peut :

- Consulter un tableau de bord personnel indiquant le taux d'avancement (tâches terminées / total) ainsi que la liste de ses tâches, chacune assortie d'un pourcentage dérivé du statut : à faire 0 %, en cours 60 %, bloqué 30 %, terminé 100 %.
- Lister ses propres tâches, regroupées par statut.
- Consulter le détail d'une tâche.

- Mettre à jour l'avancement d'une tâche qui lui est assignée, en renseignant le statut, le texte « réalisé », le texte « à venir » et, le cas échéant, la note de blocage. Cette note est obligatoire lorsque le statut choisi est « bloqué ».
- Consulter l'historique de ses points d'avancement, sous la forme d'une liste paginée.

Le membre ne modifie pas le titre, la description ou l'assigné d'une tâche. Ces attributs relèvent du chef d'équipe. Le geste métier du membre est le *point d'avancement* structuré.

Chef d'équipe

Le chef d'équipe hérite des besoins du membre pour ses propres tâches. Il pilote uniquement les équipes dont il est le chef. Il peut :

- Consulter, sur son tableau de bord, chaque équipe dirigée, avec les statistiques suivantes : total des tâches, tâches en cours, tâches bloquées, tâches terminées dans la semaine, taux d'avancement, ainsi que les dernières tâches bloquées. Il n'y a pas de carte dédiée aux tâches terminées hors de ce décompte hebdomadaire.
- Lister les projets rattachés aux équipes qu'il gère.
- Consulter le détail d'un projet et y créer une tâche (titre, description, assigné, statut), l'assigné devant appartenir à l'équipe de contexte.
- Modifier une tâche existante relevant des équipes qu'il dirige.
- Consulter les tâches d'équipe, avec un filtre par statut.
- Consulter la fiche d'un membre (tâches et mises à jour récentes), uniquement si ce membre partage une équipe gérée par le chef.

Le chef d'équipe ne crée ni projets ni utilisateurs. Il n'administre pas le référentiel. Il organise le travail à l'intérieur des projets déjà rattachés à ses équipes et lit l'avancement de ses membres.

Administrateur

L'administrateur gère la structure (utilisateurs, équipes, projets) et consulte une vue d'ensemble. Il peut :

- Consulter un tableau de bord administrateur : effectifs (équipes, projets, tâches), taux d'avancement global, graphique de répartition par statut, listes de taux d'avancement par équipe et par projet.

- Gérer les utilisateurs non administrateurs : création, consultation et mise à jour (nom, intitulé de poste, courriel et mot de passe à la création, identifiants d'équipes avec un minimum d'une équipe, identifiants de chef d'équipe nécessairement inclus dans les équipes sélectionnées). Un nouvel utilisateur est créé en tant que membre. Son rôle est ensuite synchronisé à partir des drapeaux de chef d'équipe. Le compte administrateur n'apparaît pas dans cette liste. Le formulaire ne permet pas de le réaffecter.
- Gérer les équipes : création, consultation et mise à jour (nom, chef, membres, projets rattachés).
- Gérer les projets et y rattacher des équipes : création, consultation et mise à jour.

L'administrateur ne gère pas de tâches personnelles. S'il emprunte les routes d'exécution, le système le ramène vers son tableau de bord.

2.1.3 Analyse des besoins non fonctionnels

Nous retenons les besoins non fonctionnels suivants, qui contraignent autant la conception que la réalisation.

Sur le plan de la sécurité, Briefly est une application fermée. L'authentification web s'appuie sur une session. L'authentification mobile s'appuie sur un jeton d'accès. L'administration, le pilotage d'équipe et l'exécution des tâches sont cloisonnés par rôle. Au-delà du rôle, une règle précise qui peut voir, créer ou modifier une ressource. Un chef ne voit un projet que s'il est rattaché à l'une de ses équipes. Un membre ne met à jour que ses propres tâches. Les formulaires web sont protégés contre la falsification de requête.

L'information d'avancement est présentée par regroupement par statut, pourcentages dérivés, tableaux de bord différenciés selon le rôle, filtre des tâches d'équipe et fiche membre. Les parcours d'erreur (identifiants invalides, note de blocage manquante) renvoient l'utilisateur vers le formulaire avec un message explicite.

L'interface est disponible en français et en anglais. Côté web, le choix de langue est mémorisé pour l'utilisateur et s'applique aux libellés comme aux messages de validation. Côté mobile, la langue est locale à l'appareil. Elle ne change pas la langue retenue côté serveur.

Les règles métier (qui peut assigner, qui peut mettre à jour un statut, qui peut voir une fiche membre) vivent côté serveur. Un nouveau client ne doit pas les réécrire. Un même utilisateur peut diriger plusieurs équipes sans qu'un quatrième acteur soit nécessaire.

Les listes d'historique sont paginées. Les tableaux de bord s'appuient sur des agrégats (comptages, taux) et n'y versent pas l'historique complet. Ces choix visent un temps de réponse acceptable pour un usage quotidien, y compris lorsque plusieurs équipes et projets coexistent.

En tant qu'outil interne de suivi, l'application doit rester joignable pendant les plages de travail des équipes. L'indisponibilité de l'authentification ou des tableaux de bord priverait les membres de leur point d'avancement et les chefs de leur vue d'ensemble.

Briefly s'adresse à un client web et à un client mobile. Les deux clients exposent les mêmes droits et les mêmes invariants (assigné membre de l'équipe de contexte, note de blocage obligatoire, redirection de l'administrateur, etc.). La logique métier est spécifiée une fois, puis servie aux deux canaux.

2.2 Spécification semi-formelle

Nous poursuivons en UML d'analyse. Les diagrammes de cas d'utilisation donnent une vue d'ensemble par acteur. Les diagrammes de séquences décrivent, en boîte noire, l'authentification et le mot de passe oublié. Les maquettes fil de fer donnent la disposition des écrans principaux. Le backlog produit traduit ces exigences en sprints.

2.2.1 Diagrammes de cas d'utilisation

La figure 2.1 donne une vue d'ensemble des trois acteurs et des cas principaux. Les figures 2.2 à 2.4 détaillent ensuite chaque rôle. Les relations « include » et « extend » n'apparaissent que lorsqu'elles correspondent à une règle réelle du système.

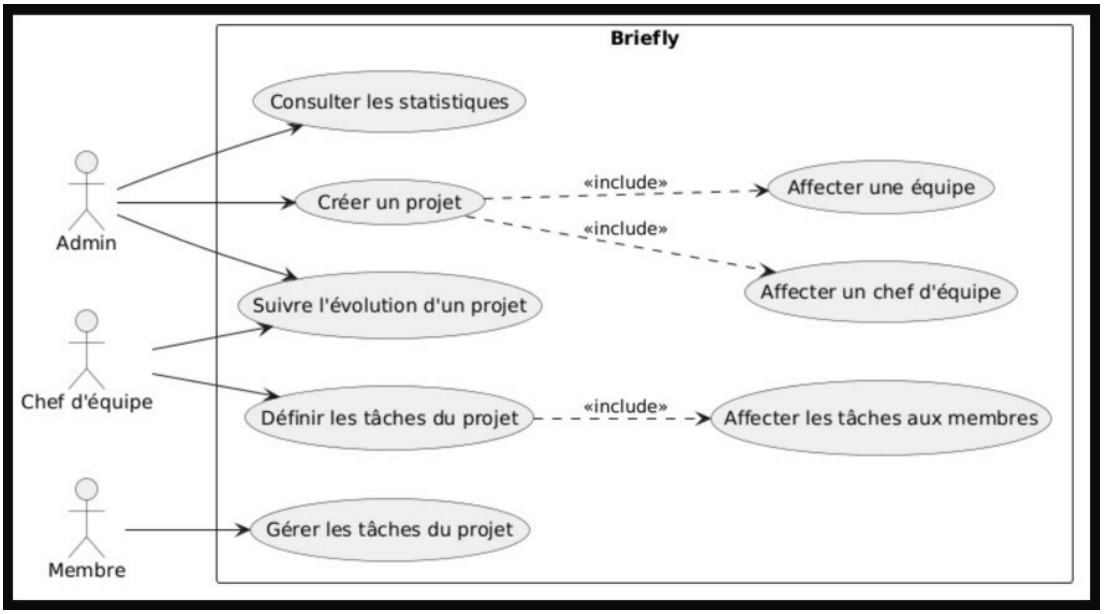


Figure 2.1 – Vue d'ensemble des cas d'utilisation de Briefly

L'administrateur s'authentifie, gère son profil, puis opère exclusivement dans l'administration. Un utilisateur ne peut pas être créé sans au moins une équipe. « Affecter au moins une équipe » est donc un « include » de « Gérer les utilisateurs », et non un « extend ». La création fixe aussi le rôle selon les équipes qu'il dirige. La gestion des projets inclut le rattachement d'équipes.

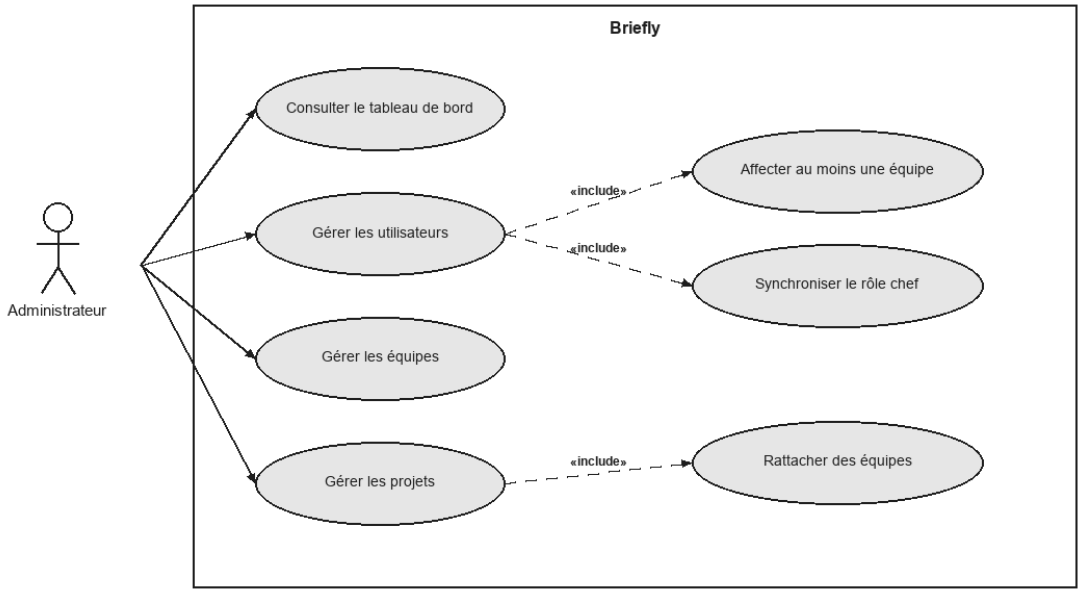


Figure 2.2 – Cas d'utilisation de l'administrateur

Le chef d'équipe hérite des cas du membre : tableau de bord personnel, tâches assignées, points d'avancement. Les cas propres au pilotage s'y ajoutent. La création d'une tâche inclut

la vérification que l'assigné appartient à l'équipe de contexte. La consultation d'une fiche membre inclut la vérification d'une équipe gérée en commun.

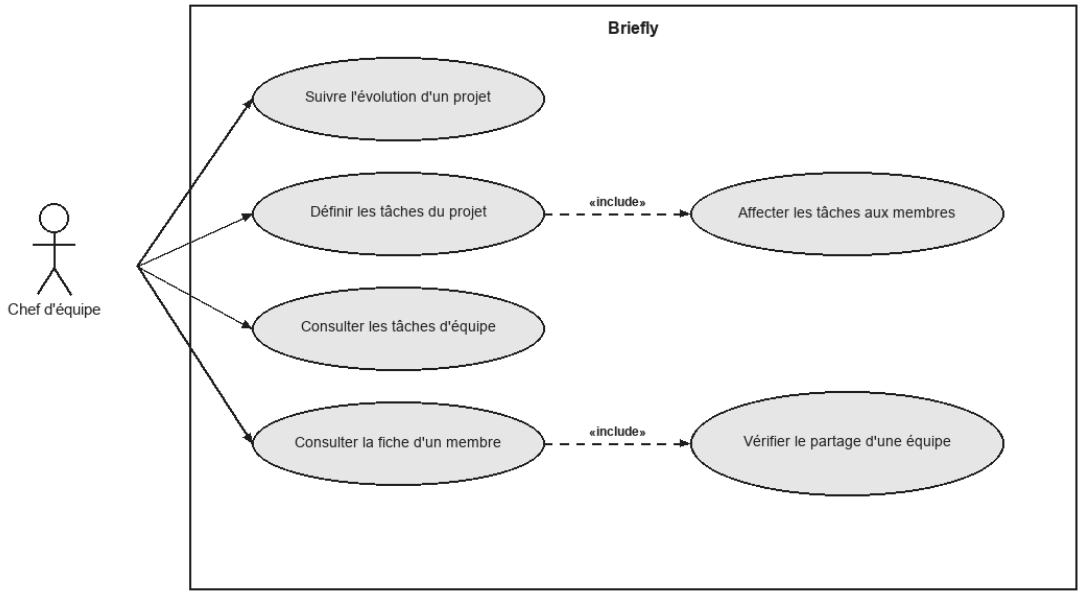


Figure 2.3 – Cas d'utilisation du chef d'équipe

Le membre s'authentifie, gère son profil, puis travaille sur son suivi personnel. « Valider la note de blocage » étend « Mettre à jour l'avancement ». Cette validation n'a lieu que lorsque le statut choisi est « bloqué ». Un « include » aurait signifié qu'elle s'exécute à chaque mise à jour.

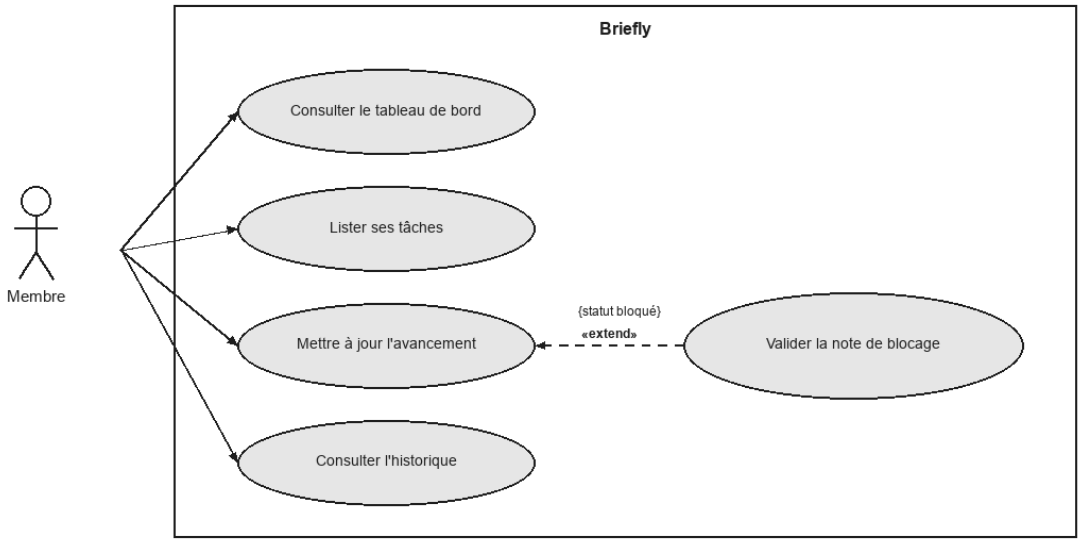


Figure 2.4 – Cas d'utilisation du membre

Les figures n'incluent pas la création de projet par le chef d'équipe, l'auto-inscription, ni la gestion de tâches personnelles par l'administrateur.

2.2.2 Diagrammes de séquences (analyse, boîte noire : utilisateur ↔ système)

Seuls l'utilisateur et le système apparaissent. Les fragments UML « opt » et « alt » marquent une option (cocher « se souvenir de moi » sur le web) et une alternative (succès ou refus).

Pré-conditions. Un compte a été créé par un administrateur. L'utilisateur n'est pas authentifié, ou sa session (respectivement son jeton) a expiré. Il connaît l'adresse de courriel associée au compte.

Scénario nominal.

1. L'utilisateur ouvre l'écran de connexion.
2. Il saisit son courriel et son mot de passe.
3. Sur le client web, il peut cocher « se souvenir de moi ».
4. Il soumet le formulaire.
5. Le système vérifie les identifiants.
6. En cas de succès :
   - Sur le client web, le système établit une session et redirige vers le tableau de bord administrateur si l'utilisateur est administrateur, vers le tableau de bord personnel sinon.
   - Sur le client mobile, le système émet un jeton d'accès et renvoie ce jeton ainsi que le profil de l'utilisateur.
7. En cas d'échec (identifiants invalides) :
   - Sur le client web, le système renvoie l'utilisateur au formulaire, avec les erreurs, sans réafficher le mot de passe.
   - Sur le client mobile, le système refuse la connexion et renvoie le message d'erreur.

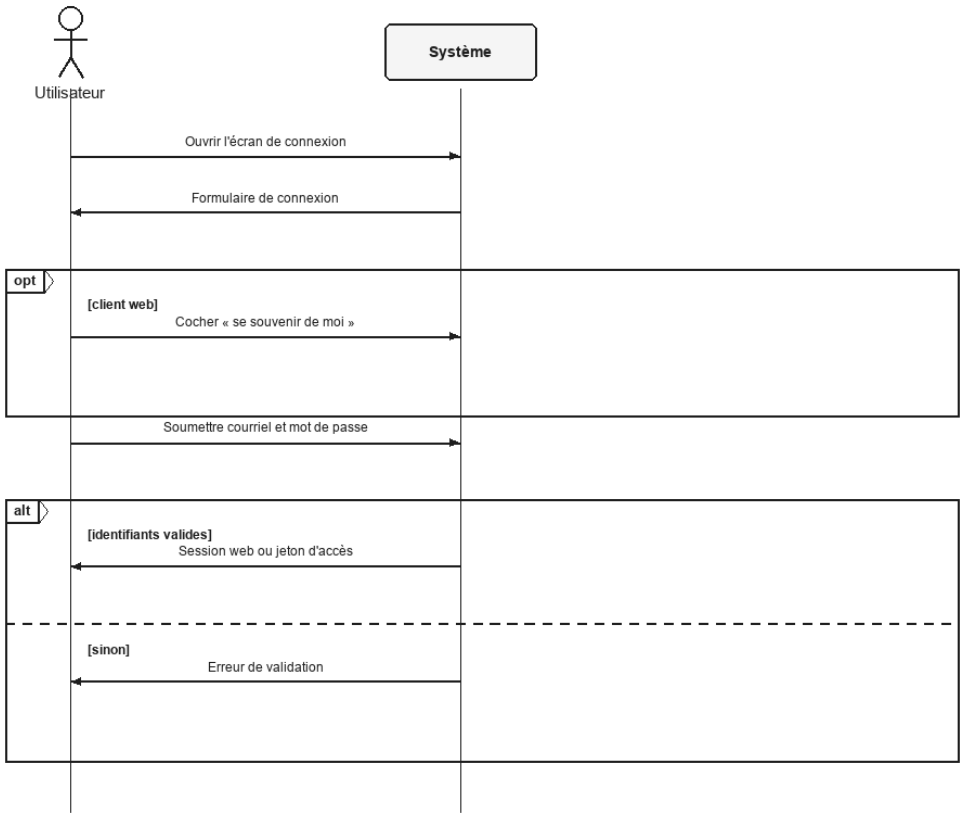


Figure 2.5 – Authentification

Post-conditions (succès). Session web ou jeton d'accès actif ; accès au tableau de bord du rôle.

Post-conditions (échec). Aucune session ni jeton durable ; parcours de connexion inchangé.

Pré-conditions. L'utilisateur n'est pas authentifié. Il dispose de l'adresse de courriel de son compte.

Scénario nominal.

1. L'utilisateur demande un lien de réinitialisation en saisissant son courriel.
2. Le système enregistre la demande et, si l'adresse correspond à un compte, envoie un message contenant un jeton et un lien.
3. L'utilisateur ouvre le lien (ou saisit le jeton, côté mobile).
4. Il choisit un nouveau mot de passe, confirmé, d'une longueur minimale de huit caractères.
5. Le système vérifie le jeton, met à jour le mot de passe et confirme le succès.

6. L’utilisateur peut alors s’authentifier avec le nouveau mot de passe.

En cas d’adresse inconnue, le système refuse la demande avec une erreur de validation visible, sur le web comme sur le mobile. En cas de jeton invalide, le mot de passe n’est pas modifié.

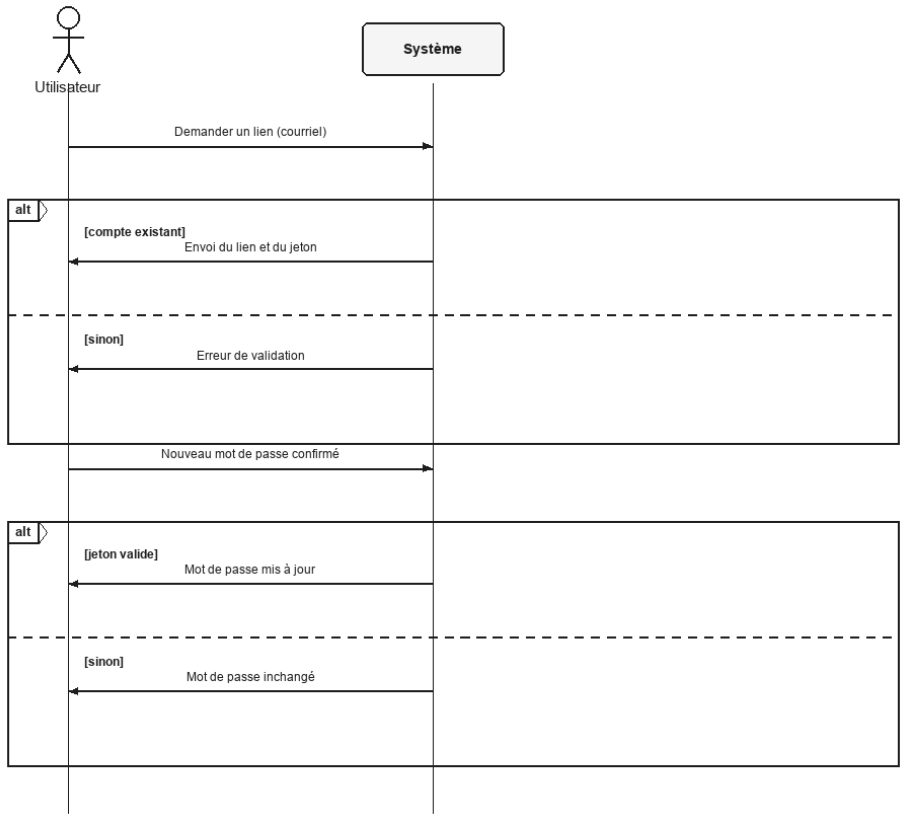


Figure 2.6 – Mot de passe oublié

Post-conditions (succès). Mot de passe remplacé ; nouvelle connexion requise.

Post-conditions (échec). Mot de passe antérieur inchangé.

2.2.3 Maquettage des interfaces

Nous avons dessiné dans Figma des maquettes fil de fer (*wireframes*) des écrans principaux. Une maquette fil de fer fixe la disposition d’un écran sans couleur ni contenu réel. Un rectangle barré tient la place d’une icône ou d’une image, une barre grise celle d’un texte variable. Ces maquettes associent chaque cas d’utilisation de la section 2.2.1 à un écran et permettent de vérifier que chaque acteur y trouve les informations et les commandes dont il a besoin.

Nous avons retenu les écrans qui portent les cas d'utilisation principaux de chaque acteur : la connexion, commune à tous, le tableau de bord et le détail de tâche du membre, la vue d'équipe du chef et la vue d'ensemble de l'administrateur. Les écrans web partagent la même structure. Un menu latéral propose des entrées qui dépendent du rôle. Un en-tête rappelle la page courante et porte le sélecteur de langue. La zone de contenu est organisée en cartes.

L'écran de connexion ne propose aucun lien d'inscription, puisque seul l'administrateur crée les comptes. Il donne accès au parcours de mot de passe oublié.

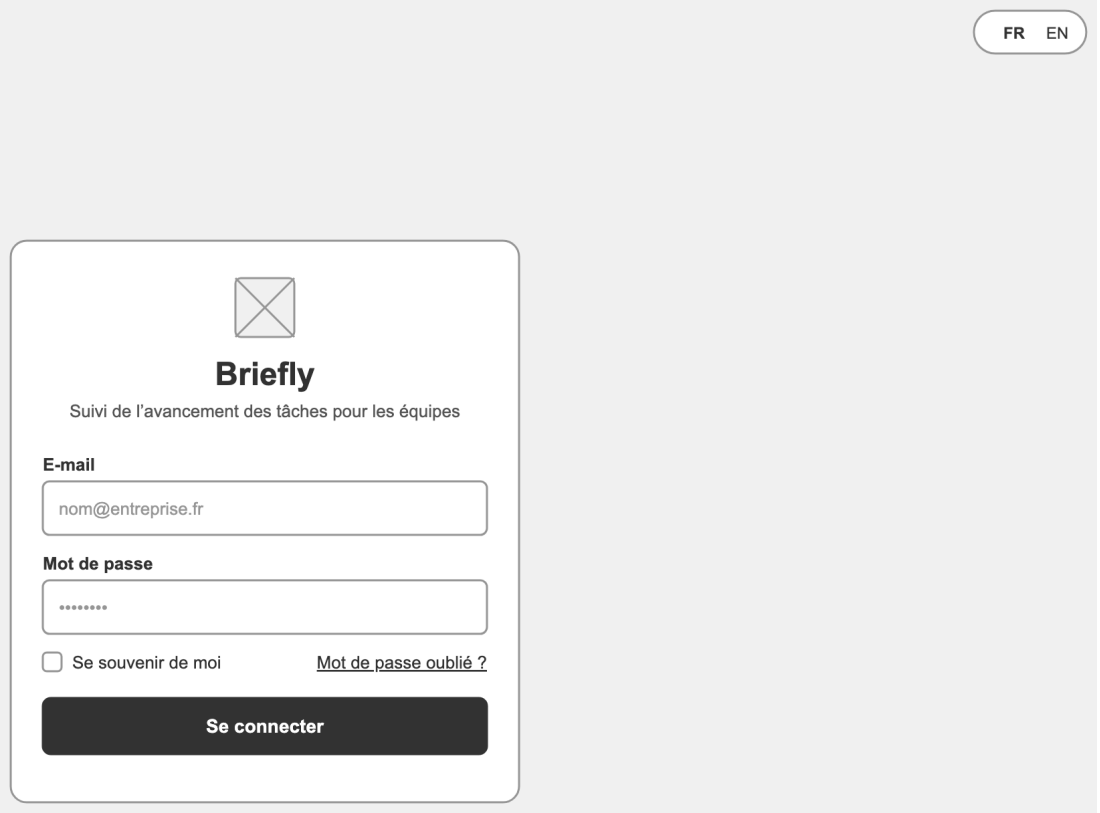


Figure 2.7 – Maquette fil de fer de l'écran de connexion (web)

Le tableau de bord du membre affiche son taux d'accomplissement, puis la liste de ses tâches. Chaque tâche porte son statut et une barre d'avancement dont le pourcentage découle du statut.

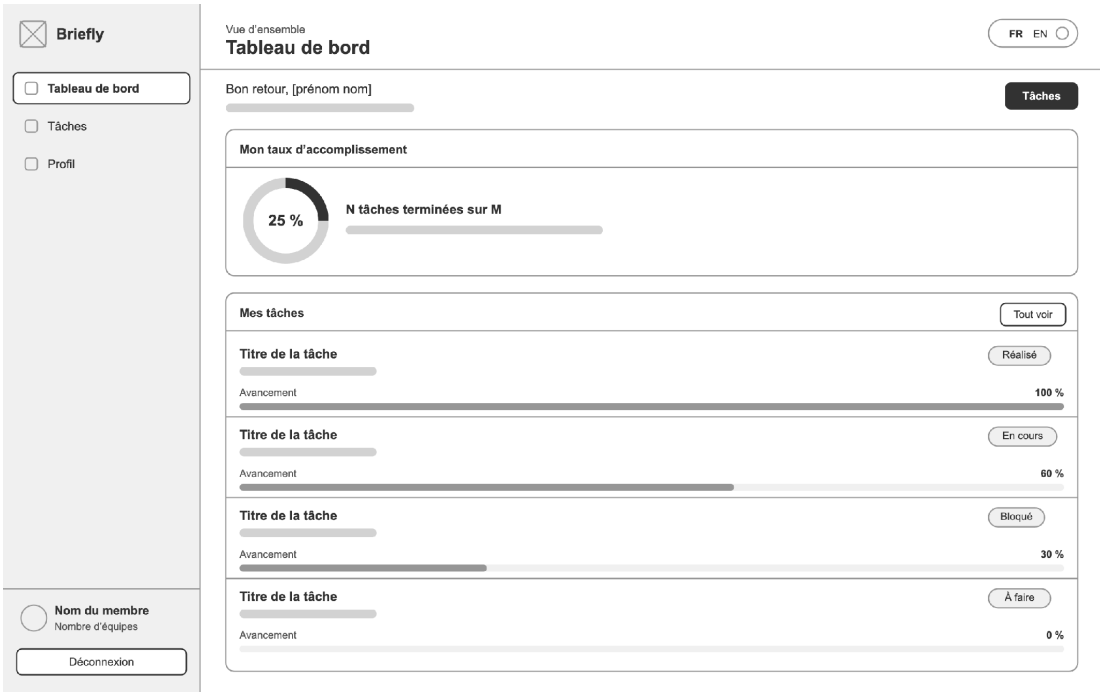


Figure 2.8 – Maquette fil de fer du tableau de bord du membre (web)

Le détail d'une tâche place à gauche les informations de la tâche et son historique, à droite le formulaire de mise à jour. Le champ de note de blocage n'est exigé que pour le statut « Bloqué ».

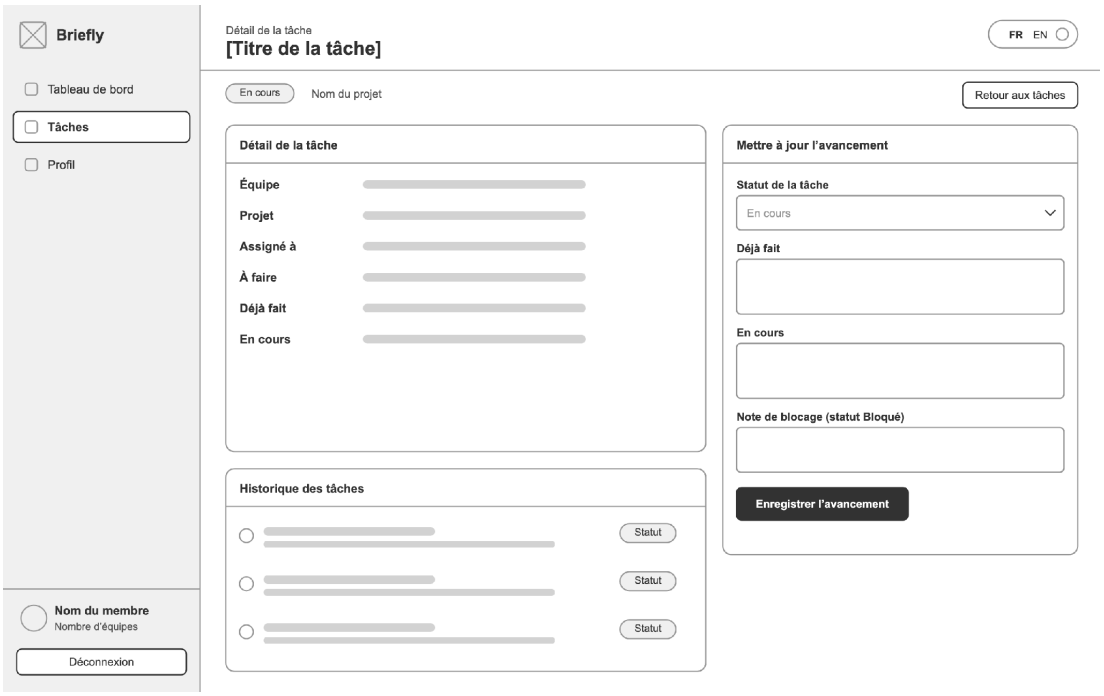


Figure 2.9 – Maquette fil de fer du détail de tâche et de la mise à jour d'avancement (web)

La vue d’équipe donne au chef quatre indicateurs, un filtre par statut et le tableau des tâches de ses équipes. La colonne « Blocage » lui permet de repérer les tâches à débloquer.



Figure 2.10 – Maquette fil de fer de la vue des tâches d’équipe du chef (web)

La vue d’ensemble de l’administrateur présente les effectifs, les taux d’accomplissement par projet et par équipe, ainsi que la répartition des tâches par statut. Son menu ne contient pas d’entrée « Tâches », car l’administrateur n’exécute pas de tâche.

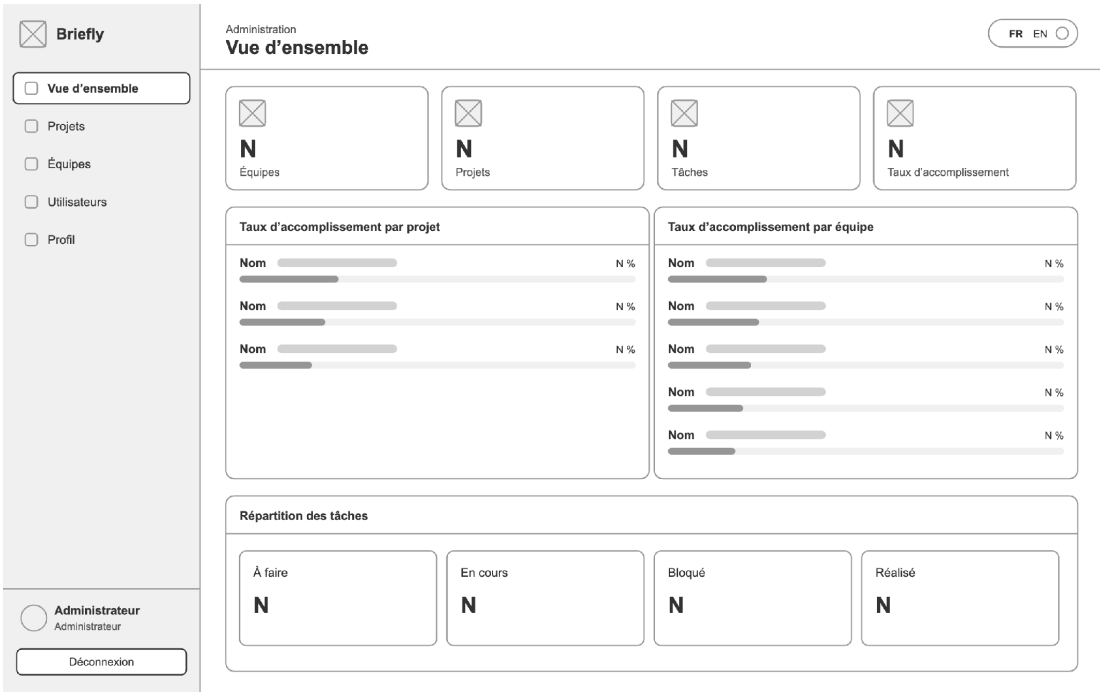


Figure 2.11 – Maquette fil de fer de la vue d'ensemble de l'administrateur (web)

Sur mobile, le menu latéral devient une barre de navigation inférieure à trois onglets. Les écrans conservent les champs du web. La saisie d'avancement comporte le statut, ce qui a été fait, la prochaine étape et la note de blocage.

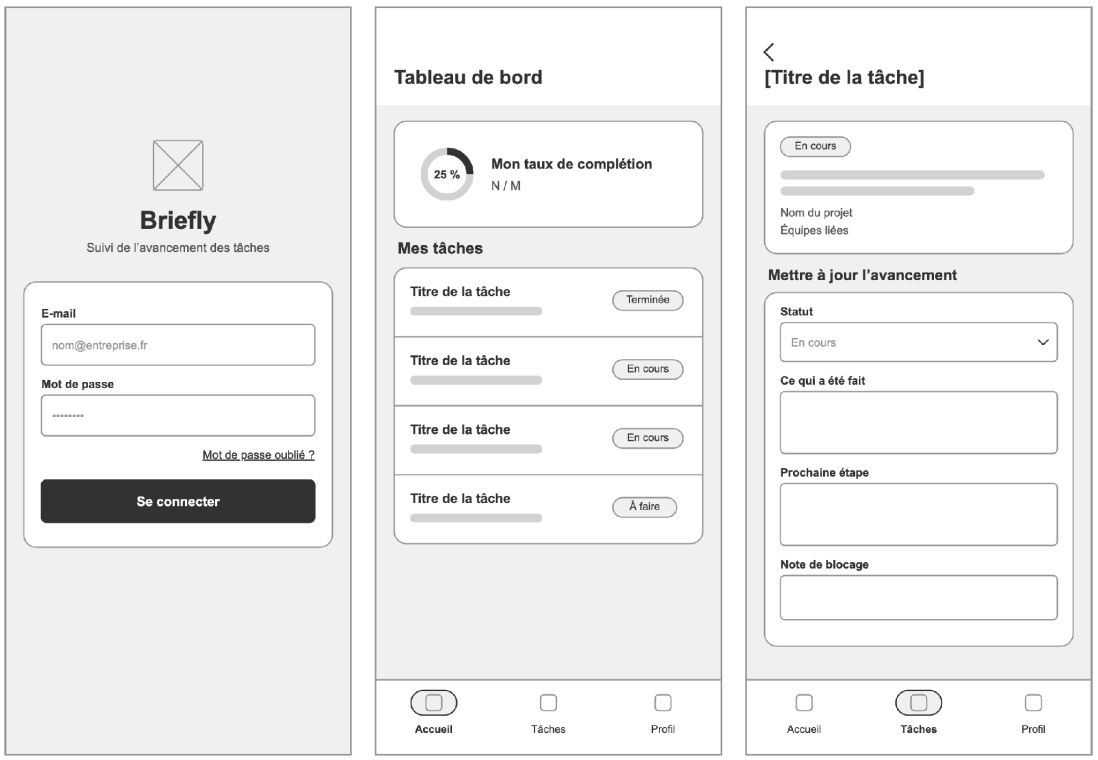


Figure 2.12 – Maquettes fil de fer mobiles : connexion, tableau de bord et détail de tâche

2.2.4 Planification des sprints

L'analyse se traduit en un backlog produit, découpé selon le plan Scrum du projet. Chaque sprint livre un ensemble cohérent, dans l'ordre des dépendances : le socle d'accès d'abord, puis l'exécution membre, le pilotage chef d'équipe, l'administration, la consolidation des tests et de la sécurité, enfin le client mobile. Les durées ci-dessous correspondent au calendrier 2026 du mémoire. Les sprints 1 à 4 livrent le client web. Le sprint 5 renforce la suite de tests, formalise la sécurité et branche l'intégration continue. Le sprint 6 développe le client mobile en parallèle du module mobile en formation, jusqu'à la mise en ligne début octobre.

Tableau 2.1 – Backlog du produit

Nom de sprint	Tâches	Durée
Sprint 1 : Authentification, profil, langue et thème	Connexion par courriel et mot de passe. Option « se souvenir de moi » (web). Déconnexion. Mot de passe oublié et réinitialisation. Modification du nom et du courriel. Changement de mot de passe. Bascule français / anglais. Thème clair / sombre. Absence d'auto-inscription	du 2 au 22 février 2026 (3 semaines)
Sprint 2 : Espace membre	Tableau de bord personnel, liste et détail des tâches, mise à jour d'avancement (note de blocage obligatoire si statut bloqué), historique paginé. Restriction de la mise à jour à l'assigné	du 23 février au 15 mars 2026 (3 semaines)
Sprint 3 : Espace chef d'équipe	Enrichissement du tableau de bord par équipe dirigée (totaux, en cours, bloquées, terminées dans la semaine, taux, dernières tâches bloquées). Liste des projets des équipes gérées. Détail de projet. Création de tâche (titre, description, assigné membre de l'équipe de contexte, statut). Modification de tâche. Vue des tâches d'équipe avec filtre de statut. Fiche membre conditionnée au partage d'une équipe gérée. Exclusion de la création de projets et d'utilisateurs	du 16 mars au 5 avril 2026 (3 semaines)
Sprint 4 : Espace administrateur	Tableau de bord administrateur (effectifs, taux global, graphique de statuts, taux par équipe et par projet). Création, consultation et mise à jour des utilisateurs (équipe minimale, indication de chef, rôle initial membre puis synchronisation, interdiction de réaffecter un administrateur). Gestion des équipes. Gestion des projets et rattachement d'équipes. L'administrateur est écarté des parcours d'exécution des tâches	du 6 au 26 avril 2026 (3 semaines)
Sprint 5 : Tests, sécurité et intégration continue	Consolidation de la suite de tests du client web. Mesures de sécurité. Intégration continue sur le dépôt	du 27 avril au 25 juillet 2026 (13 semaines)

Sprint 6 : Application mobile	Client mobile consommant les mêmes parcours (authentification par jeton, profil, espaces membre, chef d'équipe et administrateur). Module mobile en formation. Tests du client mobile. Mise en ligne début octobre	du 1er septembre au 10 octobre 2026 (6 semaines)

Ce découpage ordonne la livraison des exigences déjà analysées. Le sprint 5 consolide le socle web avant le second client. Entre le 25 juillet et le 1er septembre, le calendrier ne prévoit pas de sprint actif : le client mobile reprend à la rentrée, en parallèle du module Flutter en formation. Le sprint 6 se termine début octobre, avant la soutenance du 27 octobre.

Conclusion

Le système est nominatif. Hors connexion et mot de passe oublié, aucun visiteur n'a de parcours. L'administrateur crée les comptes, les équipes et les projets. Il n'exécute pas les tâches. Le chef d'équipe pilote ses équipes et crée des tâches. Il ne crée ni les projets ni les utilisateurs. Le membre ne met à jour que les tâches qui lui sont assignées. Une tâche bloquée exige une note. Les pourcentages d'avancement viennent du statut : 0 %, 60 %, 30 % ou 100 %.

Les cas d'utilisation, les séquences en boîte noire et les maquettes traduisent ces frontières en écrans web et mobile. Le backlog est réparti selon le tableau 2.1. Cet ensemble constitue le cahier des charges interne du projet.

Chapitre 3 : Conception

Introduction

Le chapitre 2 a fixé le périmètre fonctionnel de Briefly et les exigences qui en découlent. Nous traduisons ici ces spécifications en une architecture et en des collaborations d'objets assez précises pour orienter la réalisation, du déploiement physique jusqu'aux règles métier et aux données persistées.

Nous concevons Briefly comme une application interne unique, accessible depuis un navigateur et depuis une application mobile, et portée par un même serveur d'application. Les règles de validation, d'autorisation et de persistance forment une source de vérité unique, quel que soit le client. La conception formalise cette organisation, d'abord à l'échelle globale (architectures physique et logique), puis à l'échelle des collaborations (séquences, activité, classes) et du schéma relationnel.

La section 3.1 présente l'architecture globale : le déploiement 3-tiers, le principe de communication unifiée et le découpage logique en couches. La section 3.2 descend au niveau des objets : trois scénarios métier représentatifs, le flux d'activité de mise à jour d'une tâche, le modèle de classes du domaine, le schéma relationnel qui en découle, puis le rôle du client mobile dans cette architecture.

3.1 Conception globale

Cette section pose où s'exécute chaque responsabilité, comment les clients dialoguent avec le serveur, et selon quelles couches l'application est organisée. Les collaborations de la section 3.2 s'appuient sur ces frontières.

3.1.1 Architecture physique

Nous retenons une architecture physique **3-tiers**. Les responsabilités de présentation, de traitement et de persistance sont déployées sur des niveaux distincts.

Le **premier niveau** est celui des clients. Deux supports d'interface coexistent : le navigateur, qui affiche des pages interactives produites par le serveur, et l'application mobile, qui affiche des écrans alimentés par des réponses structurées du même serveur. Ces deux clients n'embarquent pas le métier. Ils formulent des requêtes vers le serveur d'application et en restituent le résultat à l'utilisateur.

Le **deuxième niveau** est le serveur d'application. Il accueille le routage, l'authentification, l'autorisation, la validation des entrées, les traitements métier et l'accès aux données. C'est à ce niveau que se concentrent les règles que nous ne voulons pas dupliquer d'un client à l'autre.

Le **troisième niveau** est le serveur de données relationnel. Il conserve les utilisateurs, les équipes, les projets, les tâches et l'historique des points d'avancement. Les clients n'y accèdent jamais directement.

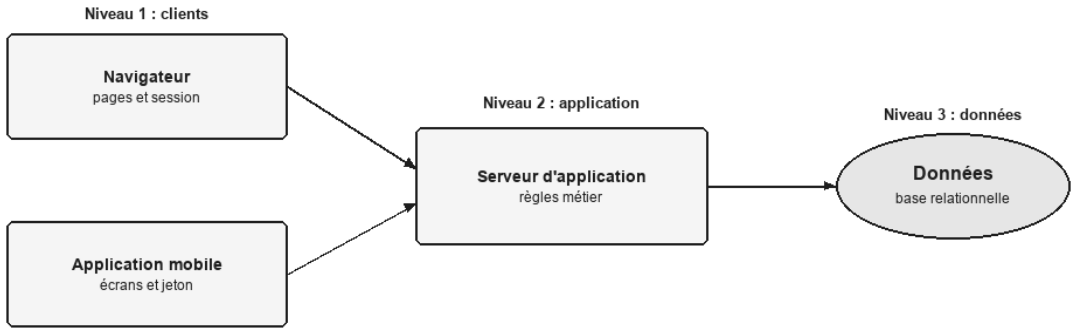


Figure 3.1 – Architecture physique de Briefly (déploiement 3-tiers)

Ce découpage se justifie, dans le contexte de Briefly, par trois critères.

Flexibilité. Les deux clients évoluent à des rythmes différents : l'interface web et l'interface mobile peuvent être enrichies indépendamment, tant que le contrat d'échange avec le serveur reste stable. Le serveur d'application peut être déployé ou mis à jour indépendamment du schéma de données, dès lors que les évolutions de structure restent maîtrisées.

Sécurité. Les identifiants de base de données, les secrets d'authentification et les données sensibles ne quittent pas le périmètre serveur. Les clients ne reçoivent que les informations autorisées pour leur rôle. L'autorisation s'exerce avant tout accès aux données métier, ce qui empêche un client, même authentifié, d'outrepasser son rôle. L'absence d'auto-inscription, déjà posée au chapitre 2, se traduit par un point d'entrée unique : la création d'utilisateur est une opération d'administration, pas un parcours public.

Performance. Le serveur d'application concentre les agrégations utiles aux tableaux de bord, le chargement des relations et la pagination de l'historique. Il évite aux clients de multiplier les accès directs à la base. Le navigateur peut s'appuyer sur un rendu préparé par le serveur pour les pages métier. Le mobile ne récupère que les données nécessaires à l'écran courant. Le dimensionnement du serveur de données reste indépendant des clients.

Communication unifiée : un catalogue d'opérations, deux formes de réponse

Briefly ne duplique pas deux catalogues d'opérations, l'un pour le web et l'autre pour le mobile. Les opérations métier (connexion, consultation d'un tableau de bord, mise à jour d'avancement, administration des comptes, etc.) sont exposées une seule fois côté serveur. Selon le client, la même opération produit soit une page ou une redirection adaptée au navigateur, soit une réponse structurée adaptée au mobile.

Ce choix unifie le contrat fonctionnel. Une création de tâche, une mise à jour d'avancement ou une consultation de tableau de bord empruntent le même chemin logique, que l'acteur soit devant le navigateur ou devant l'application mobile. Les règles de validation et d'autorisation s'appliquent une seule fois, ce qui limite les écarts entre clients.

L'authentification s'adapte au mode d'usage, tout en protégeant les mêmes opérations :

- **Navigateur.** L'utilisateur s'authentifie par session. Après une connexion réussie, il accède au tableau de bord correspondant à son rôle (administration ou suivi opérationnel).
- **Mobile.** Le même parcours de connexion peut aboutir à un jeton d'accès durable, transmis avec les requêtes suivantes. La déconnexion invalide ce jeton côté serveur.

Le navigateur s'appuie sur une session, le mobile sur un jeton. Ces deux modes protègent les mêmes opérations : le serveur distingue le contexte du client sans réécrire le métier.

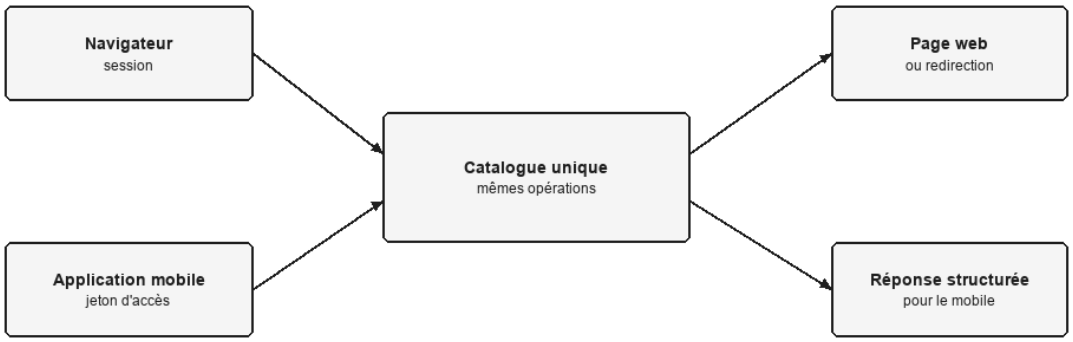


Figure 3.2 – Communication unifiée entre clients et serveur

Tableau 3.1 – Filtres d'accès (conception)

Filtre	Rôle de conception
Espace administrateur	Les opérations de gestion des comptes, des équipes et des projets sont réservées à l'administrateur.

Filtre	Rôle de conception
Espace chef d'équipe	La création de tâches et la vue d'ensemble d'équipe sont réservées au chef d'équipe.
Exclusion de l'administrateur des tâches	L'administrateur ne suit pas les tâches comme un membre : il est tenu hors des parcours d'exécution et de mise à jour d'avancement.
Langue d'interface	Sur le web, la langue est une préférence de l'utilisateur. Sur mobile, elle est choisie sur l'appareil et ne modifie pas les données métier.

Ces filtres s'appliquent aux groupes de parcours. Des règles plus fines, liées à une ressource précise (un projet, une tâche), complètent ce dispositif au moment de l'action, comme nous le précisons à la sous-section suivante.

3.1.2 Architecture logique

Nous organisons le serveur d'application en couches à responsabilités séparées. La couche de coordination reçoit la requête, vérifie les droits et la forme des données, puis délègue le traitement métier. Les règles « qui peut agir sur quoi » ne se confondent pas avec les règles « comment persister ou calculer ».

Les couches retenues sont les suivantes.

Présentation (côté clients). Le navigateur et le mobile collectent la saisie, affichent les tableaux de bord et transmettent les actions utilisateur. Cette couche n'embarque pas les invariants métier : elle peut toutefois guider la saisie (par exemple signaler qu'une note de blocage manque) sans se substituer au serveur.

Coordination. Cette couche achemine la requête vers le bon traitement, contrôle la forme des données (champs obligatoires, formats, cohérence des listes) et choisit la réponse adaptée au client (page ou données structurées).

Autorisation. Elle répond à la question : l'utilisateur authentifié a-t-il le droit d'effectuer cette action sur cette ressource ? Exemples structurants pour Briefly : un chef d'équipe n'accède à un projet que si l'une de ses équipes y est rattachée ; seul le membre assigné peut mettre à jour l'avancement d'une tâche ; l'administrateur ne crée pas de tâche pour son propre compte d'exécution.

Application / métier. Elle regroupe les traitements qui dépassent une simple écriture en base : synchronisation des appartenances aux équipes et dérivation du rôle chef, vérification que l'assigné d'une tâche appartient à l'équipe de contexte, enregistrement d'un point d'avancement avec historique, calculs des indicateurs de tableaux de bord.

Domaine. Entités et énumérations portent l'état métier (utilisateur, équipe, projet, tâche, point d'avancement, rôles, statuts de tâche). Elles expriment les relations du modèle analysé au chapitre 2, indépendamment du canal utilisé par le client.

Persistance. Le modèle relationnel matérialise le domaine. Tables, clés étrangères et tables associatives traduisent les cardinalités du diagramme de classes. Les jetons d'accès mobile et les artefacts techniques (session, cache) relèvent aussi de cette couche, sans porter de règle métier propre.

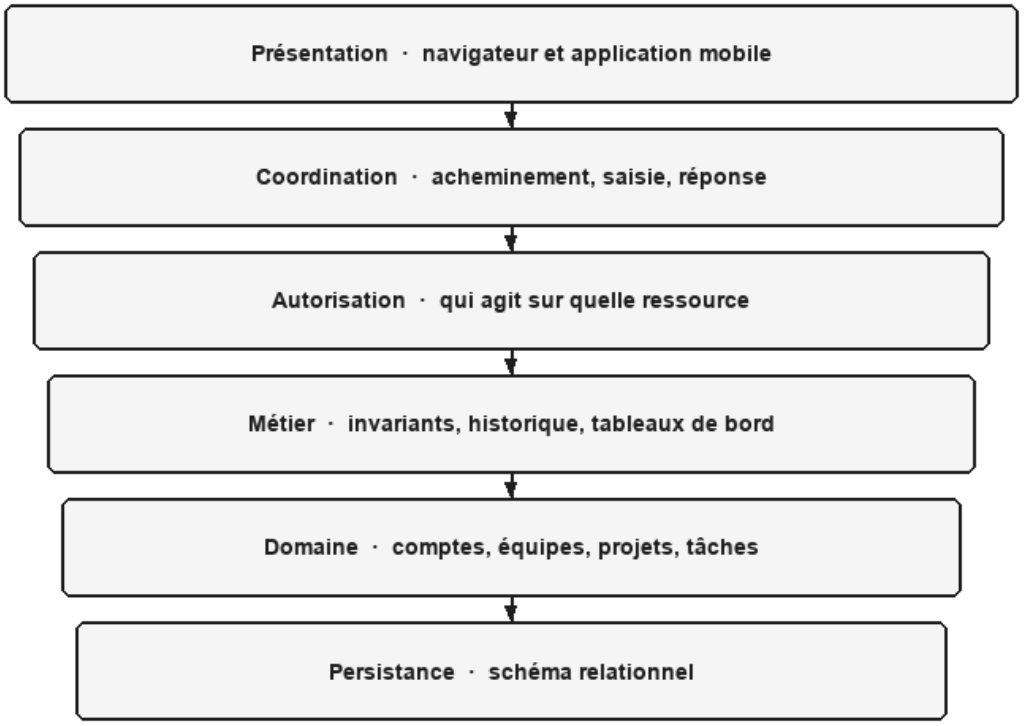


Figure 3.3 – Architecture logique en couches (dépendances descendantes)

Un scénario métier se lit de haut en bas : interface, coordination, autorisation, métier, persistance. Les diagrammes de séquence de la section 3.2 suivent cette chaîne. Dans certains cas simples, la coordination peut déléguer directement la persistance après autorisation, sans traitement métier intermédiaire.

3.2 Conception détaillée

Nous détaillons les collaborations d'objets pour l'entrée d'un utilisateur dans le système (par l'administrateur), la création d'une tâche par un chef d'équipe, et le point d'avancement saisi par le membre assigné. Suivent le flux d'activité de mise à jour, le diagramme de classes du domaine, le schéma relationnel, puis la place du client mobile.

3.2.1 Partie métier

Les trois séquences ci-dessous mettent en scène les mêmes types d'objets : **Acteur**, **Interface**, **Contrôleur**, **Service**, **Modèle**. L'interface désigne indifféremment l'écran web ou mobile ; le contrôleur représente le point d'entrée serveur commun aux deux clients.

Création d'un utilisateur par l'administrateur

Il n'existe pas de parcours public d'inscription. Seul un administrateur authentifié peut créer un compte.

Le contrat de saisie impose :

- identité et adresse électronique (unique dans l'organisation) ;
- mot de passe confirmé, d'une longueur minimale fixée ;
- appartenance à au moins une équipe existante ;
- indication optionnelle des équipes dont l'utilisateur est chef, choisies parmi celles auxquelles il appartient.

Le service métier crée le compte avec un rôle initial de membre, rattache les équipes, pose le statut de chef lorsque demandé, puis dérive le rôle « chef d'équipe » si au moins une équipe est dirigée. Un compte administrateur existant n'est ni créé ni réaffecté par ce parcours.

Pré-conditions. L'acteur est authentifié en tant qu'administrateur. Au moins une équipe existe. L'adresse électronique n'est pas déjà utilisée.

Scénario nominal. L'administrateur ouvre le formulaire de création, saisit l'identité, le mot de passe confirmé et les équipes, éventuellement en marquant certaines d'entre elles comme équipes de chefferie, puis valide. Le serveur contrôle les droits et la forme des données, délègue au service métier, qui persiste le compte et les appartenances. Le client affiche la confirmation ou la liste mise à jour.

Le fragment « alt » sépare la saisie acceptée de la saisie refusée. Dans le cas accepté, « loop » parcourt chaque équipe pour attacher l'utilisateur. « opt » pose le statut de chef seulement pour les équipes également cochées comme équipes dirigées.

Post-conditions. Un compte utilisateur existe, lié à au moins une équipe. Le rôle est membre ou chef d'équipe selon les équipes qu'il dirige. Aucune session n'est ouverte pour le nouvel utilisateur : il devra s'authentifier.

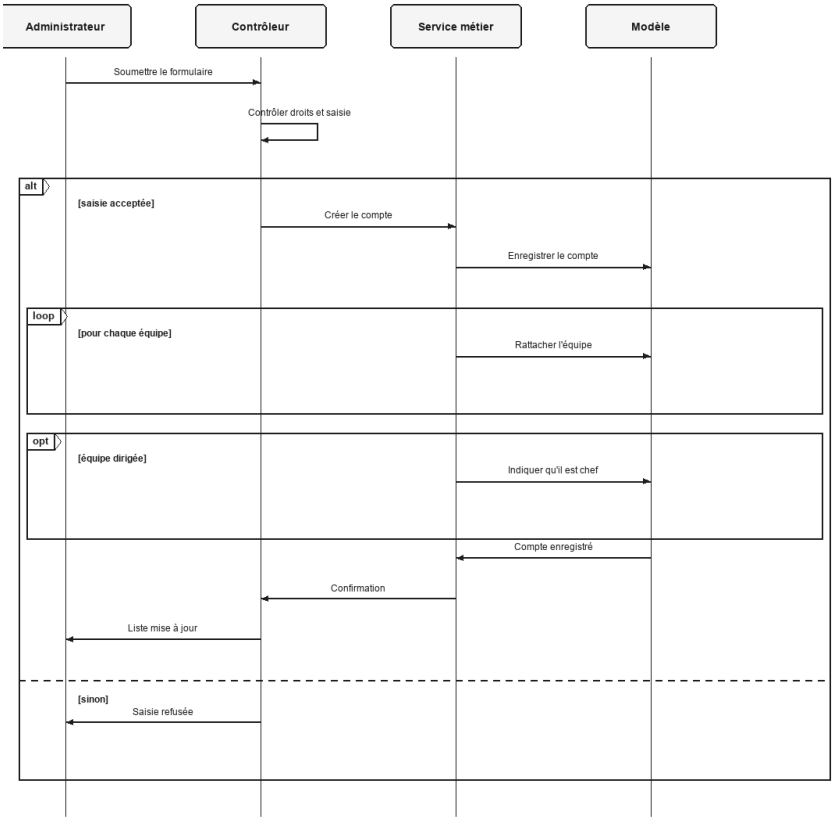


Figure 3.4 – Diagramme de séquence : création d'un utilisateur par l'administrateur

Création d'une tâche par le chef d'équipe

La création d'une tâche s'inscrit dans un contexte d'équipe. Le chef d'équipe n'accède au projet que si l'une des équipes qu'il dirige y est rattachée. Il doit diriger au moins une équipe pour créer une tâche. Le serveur retient alors une équipe de contexte parmi les équipes qu'il dirige et qui sont liées au projet. Si aucune ne l'est, l'action est refusée.

Il reste à vérifier que l'assigné appartient à cette équipe de contexte. C'est le seul invariant métier qui s'ajoute à l'autorisation sur le projet.

Les données de saisie comprennent un titre obligatoire, une description optionnelle, un membre assigné et un statut initial parmi les valeurs du cycle de vie de la tâche.

Pré-conditions. L'acteur est authentifié en tant que chef d'équipe. Le projet visé est lié à l'une de ses équipes. L'équipe de contexte compte au moins un membre assignable.

Scénario nominal. Le chef ouvre le formulaire rattaché au projet, choisit un membre de l'équipe, un titre, un statut initial, éventuellement une description, puis valide. Après contrôle des droits et des données, le service métier vérifie le projet et l'assigné, puis crée la tâche rattachée au projet.

Le fragment « alt » sépare le cas où l'assigné appartient à l'équipe de contexte du cas où cette condition n'est pas remplie (refus).

Post-conditions. Une tâche existe, rattachée au projet et à l'assigné, avec le statut initial saisi. Aucun point d'avancement historisé n'existe encore : l'historique commence au premier enregistrement de progression.

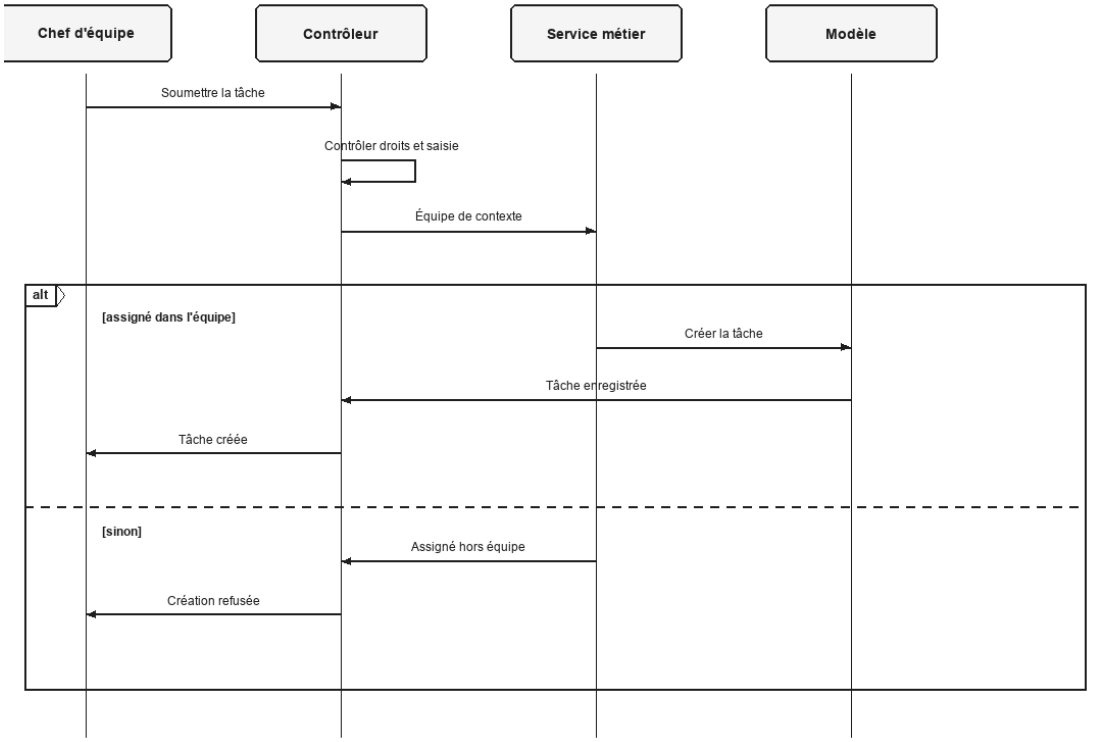


Figure 3.5 – Diagramme de séquence : le chef d'équipe crée une tâche

Mise à jour de l'avancement par le membre assigné

Seul le membre auquel la tâche est assignée peut enregistrer un point d'avancement. Le contrôle porte sur l'égalité entre l'utilisateur courant et l'assigné.

La saisie comprend un statut, des textes optionnels « réalisé » et « à venir », et une **note de blocage obligatoire** lorsque le statut est « bloqué ».

Le service métier applique ensuite les effets attendus :

- mise à jour de la tâche (statut, textes, note de blocage le cas échéant) ;
- création d'un enregistrement d'historique qui fige le même instantané ;
- date de clôture renseignée lorsque la tâche passe au statut « terminé », effacée si l'on sort de ce statut ;

- effacement de la note de blocage sur la tâche lorsque le statut n'est plus « bloqué ».

Le pourcentage affiché dans les tableaux de bord n'est pas saisi : il est **dérivé du statut** selon la convention retenue (à faire 0 %, en cours 60 %, bloqué 30 %, terminé 100 %). Cette convention évite un second indicateur contradictoire avec le statut.

Pré-conditions. L'acteur est authentifié, n'est pas administrateur, et la tâche lui est assignée.

Scénario nominal. Le membre ouvre la tâche, choisit un statut, renseigne le réalisé et le prochain pas, ajoute une note de blocage si nécessaire, puis enregistre. Le service met à jour la tâche et insère un point d'avancement dans l'historique.

Le fragment « alt » refuse la requête lorsque le statut est « bloqué » et que la note est absente. Dans l'autre branche, « opt » ne pose la date de clôture que si le statut retenu est « terminé ».

Post-conditions. La tâche reflète le nouveau statut et les textes d'avancement. Un enregistrement d'historique relie la tâche et l'utilisateur. La date de clôture est cohérente avec le statut terminé. Le pourcentage dérivé correspond au statut.

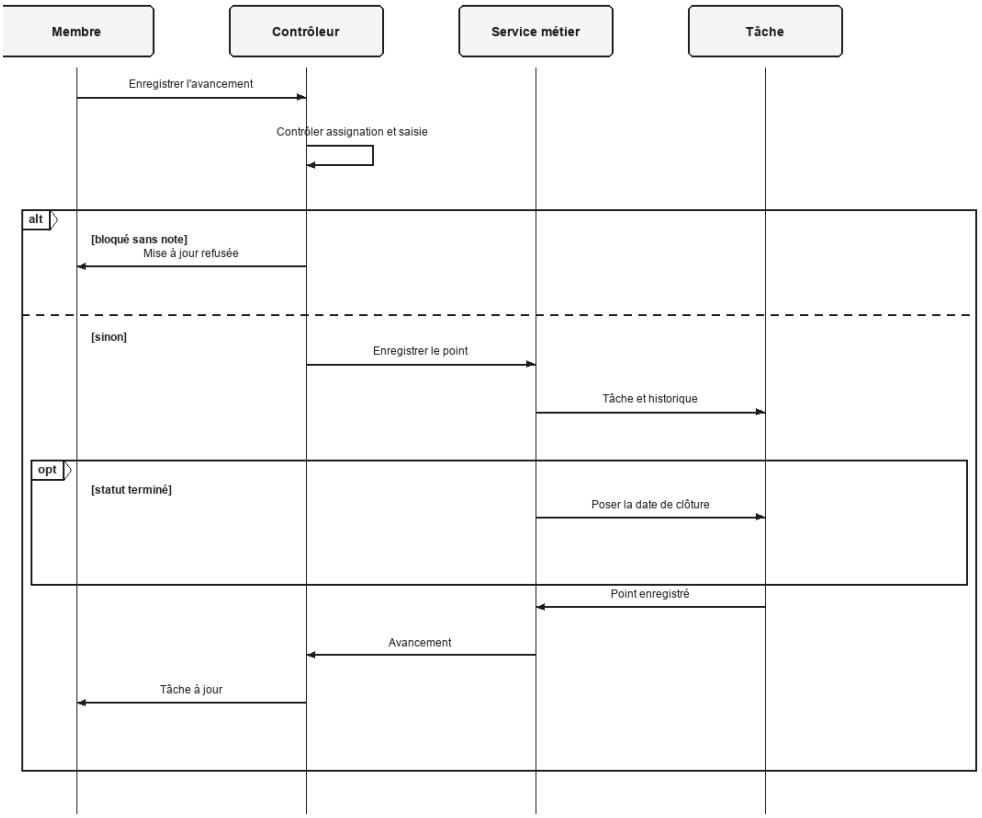


Figure 3.6 – Diagramme de séquence : le membre met à jour l'avancement

Diagramme d'activité : mettre à jour une tâche

Le diagramme de séquence décrit les objets. Le diagramme d'activité isole la décision qui structure le métier d'avancement. Si le statut choisi est « bloqué », le système exige une note de blocage. Sans cette note, la mise à jour est rejetée. Pour tout autre statut, ou lorsque la note est présente, le système met à jour la tâche et crée un enregistrement d'historique. La date de clôture est posée à l'entrée dans le statut « terminé » et remise à vide lorsqu'on en sort.

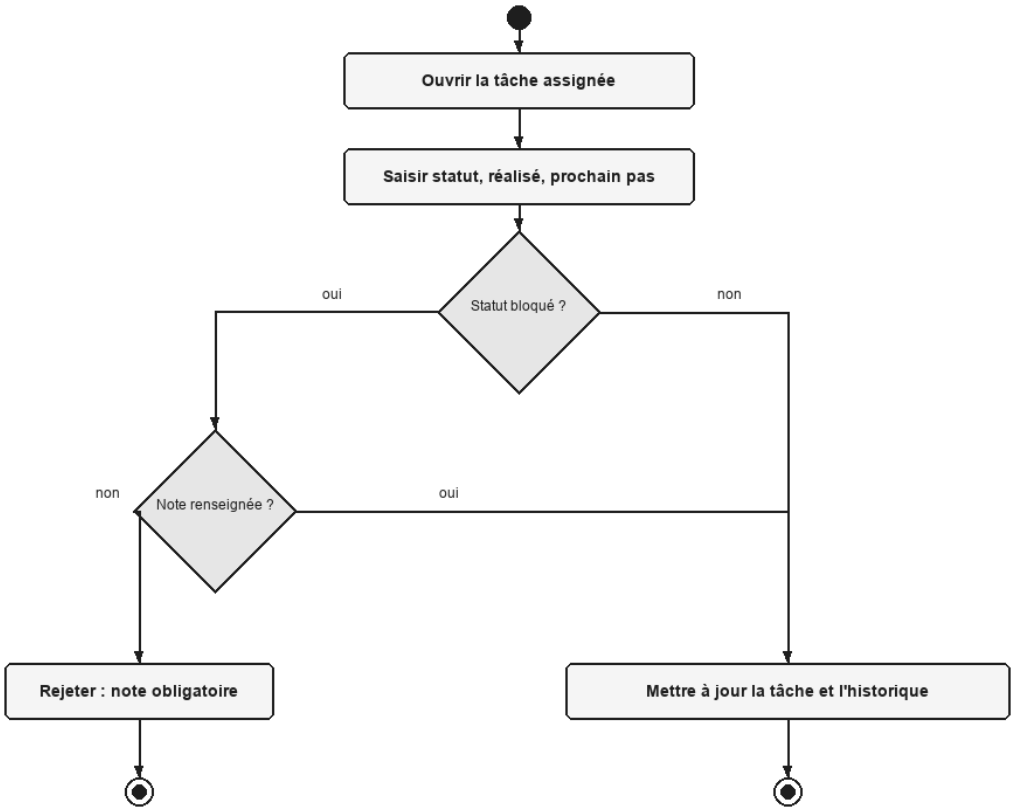


Figure 3.7 – Diagramme d'activité : mise à jour d'une tâche

Cette activité est la même pour le navigateur et pour le mobile. Un client peut signaler à l'avance une note manquante pour améliorer l'ergonomie, mais le serveur demeure l'autorité : une demande incomplète est refusée de la même façon quel que soit le support.

Diagramme de classes

La figure 3.8 modélise le domaine Briefly. La classe Utilisateur regroupe ce que tous les comptes partagent : identité, adresse électronique, mot de passe et rôle. Deux classes en héritent. Administrateur gère les comptes, les équipes et les projets. Employé désigne un membre ou un chef d'équipe, c'est-à-dire tout compte qui reçoit des tâches.

Un employé est lié aux équipes par deux associations. L'association « membre de » indique qu'un employé appartient à au moins une équipe, condition posée à la création de compte. L'association « chef » indique qu'une équipe a au plus un chef à un instant donné : la conception prévoit de retirer l'ancien chef avant d'en désigner un nouveau. Une équipe travaille sur plusieurs projets, et un projet peut être partagé entre plusieurs équipes ou n'en avoir encore aucune.

Un projet contient ses tâches. Nous représentons ce lien par une composition, car la suppression d'un projet supprime ses tâches en cascade. Une tâche est assignée à un seul employé, ce qui permet de décider sans ambiguïté qui peut la mettre à jour. La classe d'association Avancement est attachée à ce lien. Elle décrit un point d'avancement saisi par l'employé sur sa tâche : statut, travail réalisé, travail à venir, note de blocage et date. Un employé saisit plusieurs points sur la même tâche au fil du temps. La date distingue les avancements successifs d'un même couple employé et tâche.

L'association « /participe à » entre Employé et Projet est une association dérivée, signalée par la barre oblique. Aucune structure de stockage ne la duplique. Elle se calcule à partir des équipes de l'employé et des projets liés à ces équipes, ce qui fonde l'accès d'un chef d'équipe à un projet.

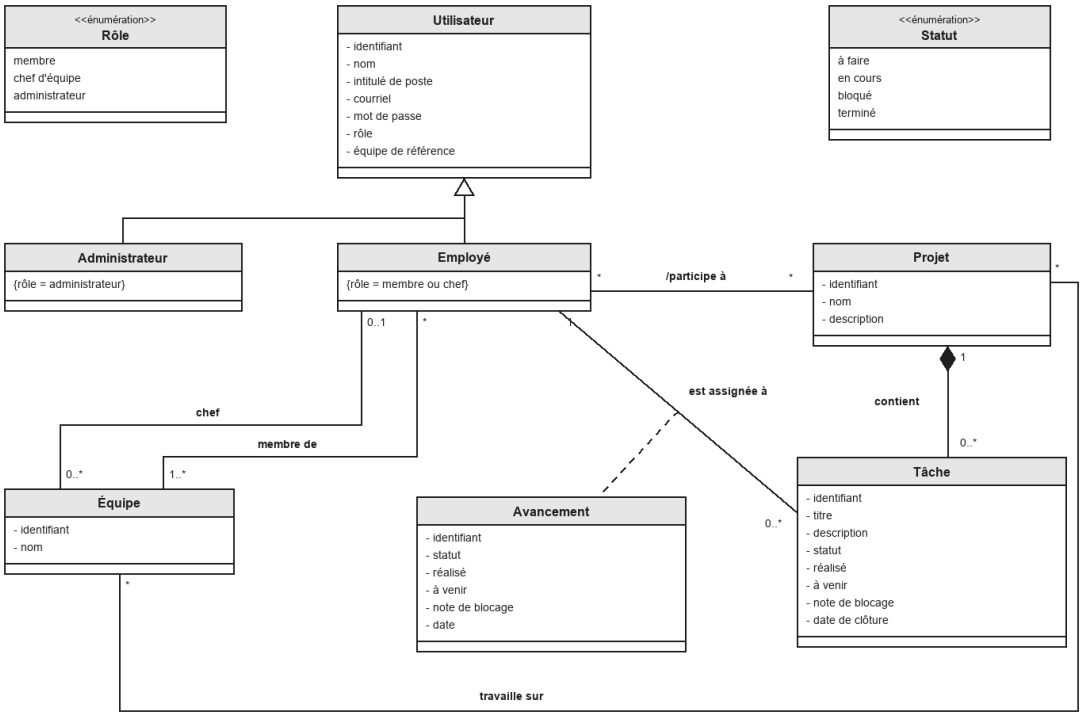


Figure 3.8 – Diagramme de classes du domaine Briefly

Tableau 3.2 – Détails des classes du domaine

Classe	Rôle dans la conception
Utilisateur	Compte interne : identité, intitulé de poste, adresse électronique unique, mot de passe, rôle et équipe de référence.
Administrateur	Spécialisation responsable de la création et de la maintenance des comptes, des équipes et des projets. Il n'exécute pas de tâches opérationnelles.
Employé	Spécialisation membre ou chef d'équipe. Le rôle chef est dérivé de l'association « chef » sur les équipes.
Équipe	Regroupe des employés et des projets. C'est l'équipe de contexte qui encadre la création d'une tâche et le tableau de bord du chef.
Projet	Regroupe les tâches. Un chef n'y accède que si l'une des équipes qu'il dirige y est liée.
Tâche	Unité de travail : titre, description, statut, textes d'avancement, note de blocage et date de clôture. Elle appartient à un projet et à un employé assigné.
Avancement	Point d'avancement historisé : statut et textes au moment de la saisie, lié à une tâche et à l'employé auteur.
Rôle	Trois valeurs : membre, chef d'équipe, administrateur. Elles distinguent les spécialisations du compte.
Statut de tâche	Quatre valeurs : à faire, en cours, bloqué, terminé. Chacune porte un pourcentage d'avancement affiché (0, 60, 30 et 100).

Schéma relationnel

Le passage du diagramme de classes au modèle relationnel suit les règles habituelles. Chaque classe du domaine devient une table, et son identifiant une clé primaire. Une association 1-N place la clé de la classe côté 1 dans la table côté N. Une association N-N devient une table associative dont la clé primaire réunit les deux clés étrangères. Les énumérations de rôles et de statuts sont stockées sous forme de codes dans les colonnes concernées.

Quatre éléments du diagramme demandent une règle particulière. L'héritage est ramené à une table unique : Utilisateur, Administrateur et Employé partagent la table des comptes, et une colonne de rôle indique la spécialisation. Les associations « membre de » et « chef » sont réunies dans une table associative, avec un indicateur de chefferie. La classe d'association Avancement devient une table d'historique, avec son propre identifiant, puisqu'un même couple employé et tâche produit plusieurs points dans le temps. Enfin, l'association dérivée « /participe à » ne produit aucune table.

Le tableau 3.3 donne la forme logique du schéma. Les clés primaires sont soulignées et les clés étrangères précédées de #.

Tableau 3.3 – Schéma relationnel logique

Table	Attributs	Contraintes
Compte	id, nom, intitulé de poste, courriel, mot de passe, rôle, #équipe de référence	Courriel unique. L'équipe de référence peut disparaître sans supprimer le compte
Équipe	id, nom	Aucune
Appartenance	#équipe, #compte, est chef	Un compte appartient à une ou plusieurs équipes. La suppression de l'équipe ou du compte retire l'appartenance
Projet	id, nom, description	Aucune
Rattachement projet–équipe	#projet, #équipe	Un projet peut être lié à plusieurs équipes. La suppression du projet ou de l'équipe retire le rattachement
Tâche	id, #projet, #assigné, titre, description, statut, réalisé, à venir, note de blocage, date de clôture	L'assigné est un compte. La suppression du projet ou de l'assigné retire la tâche
Avancement	id, #tâche, #auteur, statut, réalisé, à venir, note de blocage	Un même couple tâche et auteur peut avoir plusieurs points. La suppression de la tâche ou de l'auteur retire l'historique

3.2.2 Client mobile dans l'architecture

Le client mobile est une **seconde interface de présentation**. Il invoque les mêmes opérations métier que le navigateur et s'appuie sur les mêmes règles d'autorisation et de validation côté serveur. Aucune règle métier n'est dupliquée dans le mobile : les choix locaux concernent l'ergonomie (navigation entre écrans, langue et thème affichés, guidage de saisie), pas le calcul des droits ni des indicateurs d'avancement.



Figure 3.9 – Chaîne de communication du client mobile

Le téléphone n'accède pas directement au serveur de données. Il dialogue avec le même serveur d'application que le navigateur. Les trois scénarios de la sous-section 3.2.1 s'appliquent donc sans transcription parallèle des règles.

Conclusion

Le métier est centralisé sur le serveur d'application. Le navigateur et le mobile ne font que présenter les opérations et transmettre les actions des utilisateurs. Ils n'accèdent pas à la base de données.

La communication unifiée garantit un seul catalogue d'opérations et un seul jeu de règles, avec des réponses adaptées à chaque client. Les séquences de création d'utilisateur, de création de tâche et de mise à jour d'avancement, le diagramme d'activité et le modèle de classes matérialisent les exigences du chapitre 2 : exclusion de l'administrateur des tâches, assignation unique, note de blocage obligatoire, historisation des points d'avancement, pourcentage dérivé du statut. Le chapitre 4 détaille les choix techniques et la réalisation de cette conception.

Chapitre 4 : Réalisation

Introduction

Les chapitres précédents ont fixé le besoin, le périmètre et la conception de Briefly. Nous décrivons ici la mise en œuvre : l'environnement de travail, les outils et les technologies, puis les itérations qui ont produit l'application web Laravel et le client mobile Flutter.

Pour chaque sprint, nous indiquons l'objectif, les écrans réalisés et, le cas échéant, un jeu de tests fonctionnels couverts par PHPUnit. L'architecture en couches figure au chapitre 3, le catalogue des cas d'utilisation au chapitre 2. Le sprint 5 consolide tests, sécurité (§4.3) et intégration continue (§4.4). Le sprint 6 livre le client Flutter.

4.1 Environnement du travail

4.1.1 Configuration matérielle

Nous avons développé Briefly en 2026 sur un MacBook Air M1.

Tableau 4.1 – Configuration matérielle

Poste	Type	Processeur	Mémoire	Stockage	Système d'exploitation
Poste de développement	MacBook Air (M1)	Apple M1, 8 cœurs	8 Go	SSD 245 Go (APFS)	macOS 26.5.2 (Darwin 25.5.0)

4.1.2 Configuration logicielle

Outils

Le tableau 4.2 indique les outils utilisés pour rédiger le code, servir la base de données locale, préparer les diagrammes de conception et compiler le client iOS.

Tableau 4.2 – Outils de développement

Outil	Rôle dans le projet
Visual Studio Code	Édition du code Laravel (PHP, Blade, CSS) et Flutter (Dart)
XAMPP (MySQL)	Serveur de base de données local pour l'application web. Le schéma briefly est créé puis migré par Artisan
diagrams.net	Élaboration des diagrammes présentés au chapitre de conception
Flutter SDK (Dart 3.13)	Compilation et exécution du client mobile
Xcode	Signature, simulation et déploiement iOS du projet mobile/

Les tests automatisés PHPUnit s'exécutent sur une base SQLite en mémoire. Ils ne dépendent pas de l'instance MySQL de XAMPP.

Technologies

Le socle web repose sur PHP 8.5.5 et Laravel 13. Les vues sont rendues en Blade. L'interface web visible utilise Bootstrap 5.3 et Bootstrap Icons (CDN), plus une feuille de styles applicative. Les graphiques d'administration s'appuient sur Chart.js (CDN). Aucune compilation front n'est nécessaire. L'authentification de session (navigateur) et l'authentification par jeton (application mobile) s'appuient sur Laravel Sanctum. Les tests d'acceptation HTTP sont écrits avec PHPUnit 12. Les données métier sont persistées dans MySQL.

Le client mobile est une application Flutter (SDK Dart 3.13). L'état applicatif est géré avec Riverpod, la navigation avec GoRouter, les appels HTTP avec Dio. Le jeton Sanctum est stocké dans flutter_secure_storage. Les graphiques d'administration utilisent fl_chart. La typographie s'appuie sur google_fonts. L'interface est bilingue (français / anglais) grâce à intl et flutter_localizations.

Un même jeu de contrôleurs dessert le navigateur et le mobile (§3.1.1). La figure 4.1 montre la mise en œuvre de respond(). Lorsque le client envoie Accept: application/json (cas du mobile via Dio), la réponse est sérialisée. Sinon, l'utilisateur web reçoit la page Blade ou la redirection prévue.

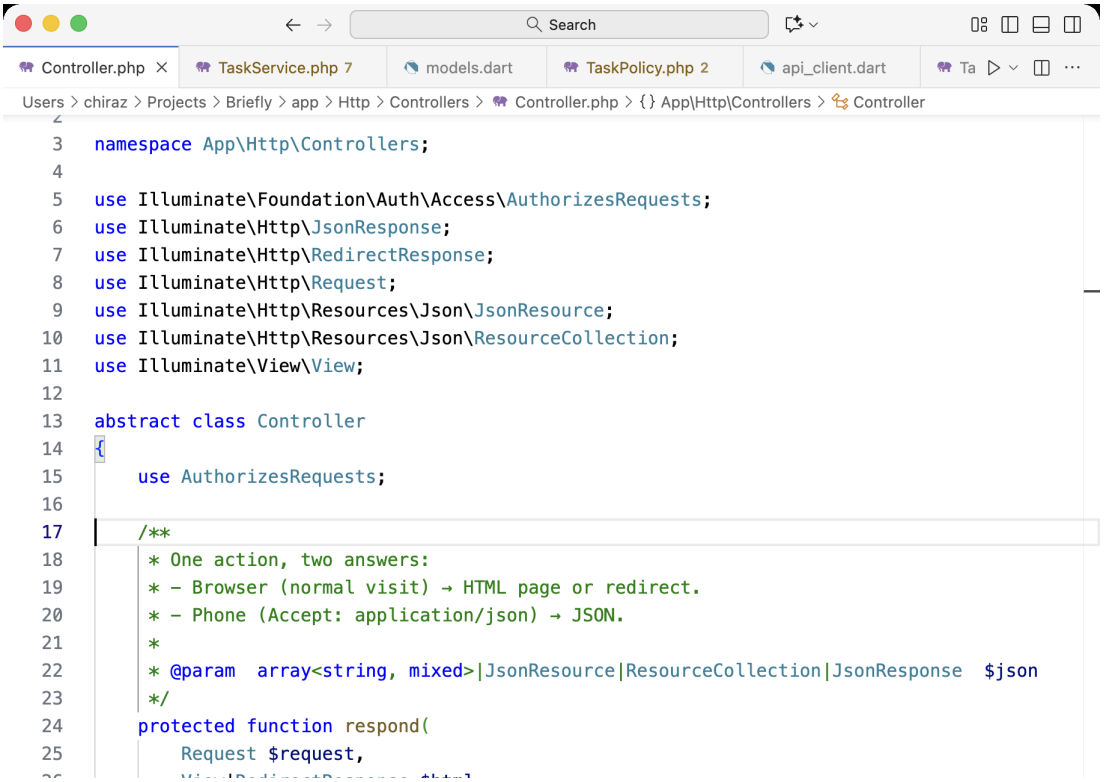


Figure 4.1 – Méthode respond() (app/Http/Controllers/Controller.php)

TaskService::recordProgress enregistre un point d'avancement. La méthode met à jour le statut de la tâche, y compris la date de clôture lorsque le statut passe à « terminé », et crée une entrée d'historique TaskUpdate. Le mobile et le web empruntent ce même service.

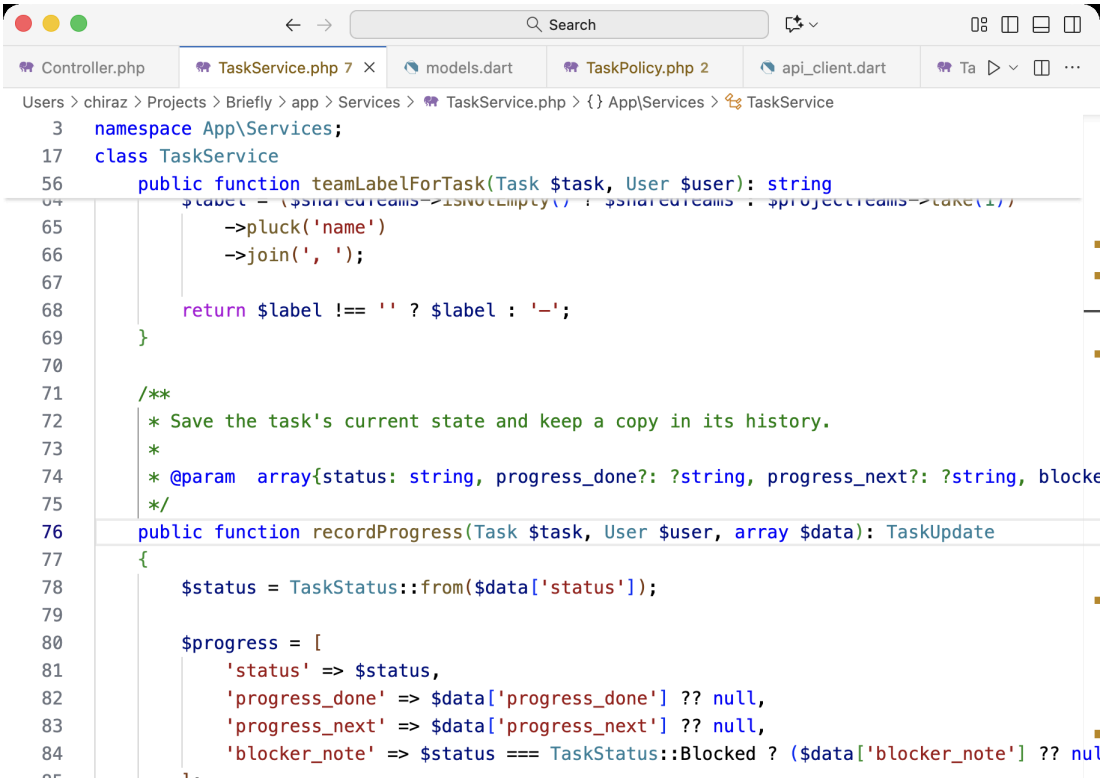


Figure 4.2 – Méthode recordProgress() (app/Services/TaskService.php)

La politique TaskPolicy::updateStatus limite la mise à jour d'avancement à l'assigné. Le client Flutter n'a pas à réimplémenter cette règle : il appelle POST /login avec device_name égal à flutter, puis envoie le jeton.

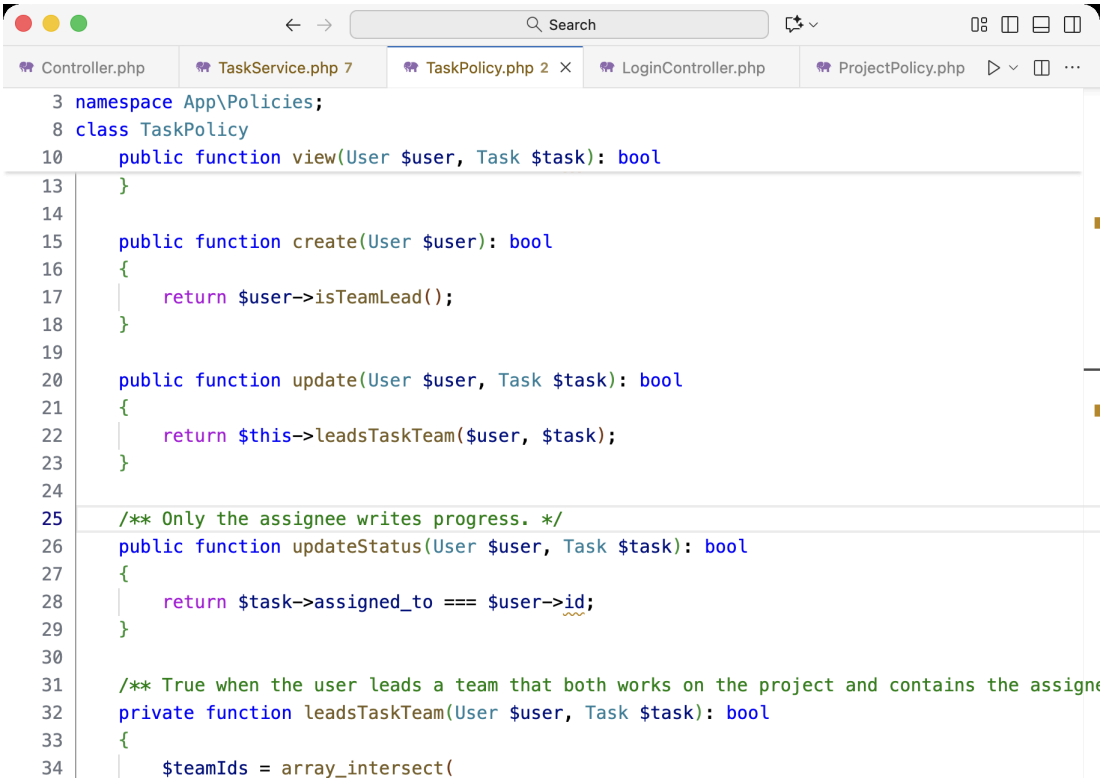


Figure 4.3 – Politique updateStatus (app/Policies/TaskPolicy.php)

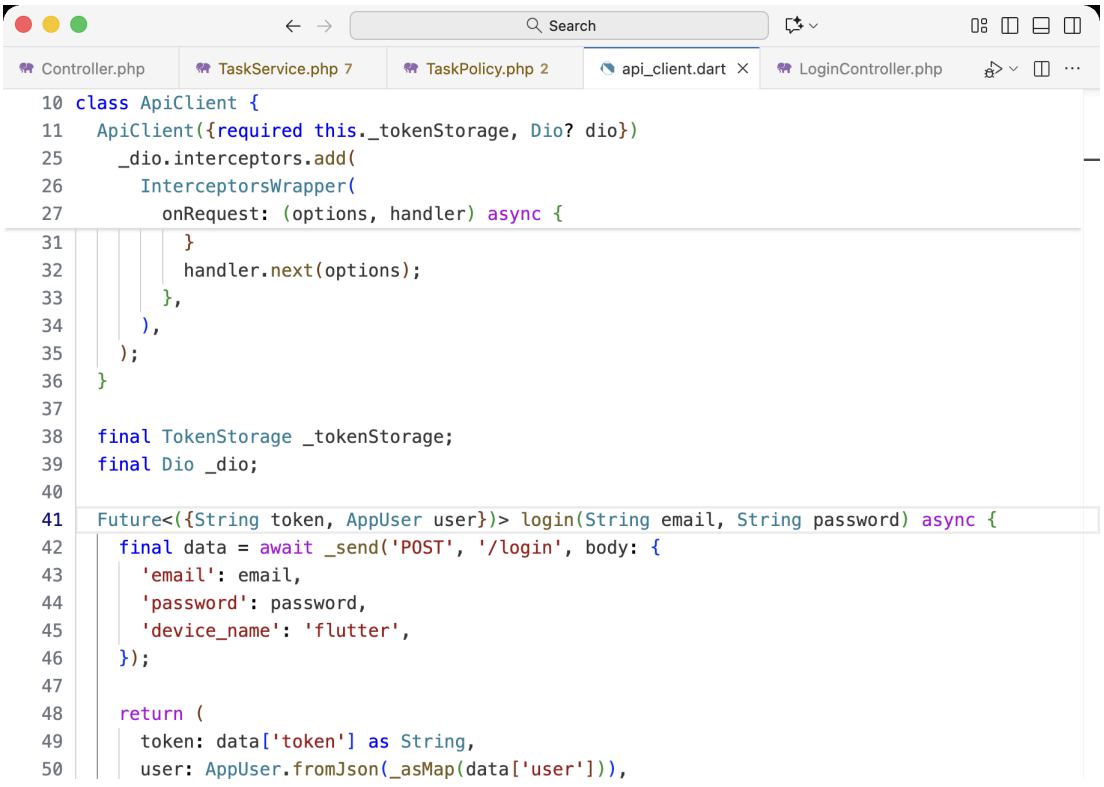


Figure 4.4 – Connexion JSON du client Flutter (mobile/lib/core/api_client.dart)

Le pourcentage affiché sur les tableaux de bord n'est pas un champ saisi. TaskStatus::progressPercent() le dérive du statut : 0 % à faire, 60 % en cours, 30 % bloqué, 100 % terminé.

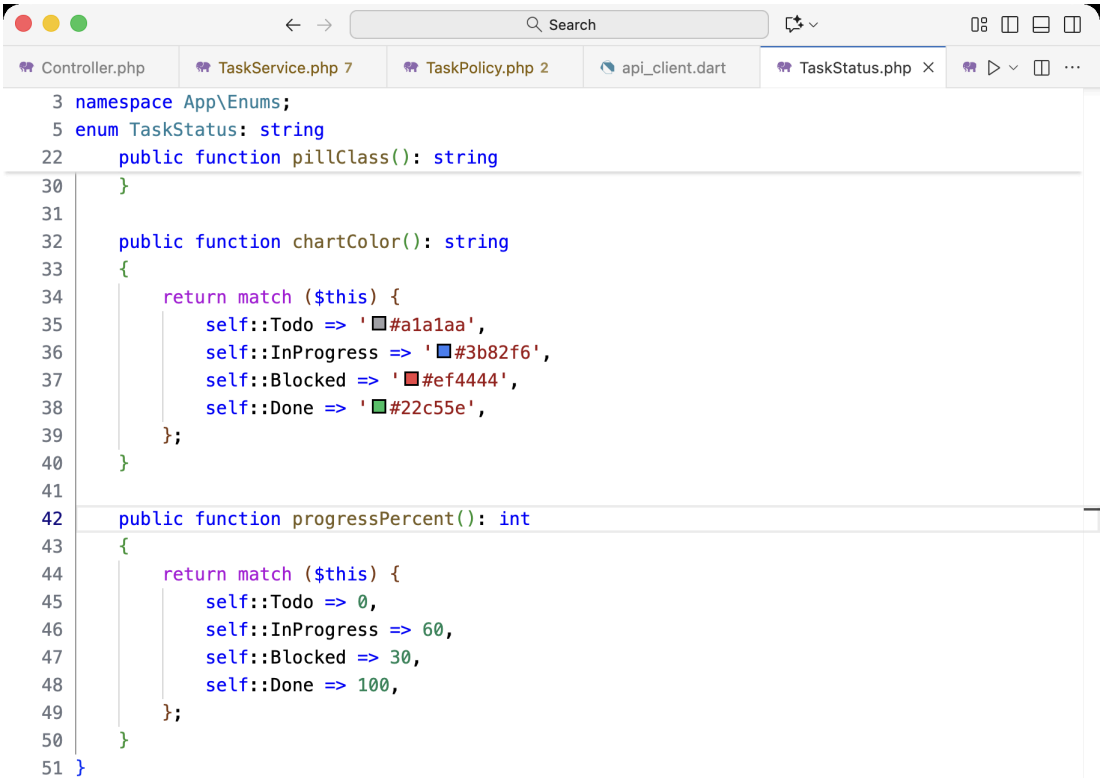


Figure 4.5 – Pourcentage d'avancement dérivé du statut (app/Enums/TaskStatus.php)

Un compte employé ne peut pas être créé sans équipe. StoreUserRequest impose team_ids avec au moins un identifiant. C'est la règle qui justifie le « include » de la figure 2.2.

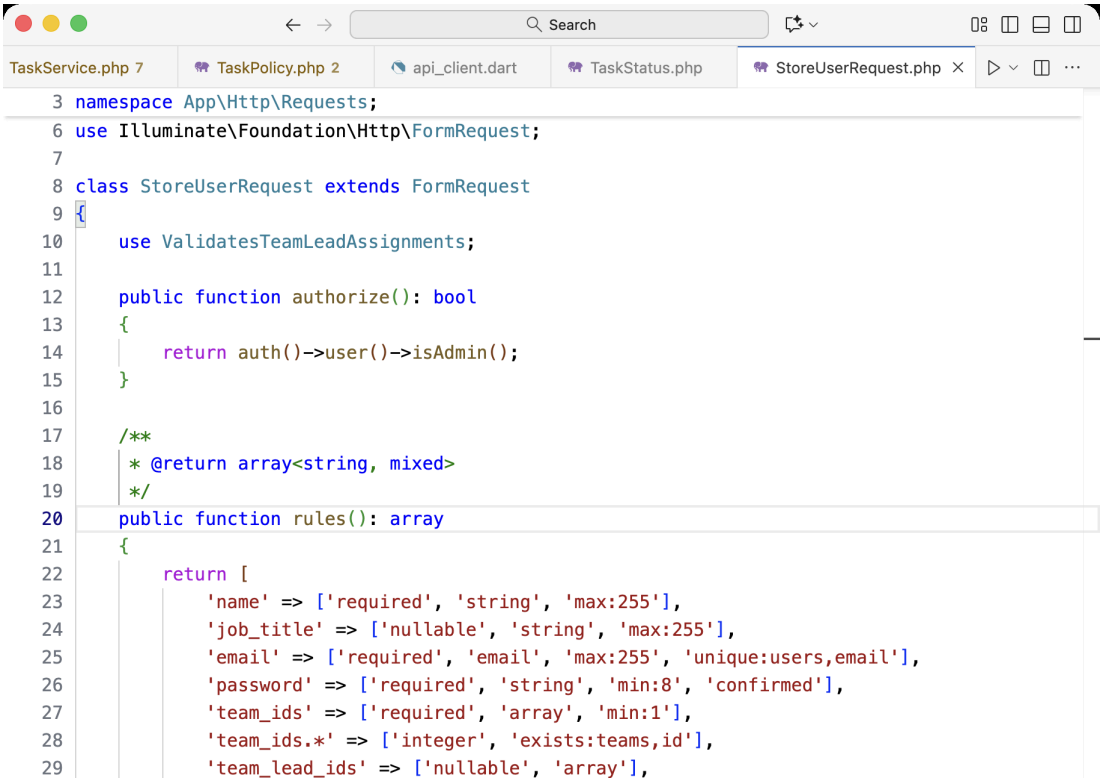


Figure 4.6 – Validation de la création d'un utilisateur (app/Http/Requests/StoreUserRequest.php)

La note de blocage n'est exigée que si le statut vaut BLOCKED. Rule::requiredIf porte cette condition. C'est la règle qui justifie le « extend » de la figure 2.4.

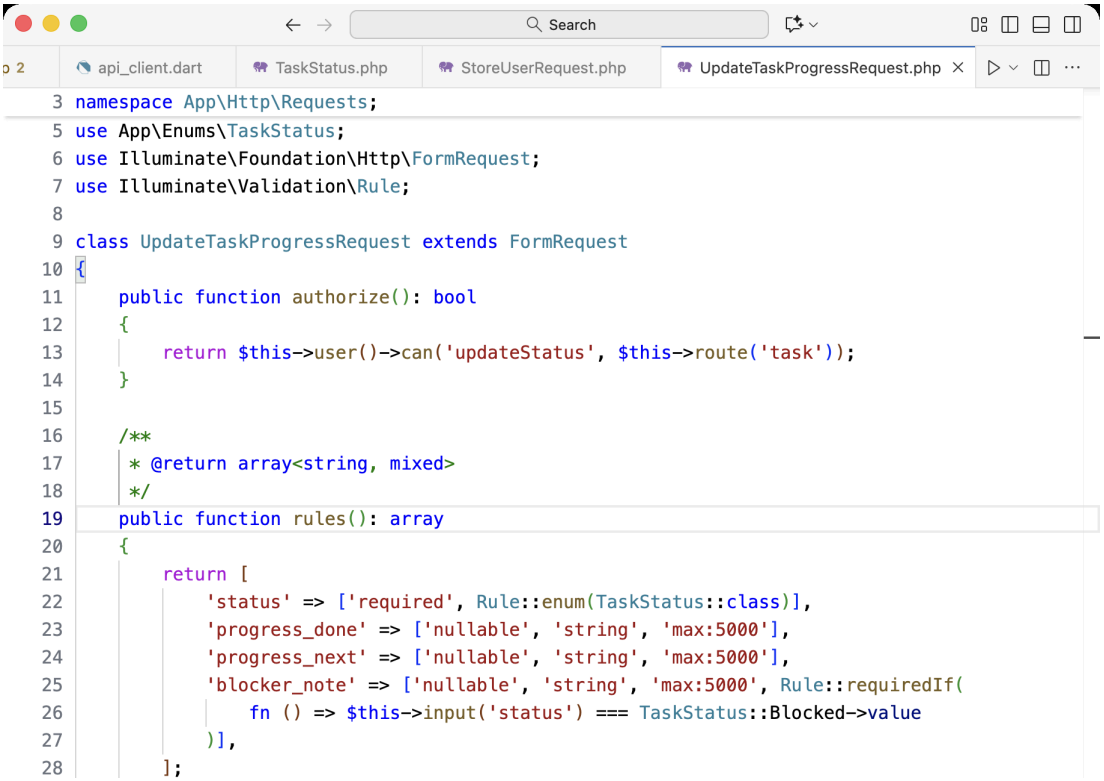


Figure 4.7 – Note de blocage obligatoire si le statut est BLOCKED (app/Http/Requests/UpdateTaskProgressRequest.php)

Tableau 4.3 – Principales technologies

Couche	Technologies
Serveur web	PHP 8.5.5, Laravel 13, Blade, Bootstrap 5.3, Chart.js, Laravel Sanctum, PHPUnit 12
Données	MySQL
Client mobile	Flutter (Dart 3.13), Riverpod, GoRouter, Dio, flutter_secure_storage, fl_chart, google_fonts, intl / flutter_localizations

Les migrations créent le schéma physique à partir du modèle logique du tableau 3.3.

Tableau 4.11 – Tables physiques

Élément logique	Table
Compte	users
Équipe	teams
Appartenance	team_user
Projet	projects
Rattachement projet–équipe	project_team
Tâche	tasks
Avancement	task_updates

Élément logique	Table
Session web	sessions
Réinitialisation du mot de passe	password_reset_tokens
Jeton d'accès mobile	personal_access_tokens

4.2 Réalisation des sprints

Nous avons découpé le travail en six sprints, selon le tableau 2.1. Les tableaux de tests qui suivent transposent, en scénarios fonctionnels, les cas réellement exercés par PHPUnit (AuthenticationTest, AdminTest, ProjectTaskTest, JsonEndpointTest, LocaleTest). La colonne « Résultat » indique la conformité constatée lors de l'exécution de la suite. Les parcours de réinitialisation du mot de passe, de mise à jour du profil, de l'option « se souvenir de moi » et du thème sont implémentés mais **non couverts** par cette suite.

Sprint 1 : Authentification, profil, préférences

Un visiteur atteint /login. Après authentification, un administrateur est envoyé vers /admin. Un membre ou un chef d'équipe rejoint /dashboard.

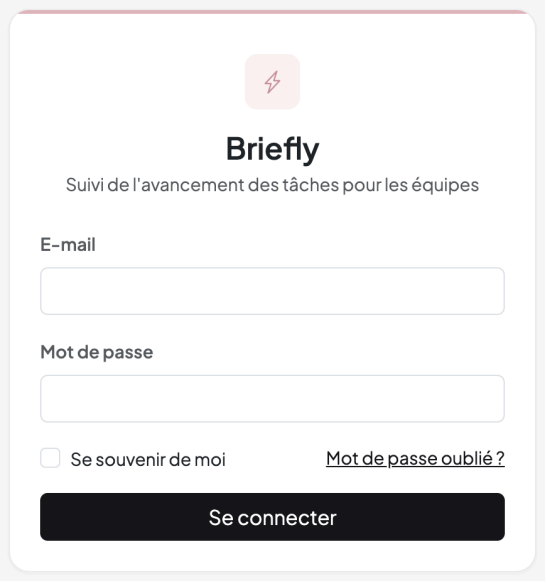


Figure 4.8 – Page de connexion web (/login)

Le formulaire de demande de lien de réinitialisation n'exige que l'adresse du compte. La page suivante, atteinte depuis le courriel, permet de saisir le nouveau mot de passe à l'aide du jeton.

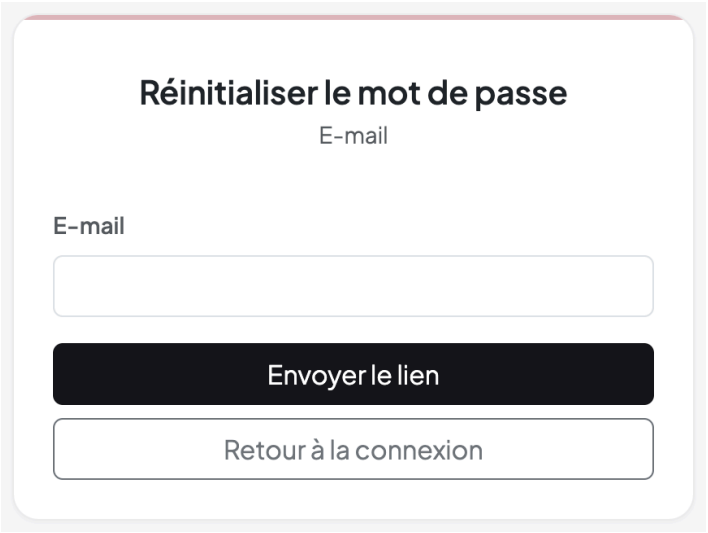


Figure 4.9 – Demande de réinitialisation du mot de passe (/forgot-password)

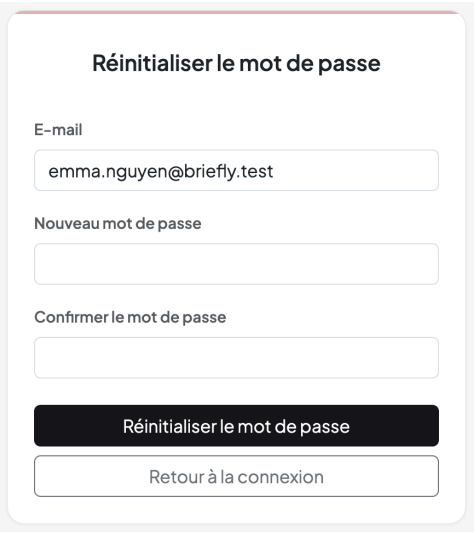


Figure 4.10 – Saisie du nouveau mot de passe (/reset-password/{token})

Le profil rassemble deux formulaires. L'un porte sur l'identité (nom, courriel), l'autre sur le changement de mot de passe (mot de passe actuel, nouveau mot de passe, confirmation). L'utilisateur y met à jour le nom et le courriel. L'intitulé de poste reste réservé à l'administration. Le rôle et l'équipe principale y sont affichés, sans pouvoir être modifiés par l'utilisateur.

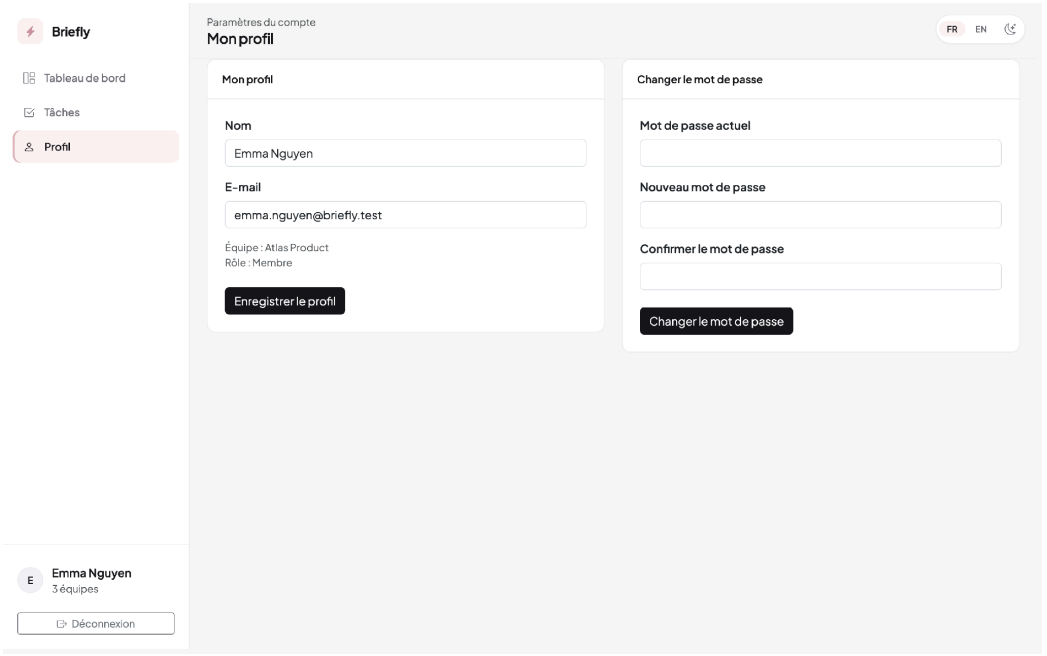


Figure 4.11 – Profil et changement de mot de passe (/profile)

La barre de préférences (langue FR/EN et bascule de thème) est visible sur toutes les pages, y compris la connexion et la récupération du mot de passe.



Figure 4.12 – Sélecteur de langue et bascule de thème (briefly-theme)

Tableau 4.4 – Tests fonctionnels du sprint 1

Cas de test	Démarche	Comportement attendu	Résultat
Affichage de la connexion	Ouvrir /login sans session	La page s'affiche	Conforme
Connexion membre	Saisir un courriel et un mot de passe valides d'un membre	Redirection vers /dashboard. Session authentifiée	Conforme
Connexion administrateur	Saisir les identifiants d'un administrateur	Redirection vers /admin	Conforme
Accès refusé au visiteur	Ouvrir /dashboard sans être connecté	Redirection vers /login	Conforme
Identifiants JSON invalides	Poster /login en JSON avec un mot de passe erroné	Réponse 422. Message d'erreur sur le courriel	Conforme
Connexion JSON	Poster /login en JSON avec device_name	Réponse 200 contenant token et user	Conforme
Langue française	Appeler /locale/fr puis ouvrir la connexion	Session locale=fr. Libellé « Se connecter »	Conforme
Langue anglaise	Depuis une session française, appeler /locale/en	Session locale=en. Libellé « Sign in »	Conforme
Locale invalide	Appeler /locale/de	Réponse 404	Conforme

Sprint 2 : Espace membre

Le tableau de bord /dashboard affiche un indicateur de taux d'avancement personnel et la liste des tâches assignées.

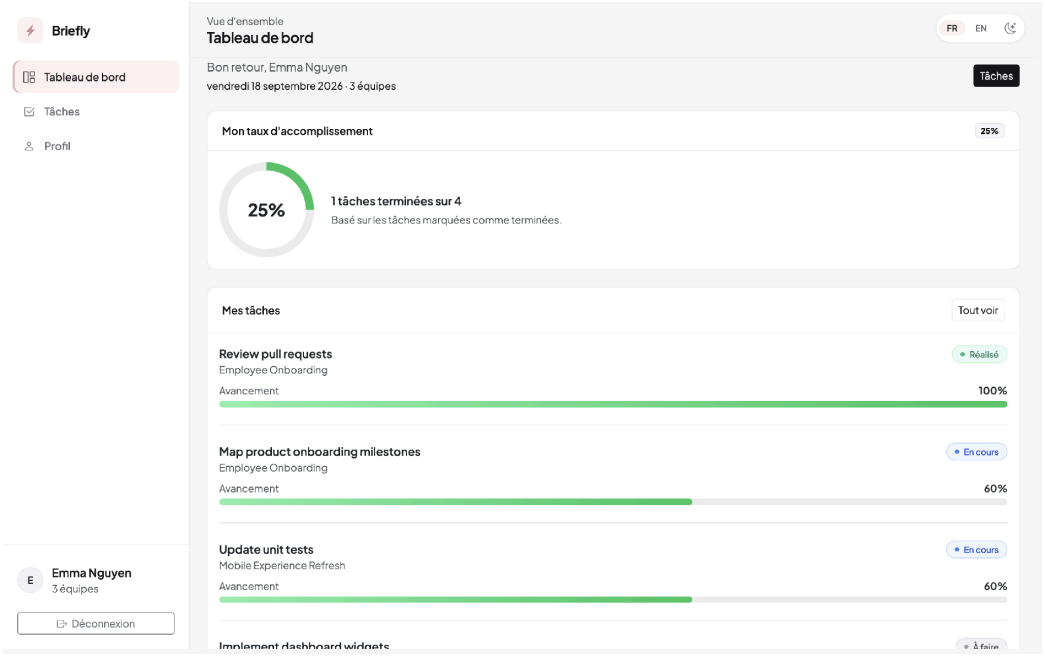


Figure 4.13 – Tableau de bord membre : anneau d'avancement et tâches personnelles (/dashboard)

La liste /tasks présente les tâches assignées, triées et regroupées selon le statut (bloqué, en cours, à faire, terminé), avec le projet, l'équipe et la date de mise à jour. Un raccourci mène à l'historique.

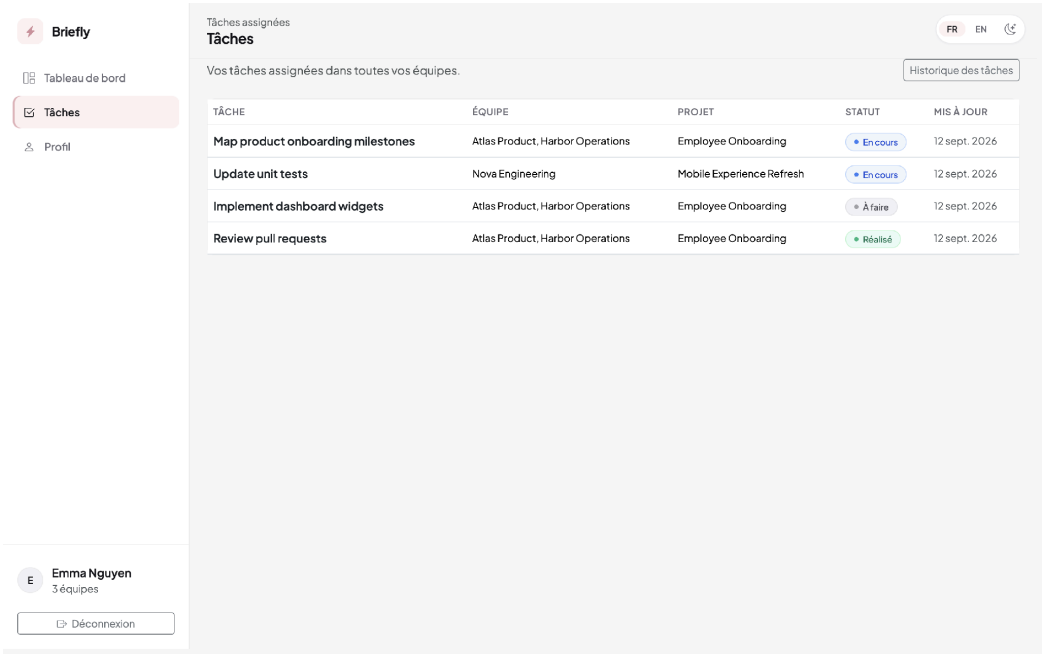


Figure 4.14 – Liste des tâches personnelles (/tasks)

Le détail d'une tâche assignée propose le formulaire de progression du §2.1.2. Un membre ne peut pas modifier la tâche d'un collègue.

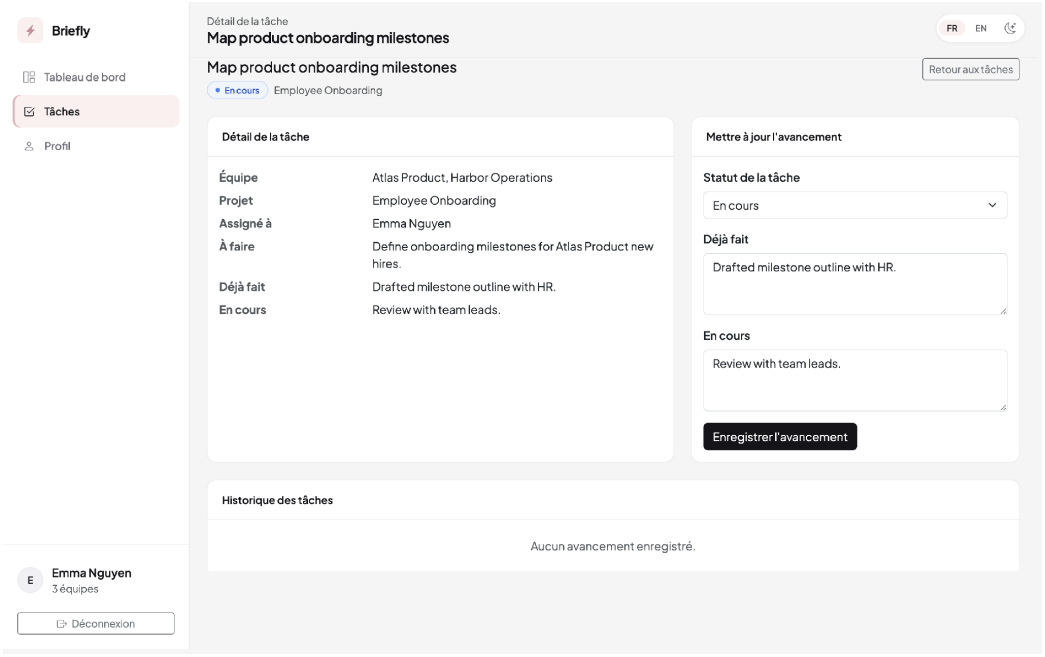


Figure 4.15 – Détail d'une tâche et formulaire d'avancement (/tasks/{id})

L'historique paginé des points d'avancement se trouve sur /tasks/history. Toute tentative d'accès à /projects est refusée.

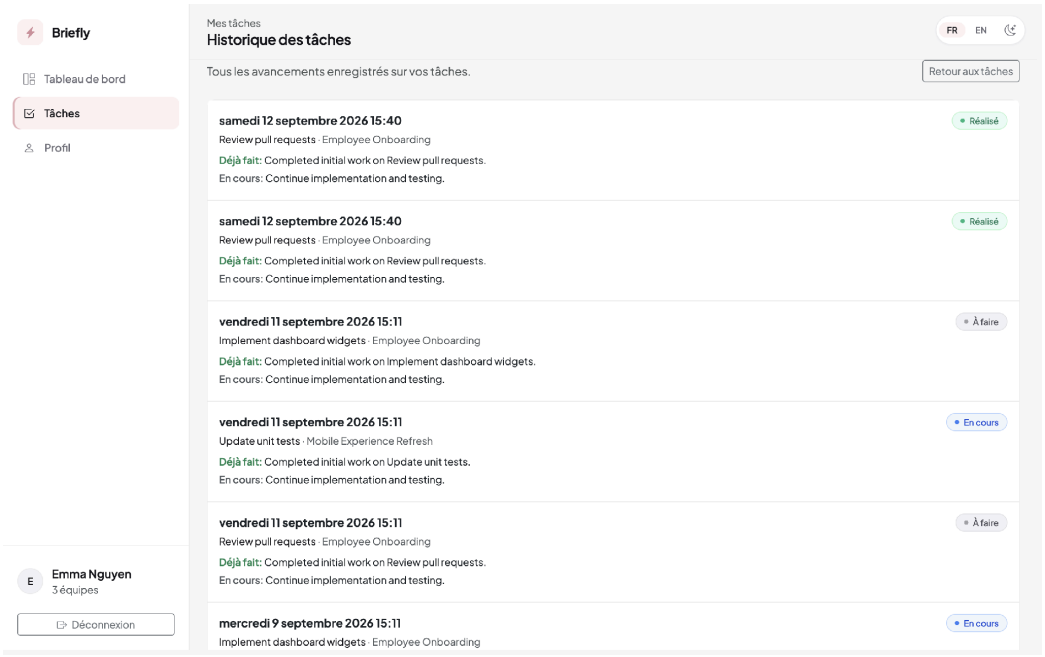


Figure 4.16 – Historique des points d'avancement (/tasks/history)

Tableau 4.5 – Tests fonctionnels du sprint 2

Cas de test	Démarche	Comportement attendu	Résultat

Tableau de bord membre	Se connecter en membre et ouvrir /dashboard	Affichage du taux d'avancement et des tâches personnelles	Conforme
Liste des tâches	Ouvrir /tasks en tant que membre assigné	Les titres et projets des tâches assignées sont visibles	Conforme
Détail et formulaire	Ouvrir /tasks/{id} pour une tâche assignée	La page s'affiche avec le formulaire « Mettre à jour l'avancement »	Conforme
Enregistrement d'un avancement	Envoyer un statut « en cours » avec le réalisé et le suivant	Redirection vers le détail. Tâche et task_updates mis à jour	Conforme
Blocage sans note	Passer au statut « bloqué » sans blocker_note	Erreur de validation sur la note de blocage	Conforme
Marquer terminé	Passer au statut « terminé »	Statut DONE et completed_at renseigné	Conforme
Tâche d'un tiers	Tenter de mettre à jour une tâche assignée à un autre membre	Réponse 403	Conforme
Historique	Enregistrer un avancement puis ouvrir /tasks/history	Le titre et le texte de progression apparaissent	Conforme
Plusieurs mises à jour	Enregistrer deux avancements successifs sur la même tâche	Deux lignes dans task_updates	Conforme
Accès projets refusé	Ouvrir /projects en tant que membre	Réponse 403	Conforme

Sprint 3 : Espace chef d'équipe

Sur le web, les cartes d'équipe affichent le total, les tâches en cours, les bloquées et les terminées dans la semaine, plus le taux d'avancement et les dernières tâches bloquées.

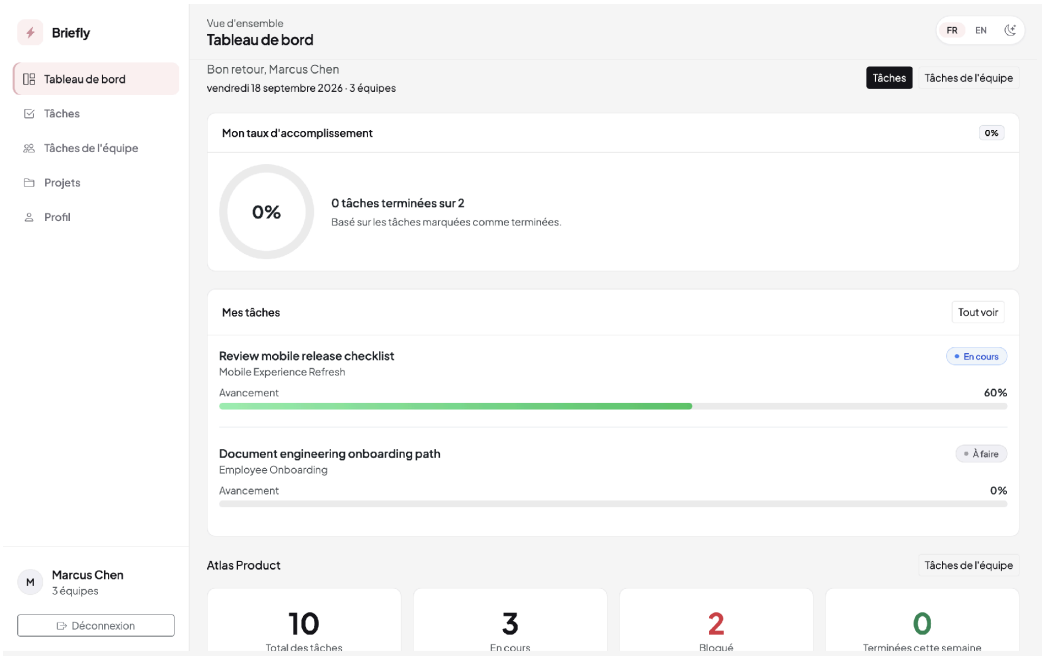


Figure 4.17 – Tableau de bord chef d'équipe : cartes des équipes dirigées (/dashboard)

La liste des projets montre les projets rattachés aux équipes dirigées, y compris lorsqu'un projet est partagé entre plusieurs équipes. Le chef d'équipe ne crée pas de projets. Le détail d'un projet liste les tâches de l'équipe et permet d'en créer une, assignée exclusivement à un membre de cette équipe. Le chef peut aussi modifier une tâche existante.

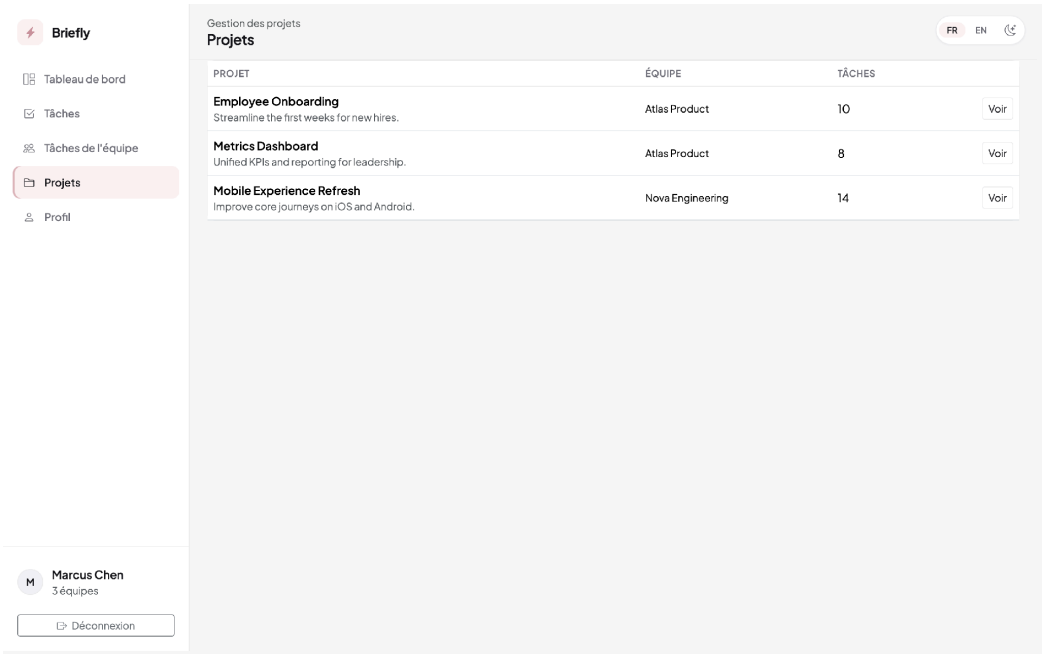


Figure 4.18 – Liste des projets du chef d'équipe (/projects)

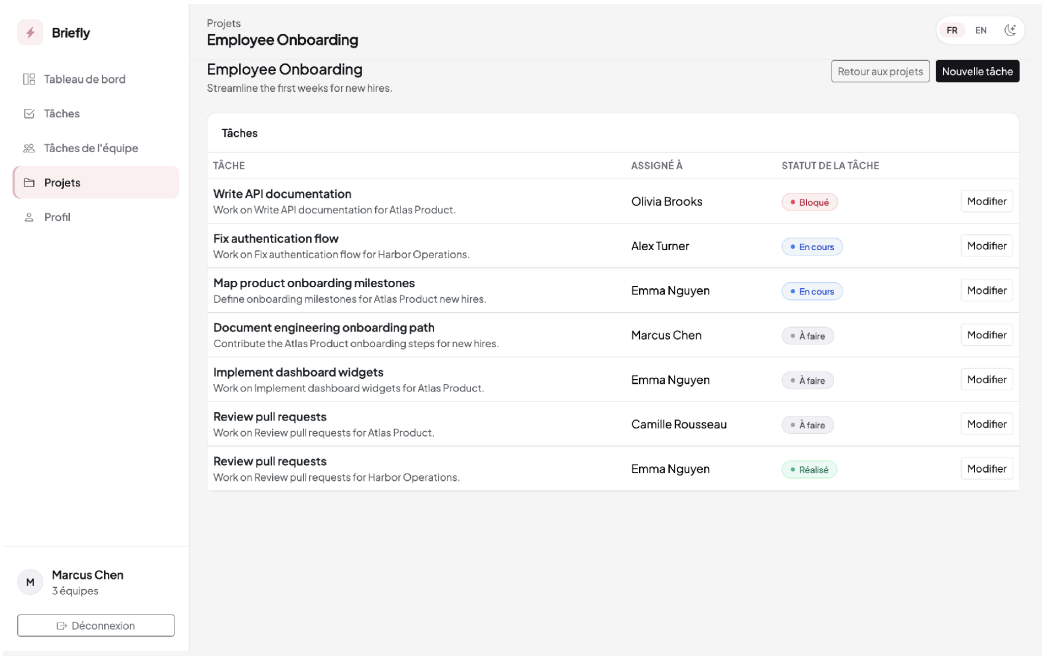


Figure 4.19 – Détail d'un projet et tâches de l'équipe (/projects/{id})

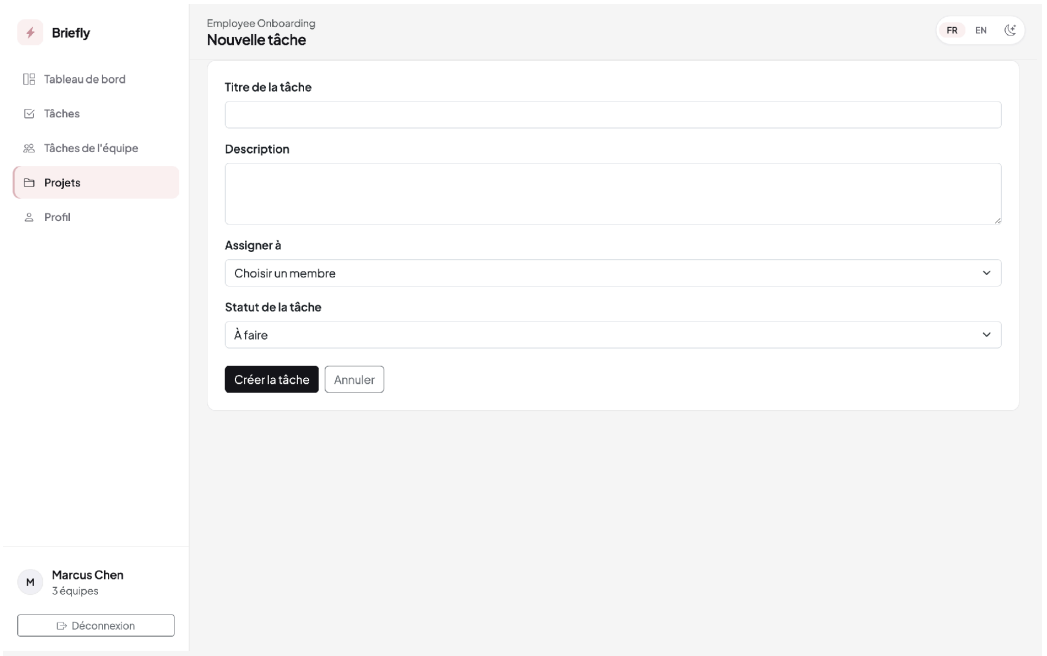


Figure 4.20 – Création ou édition d'une tâche par le chef d'équipe

La page /team/tasks combine des compteurs, un filtre par statut (tous, bloqué, en cours, à faire, terminé) et un tableau (membre, tâche, projet, avancement, blocage). Lorsqu'un chef dirige plusieurs équipes, une colonne « Équipe » distingue les lignes. La fiche membre rassemble les tâches assignées et l'historique d'avancement de la personne.

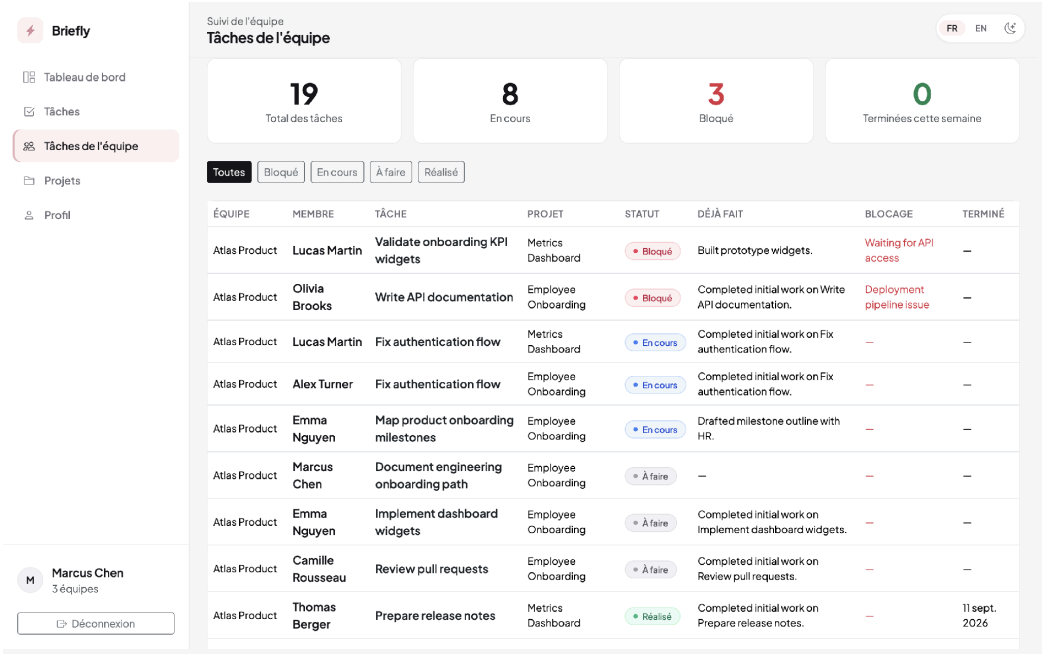


Figure 4.21 – Suivi des tâches d'équipe avec filtre de statut (/team/tasks)

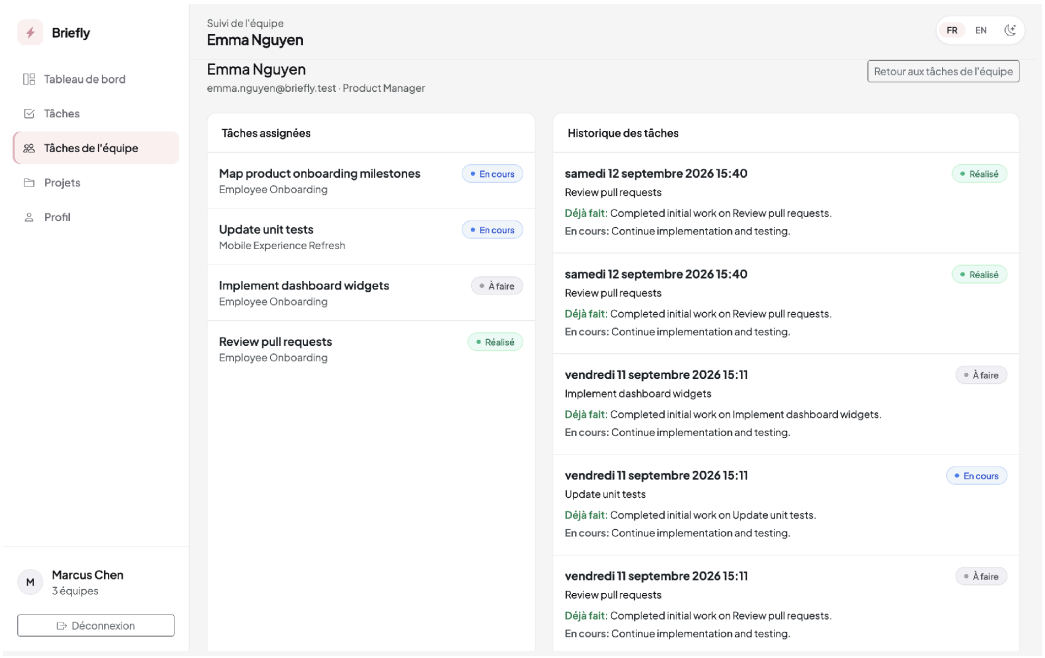


Figure 4.22 – Fiche d'un membre d'équipe (/team/members/{user})

Tableau 4.6 – Tests fonctionnels du sprint 3

Cas de test	Démarche	Comportement attendu	Résultat
Création de projet interdite	Poster une création de projet en tant que chef d'équipe	Réponse 403. Aucune ligne projet créée	Conforme

Création de tâche	Créer une tâche pour un membre de l'équipe, sur un projet rattaché	Redirection vers le projet. Tâche persistée	Conforme
Assignation hors équipe	Assigner la tâche à un membre d'une autre équipe	Erreur de validation sur assigned_to	Conforme
Liste des projets	Ouvrir /projects en tant que chef	Les projets des équipes dirigées s'affichent	Conforme
Projet partagé	Diriger deux équipes liées au même projet	Les deux noms d'équipe apparaissent sur l'index	Conforme
Tâches d'équipe	Ouvrir /team/tasks	Les tâches des membres et leurs noms sont visibles	Conforme
Plusieurs équipes	Chef de deux équipes, ouvrir /team/tasks	Colonne « Équipe » et tâches des deux équipes	Conforme
Tableau de bord chef	Ouvrir /dashboard avec des tâches d'équipe	Blocs « Avancement des membres » et « Avancement des projets »	Conforme
Tâches personnelles du chef	Ouvrir /tasks en tant que chef	Uniquement ses propres tâches. Pas celles des membres	Conforme
Consultation sans formulaire	Ouvrir le détail d'une tâche d'un membre	La page s'affiche sans le formulaire d'avancement	Conforme
Drapeau chef absent	Rôle chef d'équipe mais is_team_lead faux sur le pivot	Accès à /projects refusé (403)	Conforme

Sprint 4 : Espace administrateur

L'écran /admin présente les effectifs, le taux d'avancement global, les listes d'avancement par projet et par équipe, ainsi qu'un graphique de répartition des statuts (Chart.js). Un membre ne peut pas accéder à ces routes. Un administrateur qui ouvre /tasks est renvoyé vers /admin.

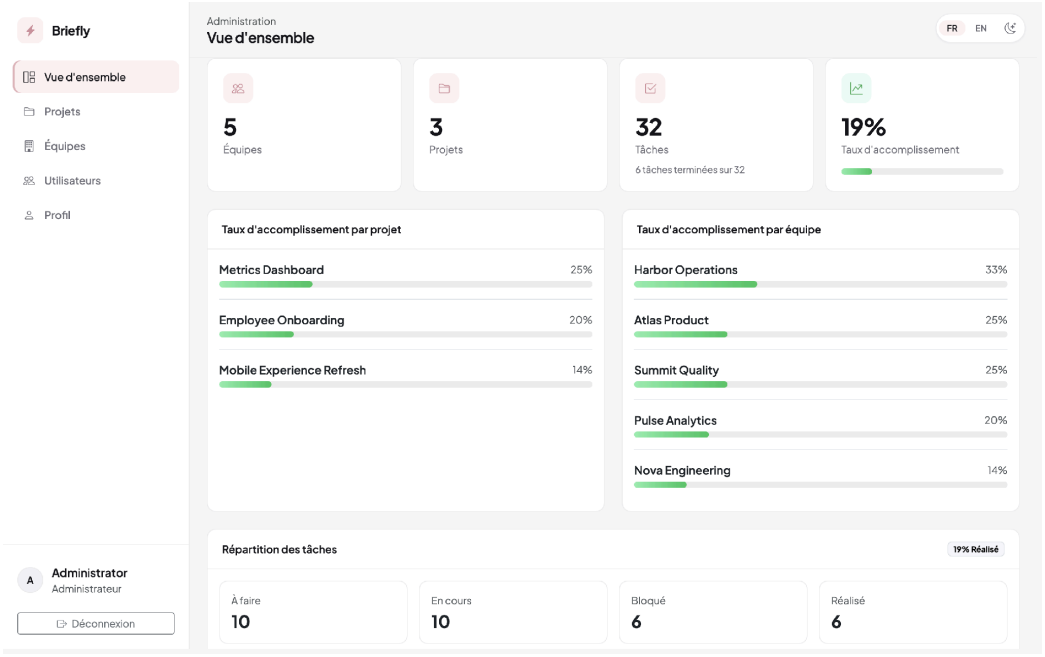


Figure 4.23 – Tableau de bord administrateur : statistiques, achèvement et graphiques (/admin)

Les pages utilisateurs, équipes et projets permettent de créer et de mettre à jour ces entités. La gestion des utilisateurs liste les comptes, autorise la création (nom, courriel, mot de passe, équipes) et la mise à jour (équipes d'appartenance, drapeaux chef d'équipe). La gestion des équipes permet de nommer l'équipe, d'y rattacher des membres, de désigner un chef et d'associer des projets. Un chef ne peut pas être retiré de sa dernière équipe. La gestion des projets crée ou met à jour un projet et synchronise les équipes associées.

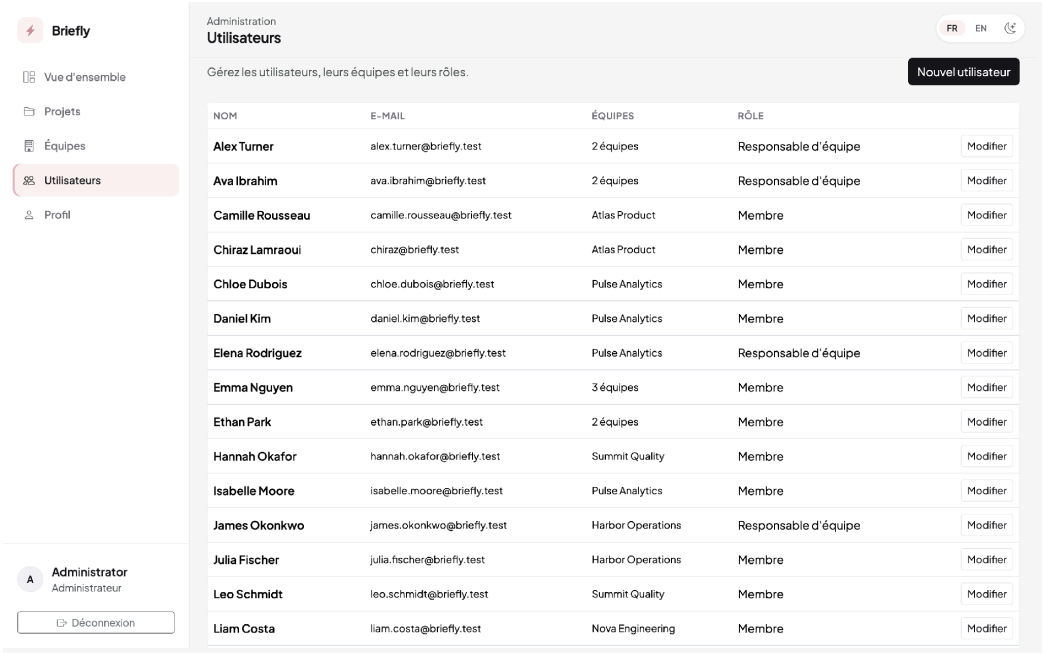


Figure 4.24 – Gestion des utilisateurs (/admin/users)

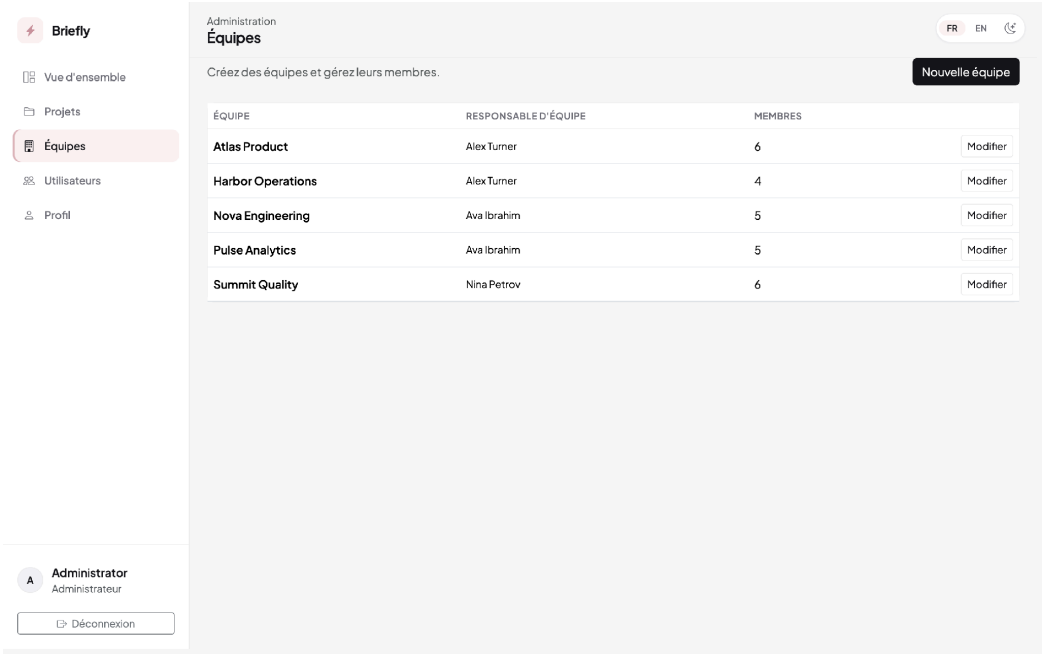


Figure 4.25 – Gestion des équipes (/admin/teams)

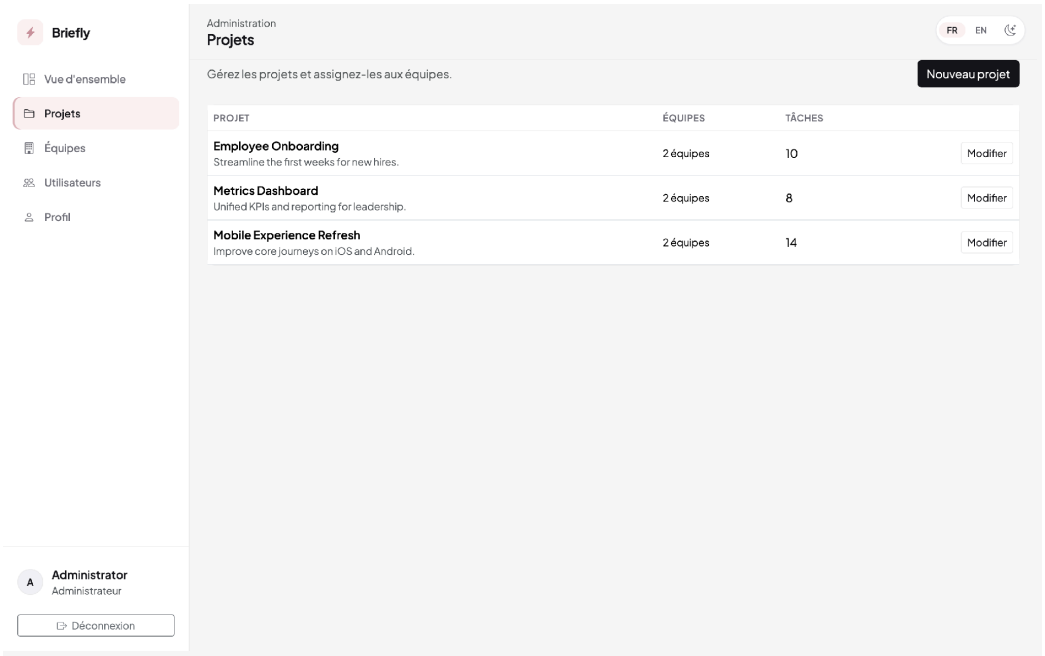


Figure 4.26 – Gestion des projets (/admin/projects)

Tableau 4.7 – Tests fonctionnels du sprint 4

Cas de test	Démarche	Comportement attendu	Résultat
Tableau de bord admin	Se connecter en administrateur et ouvrir /admin	Affichage de la vue d'ensemble, du taux d'avancement, du graphique de statuts et des listes par équipe et par projet	Conforme

Membre exclu	Ouvrir /admin en tant que membre	Réponse 403	Conforme
Redirection de l'admin	Ouvrir /tasks en tant qu'administrateur	Redirection vers /admin	Conforme
Création et mise à jour d'un projet	Créer un projet multi-équipes puis n'en conserver qu'une	Redirections vers l'index puis l'édition. Équipes synchronisées	Conforme
Chef interdit de créer un projet	Poster /admin/projects en tant que chef d'équipe	Réponse 403	Conforme
Création d'équipe	Poster un nom d'équipe	L'équipe est enregistrée. Redirection vers l'index	Conforme
Équipe avec chef et membres	Créer une équipe en désignant un chef et un membre	Rôle chef, pivot is_team_lead, appartenances correctes	Conforme
Utilisateur multi-équipes	Créer un utilisateur avec deux équipes	Les deux appartenances sont persistées	Conforme
Membre interdit de créer un compte	Poster /admin/users en tant que membre	Réponse 403	Conforme
Assignation multi-équipes	Mettre à jour un membre avec deux team_ids	Appartenance aux deux équipes. Rôle membre conservé	Conforme
Chef de plusieurs équipes	Cocher deux team_lead_ids pour un utilisateur	Rôle chef d'équipe. Drapeaux vrais sur les deux équipes	Conforme
Retrait du dernier chef	Mettre à jour une équipe en vidant user_ids alors qu'un chef y est encore	Erreur de validation. Le chef reste rattaché	Conforme
URL d'équipe	Ouvrir /admin/teams/{id}	Redirection vers le formulaire d'édition	Conforme

Sprint 5 : Tests, sécurité et intégration continue

Après le client web (sprints 1 à 4), nous avons consolidé PHPUnit, formalisé la sécurité (§4.3) et branché la CI (§4.4), conformément au tableau 2.1 (27 avril au 25 juillet 2026).

Sprint 6 : Application mobile

Le client Flutter reprend les parcours du tableau 2.1. Il a été développé en septembre et début octobre 2026, en lien avec le module mobile en formation. Le jeton Sanctum est stocké de façon sécurisée. Un administrateur est redirigé de /dashboard et /tasks vers /admin. Le thème et la langue FR/EN se règlent dans le profil (stockage local). Ils ne changent pas la locale Laravel.

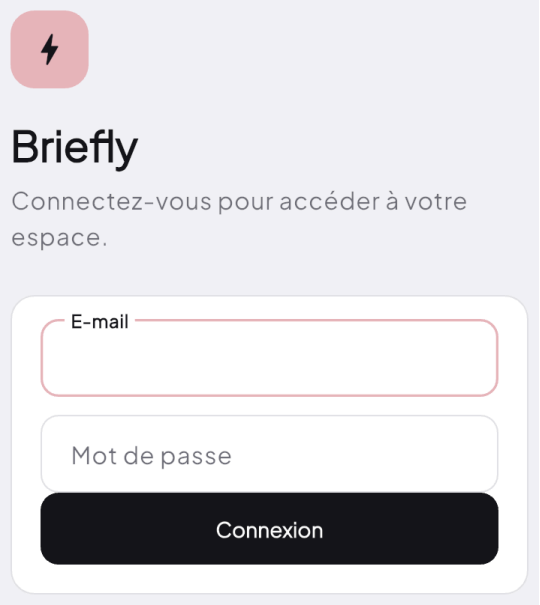


Figure 4.27 – Connexion mobile (LoginScreen)

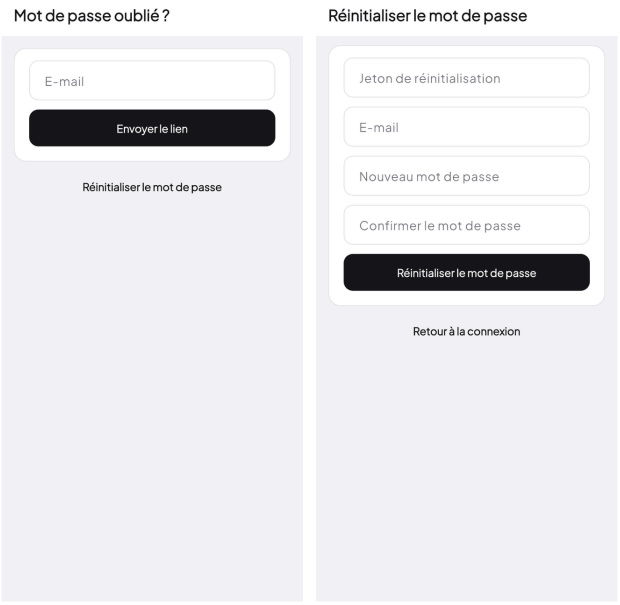


Figure 4.28 – Mot de passe oublié et réinitialisation (ForgotPasswordScreen, ResetPasswordScreen)

Sur le tableau de bord chef d'équipe, les pastilles mobiles affichent en cours / bloqué / terminé, sans le total ni « terminé cette semaine » du web.

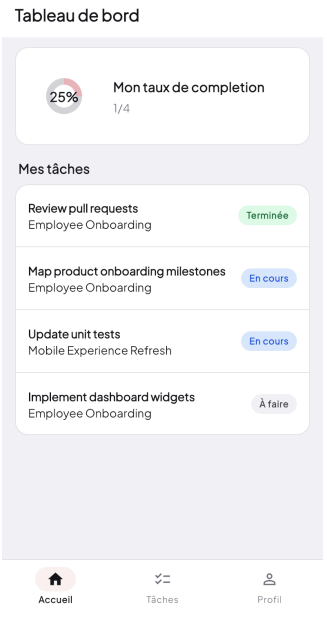


Figure 4.29 – Tableau de bord mobile : anneau d'achèvement et tâches (DashboardScreen)

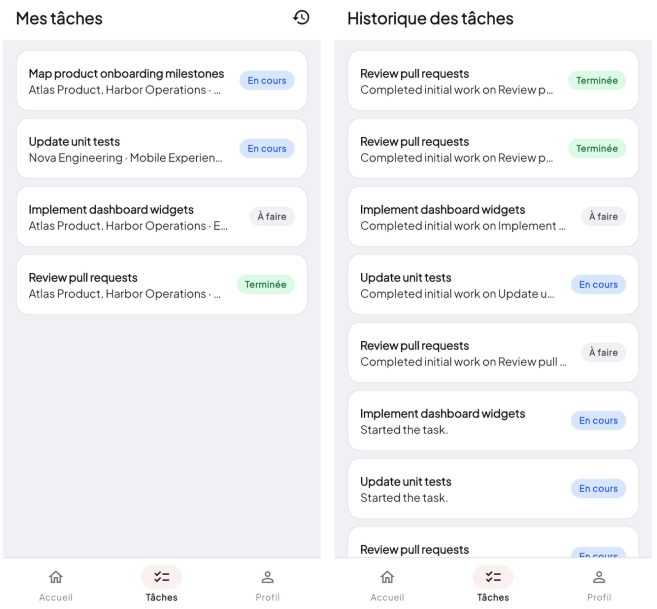


Figure 4.30 – Liste des tâches et historique (TaskListScreen, TaskHistoryScreen)

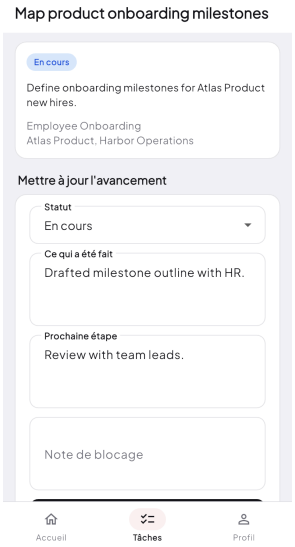


Figure 4.31 – Détail de tâche et saisie d'avancement (TaskDetailScreen)

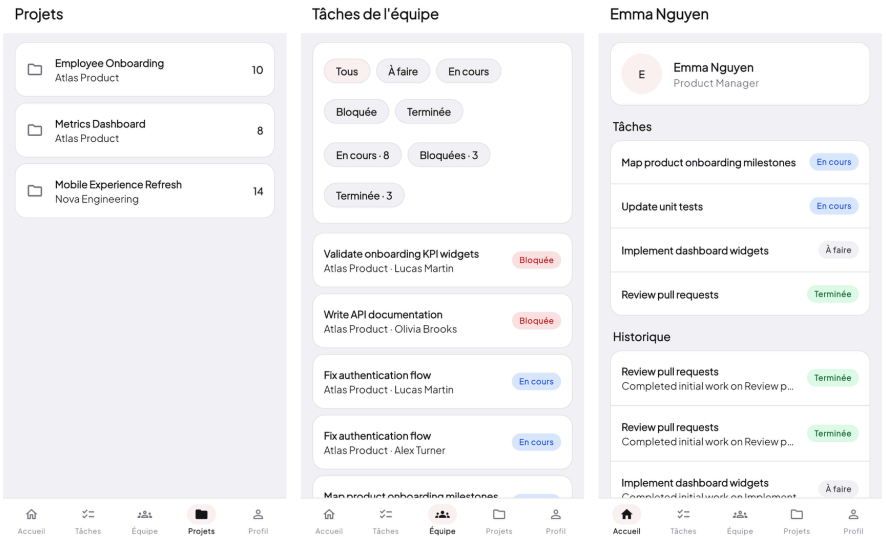


Figure 4.32 – Projets, tâches d'équipe et fiche membre (ProjectListScreen, TeamTasksScreen, TeamMemberScreen)

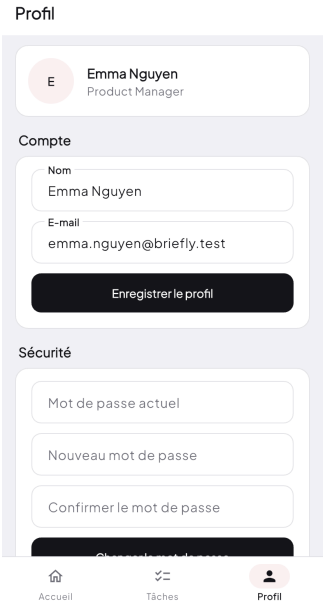


Figure 4.33 – Profil, thème et langue FR/EN (ProfileScreen)

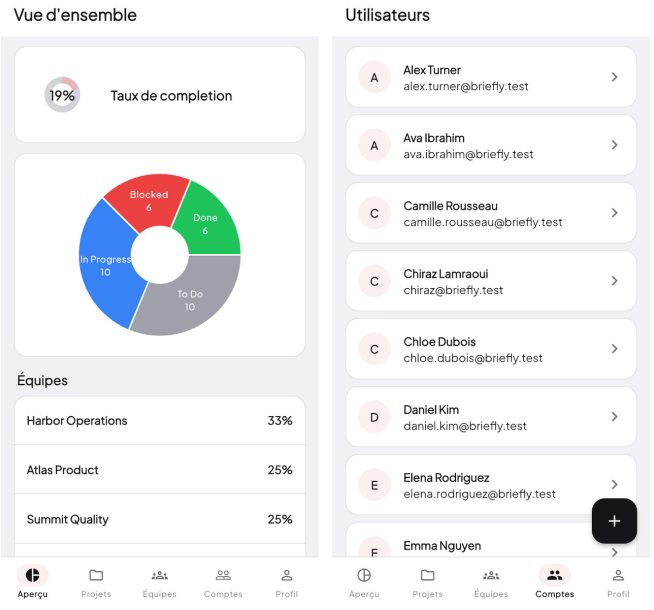


Figure 4.34 – Écrans d'administration mobile (AdminDashboardScreen et listes CRUD)

Les scénarios ci-dessous complètent le tableau 4.4. La connexion JSON y est déjà testée au sprint 1. Ils valident le contrat métier dont le client Flutter dépend.

Tableau 4.8 – Tests fonctionnels du sprint 6 (API JSON consommée par le mobile)

Cas de test	Démarche	Comportement attendu	Résultat

Tableau de bord sans jeton	GET JSON /dashboard sans Authorization	Réponse 401	Conforme
Avancement avec jeton	GET JSON du tableau de bord puis PATCH d'une tâche assignée	200. Statut et réalisé mis à jour	Conforme
Blocage JSON sans note	PATCH JSON statut « bloqué » sans note	Réponse 422 sur blocker_note	Conforme
Admin hors tâches membre	GET JSON /tasks avec un jeton administrateur	Réponse 403	Conforme
Membre hors projets	GET JSON /projects avec un jeton membre	Réponse 403	Conforme
Création de tâche JSON	Poster une tâche en tant que chef d'équipe	Réponse 201. Titre persisté	Conforme
Liste d'équipe JSON	GET JSON /team/tasks en tant que chef	Compteurs et fragment de titre présents	Conforme
CRUD admin JSON	Créer un utilisateur, une équipe et un projet en JSON	Réponses 201. Enregistrements en base	Conforme
Déconnexion	POST JSON /logout puis GET /dashboard	204. Jeton révoqué. Accès ultérieur 401	Conforme

4.3 Sécurité de l'application

Le chapitre 3 avait déjà placé l'autorisation sur le serveur. Nous détaillons ici le code retenu, puis les limites de la version actuelle.

4.3.1 Mots de passe

Le modèle User déclare le cast password => hashed (figure 4.35). Laravel hache donc le mot de passe avec bcrypt, à 12 tours (BCRYPT_ROUNDS), dès qu'il est affecté, à la création d'un compte comme lors d'un changement de mot de passe. Aucun mot de passe n'est stocké en clair. L'attribut #[Hidden] retire password et remember_token de toute sérialisation, y compris des réponses JSON envoyées au mobile.

Les formulaires imposent huit caractères au minimum et une confirmation : StoreUserRequest à la création d'un compte, NewPasswordController à la réinitialisation, UpdatePasswordRequest au changement depuis le profil. Ce dernier exige aussi le mot de passe actuel (règle current_password).

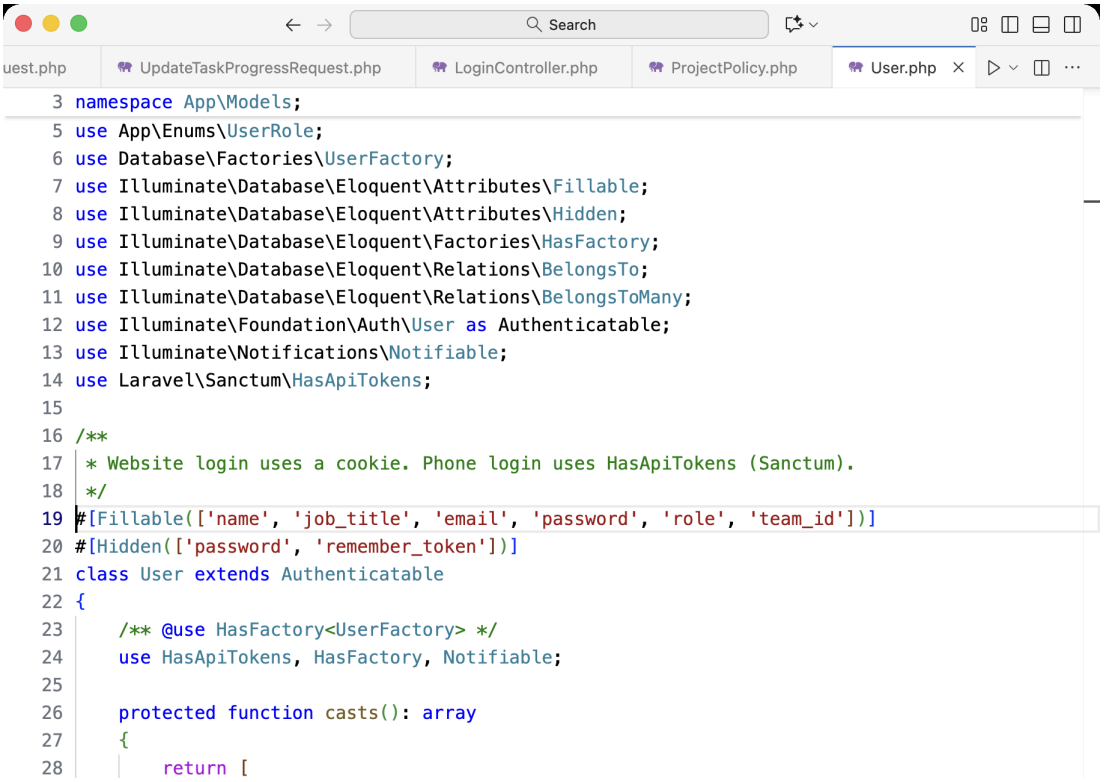


Figure 4.35 – Cast hashed et attributs masqués (app/Models/User.php)

4.3.2 Session et jeton

Après une connexion réussie, LoginController régénère l'identifiant de session, ce qui empêche la fixation de session. La déconnexion invalide la session et régénère le jeton CSRF.

Pour le mobile, la même méthode crée un jeton Sanctum (figure 4.36). Sanctum n'enregistre en base que l'empreinte SHA-256 du jeton, dans personal_access_tokens. Le texte clair n'est renvoyé qu'une fois, dans la réponse de connexion. Le client Flutter le range dans flutter_secure_storage, qui s'appuie sur le trousseau d'iOS. La déconnexion JSON supprime le jeton côté serveur.

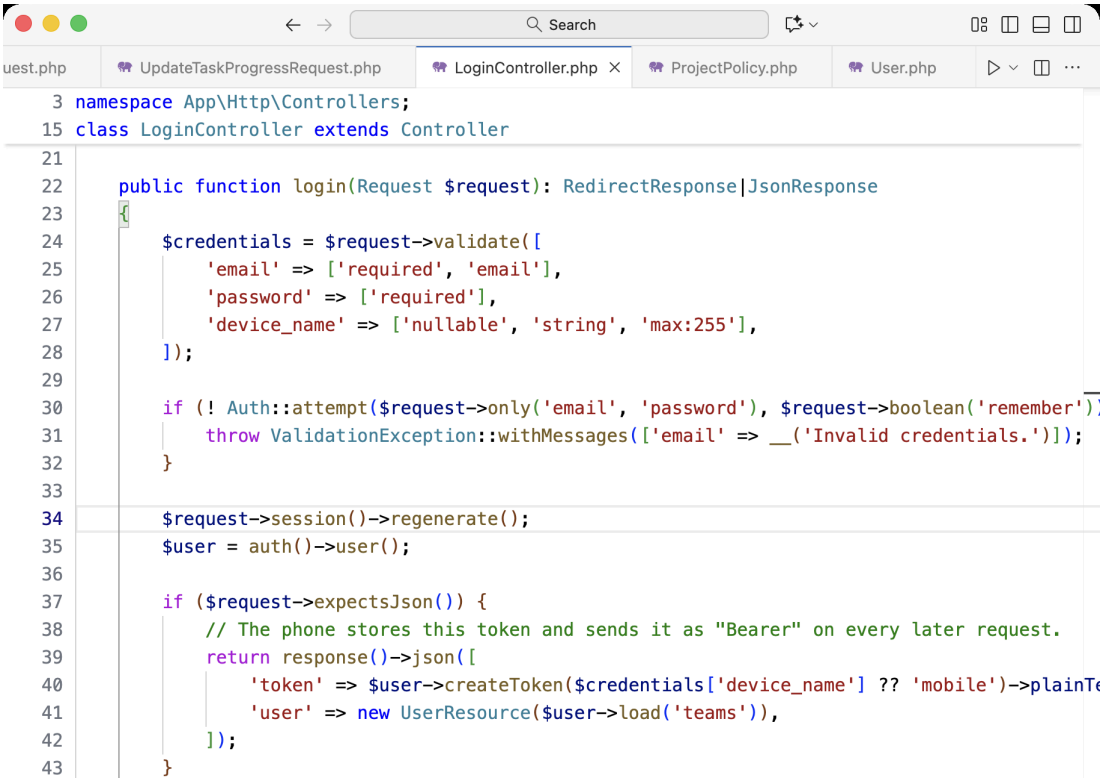


Figure 4.36 – Émission du jeton Sanctum à la connexion mobile (app/Http/Controllers/LoginController.php)

4.3.3 Protection CSRF

Les formulaires Blade portent le jeton CSRF (@csrf), que vérifie le middleware VerifyCsrfToken. Le client mobile ne dispose pas de ce jeton de formulaire. Le middleware exempte donc deux cas, et seulement ceux-là (figure 4.37) : les requêtes munies d'un jeton Bearer, et les requêtes JSON vers login, forgot-password et reset-password, qui précèdent l'obtention du jeton.

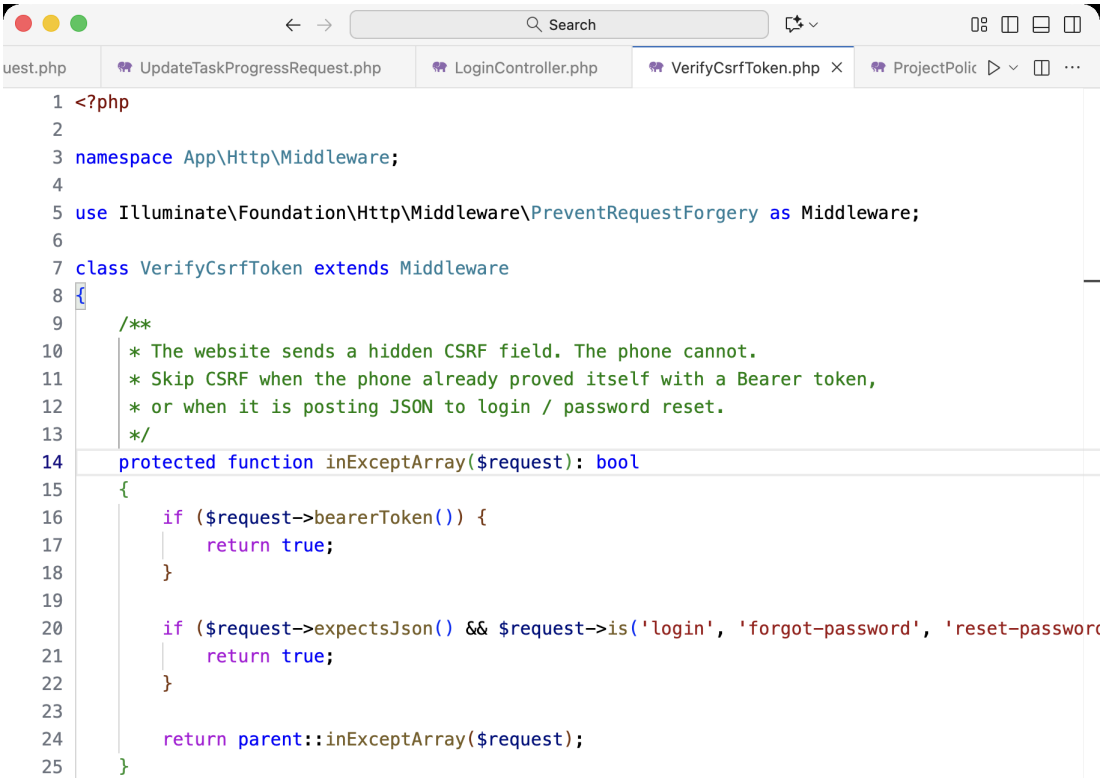


Figure 4.37 – Exemptions CSRF pour le client mobile (app/Http/Middleware/VerifyCsrfToken.php)

4.3.4 Contrôle d'accès

Trois niveaux s'enchaînent. Le middleware auth:web,sanctum refuse toute requête sans session ni jeton : redirection vers /login sur le web, réponse 401 en JSON. Les middleware de rôle du tableau 3.1 filtrent ensuite des groupes de routes entiers. EnsureAdmin (figure 4.38) refuse tout utilisateur qui n'est pas administrateur avec une réponse 403. EnsureTeamLead fait de même pour les routes de chef d'équipe. Les politiques décident ensuite au niveau de la ressource : TaskPolicy::updateStatus réserve l'avancement à l'assigné (figure 4.3), et ProjectPolicy::view n'accorde un projet qu'à un chef dont une équipe dirigée y est liée (figure 4.39).

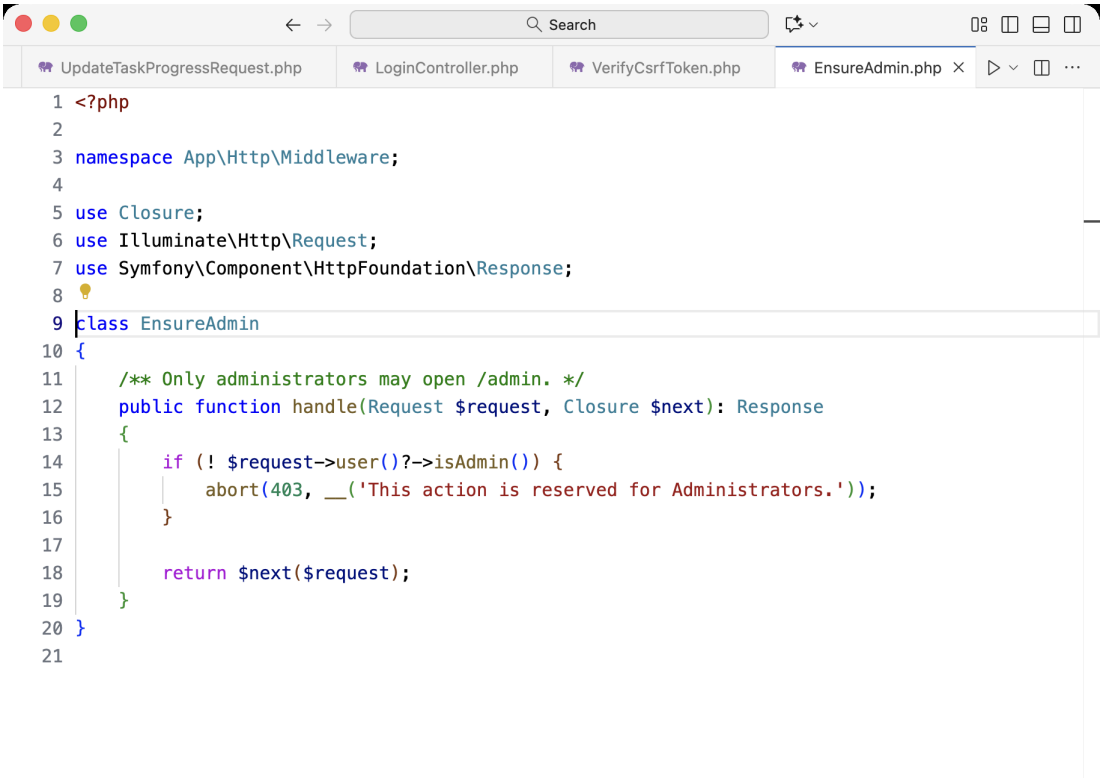


Figure 4.38 – Middleware EnsureAdmin (app/Http/Middleware/EnsureAdmin.php)

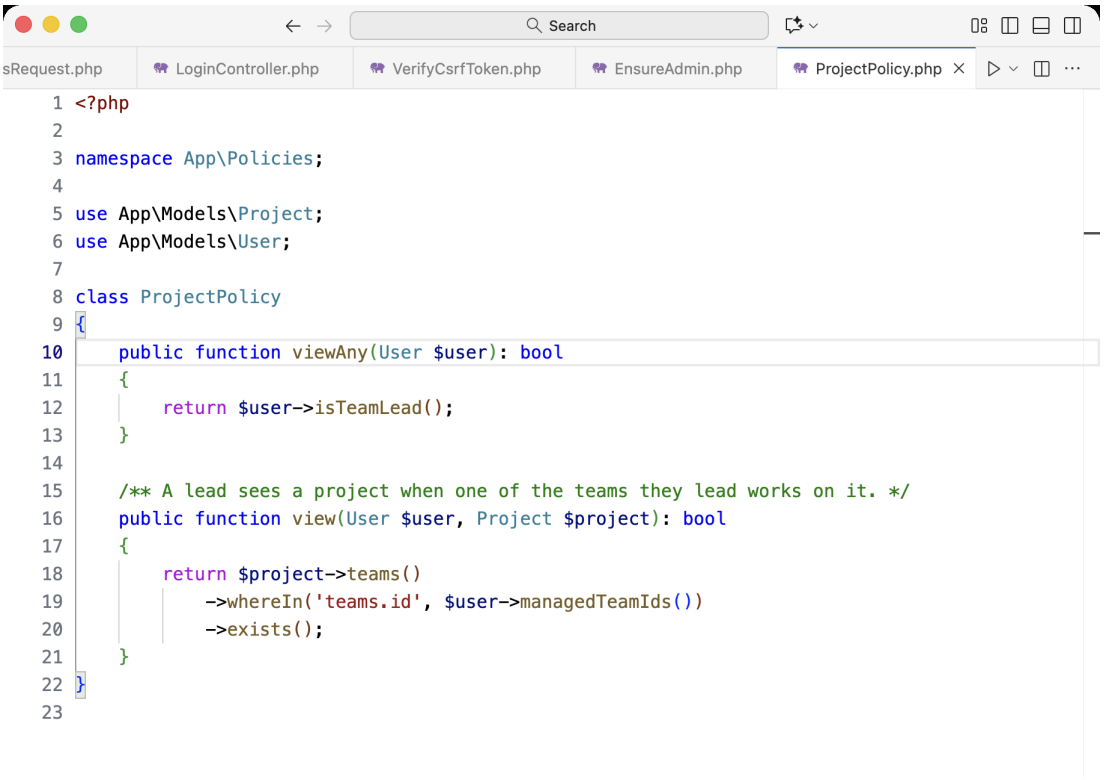


Figure 4.39 – Politique view d'un projet (app/Policies/ProjectPolicy.php)

Ces contrôles s'appuient sur le modèle User. isAdmin() lit le rôle. isTeamLead() vérifie qu'au moins une ligne team_user porte is_team_lead (figure 4.40).

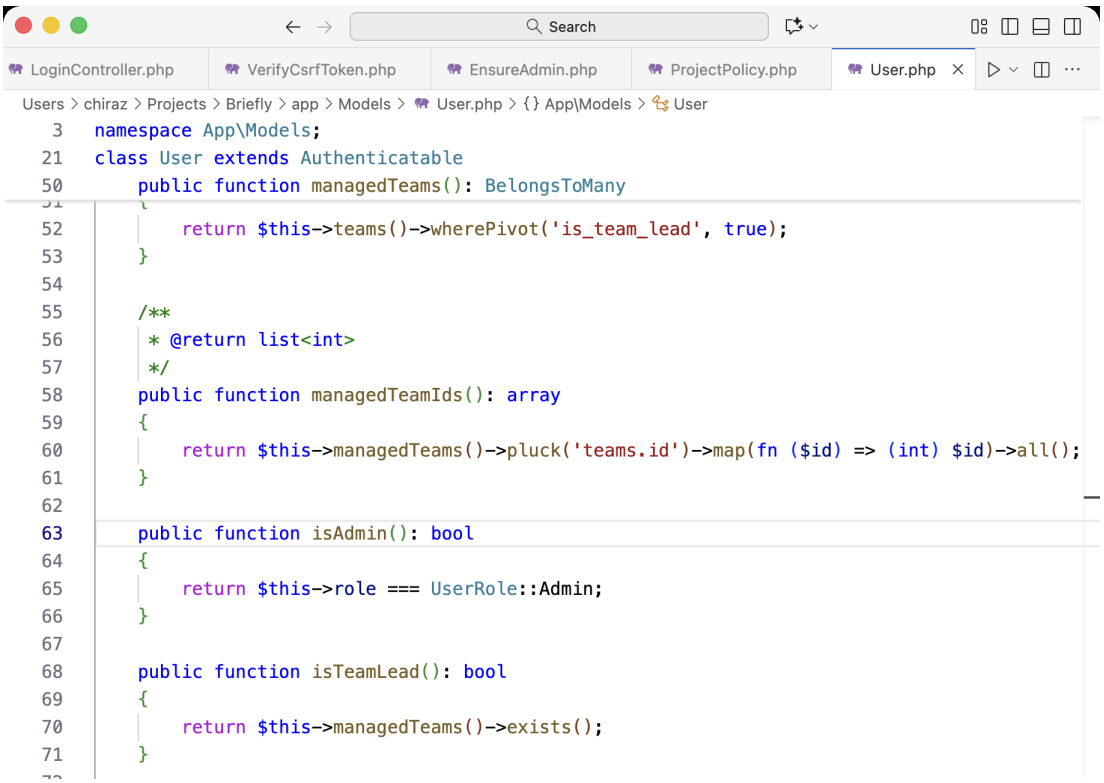


Figure 4.40 – Méthodes isAdmin() et isTeamLead() (app/Models/User.php)

Lors de la création d'une tâche, TaskService::ensureAssigneeOnTeam refuse un assigné qui n'appartient pas à l'équipe de contexte (figure 4.41). C'est la règle derrière le fragment « alt » de la figure 3.5.

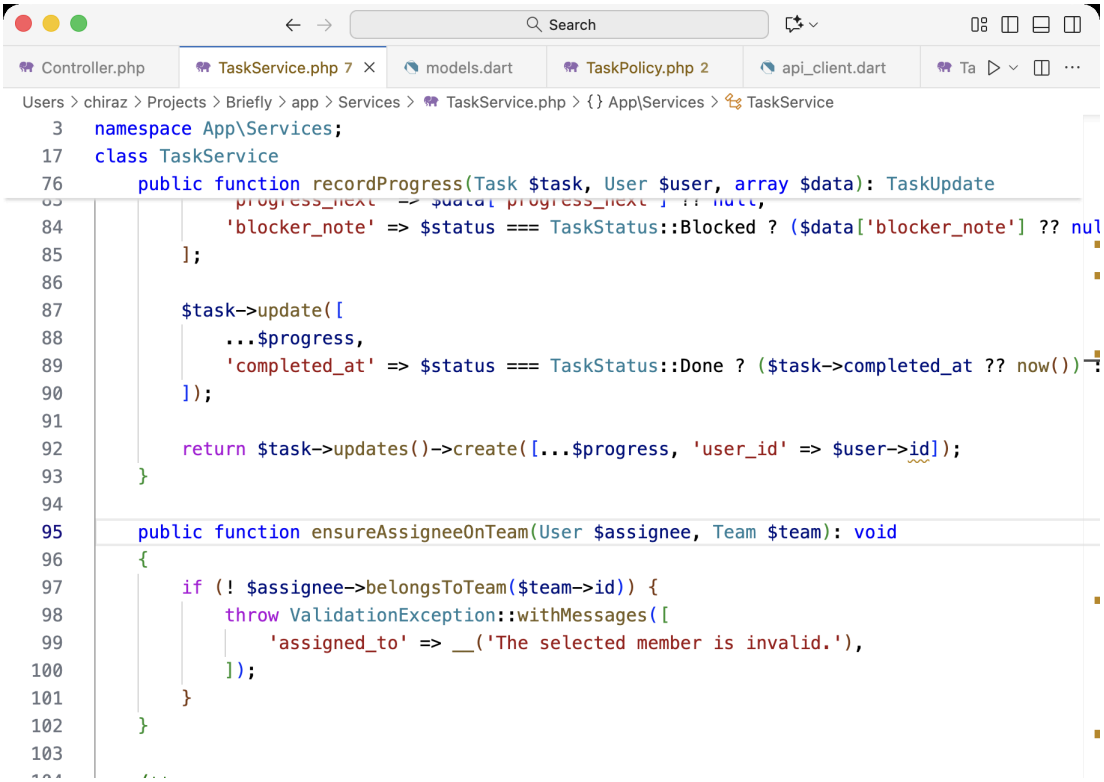


Figure 4.41 – Vérification que l'assigné appartient à l'équipe (app/Services/TaskService.php)

Côté mobile, le client Flutter applique la même règle de note de blocage avant l'envoi (figure 4.42). C'est un confort d'interface. Le serveur reste l'autorité, via UpdateTaskProgressRequest (figure 4.7).

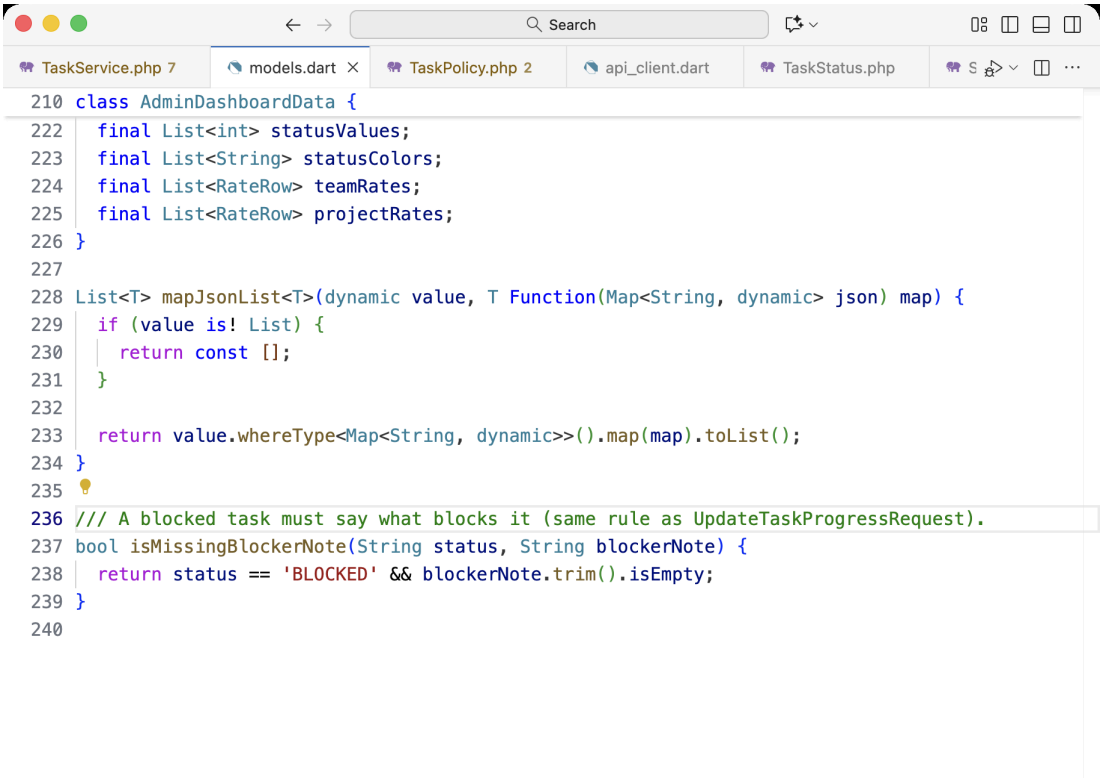


Figure 4.42 – Contrôle local de la note de blocage (mobile/lib/models/models.dart)

Les tableaux 4.4 à 4.8 vérifient ces refus par des réponses 401 et 403.

4.3.5 Formulaires et données

Eloquent transmet les valeurs à MySQL par des requêtes préparées (PDO), ce qui protège contre l'injection SQL. Les seules expressions SQL brutes du code, deux orderByRaw dans TaskService, sont des constantes qui ne contiennent aucune donnée saisie. Les vues Blade affichent les données avec {{ }}, qui échappe le HTML. Aucune vue n'emploie la syntaxe non échappée {!! !!}. Enfin, chaque écriture est validée avant toute mutation : par un Form Request pour les profils, les comptes, les équipes, les projets et les tâches, par validate() dans les contrôleurs de connexion et de réinitialisation du mot de passe.

Le tableau 4.9 résume ces mesures.

Tableau 4.9 – Mesures de sécurité de Briefly

Risque	Mesure	Emplacement
Vol de la base de données	Mots de passe hachés avec bcrypt. Jetons stockés sous forme d'empreinte	User::casts(), Sanctum
Fuite de secrets dans les réponses	password et remember_token masqués	#[Hidden] sur User

Fixation de session	Régénération de la session à la connexion, invalidation à la déconnexion	LoginController
Accès sans authentification	Garde auth:web,sanctum sur toutes les routes métier	routes/web.php
Élévation de privilèges	Middleware de rôle, puis politiques par ressource	EnsureAdmin, EnsureTeamLead, EnsureNotAdmin, TaskPolicy, ProjectPolicy
Falsification de requête (CSRF)	Jeton @csrf vérifié sur les formulaires web	VerifyCsrfToken
Injection SQL	Requêtes préparées par Eloquent	Modèles et services
Injection de script (XSS)	Échappement systématique par {{ }}	Vues Blade
Données invalides	Validation avant écriture	Form Requests de app/Http/Requests, validate() des contrôleurs d'authentification

4.3.6 Limites

Deux points restent à traiter. La route POST /login ne limite pas le nombre de tentatives : aucun limiteur de débit (RateLimiter ou middleware throttle) ne la protège contre les essais répétés. Les jetons Sanctum n'expirent pas, puisque expiration vaut null dans config/sanctum.php. Un jeton reste valide jusqu'à la déconnexion. Ces deux limites figurent parmi les perspectives.

4.4 Intégration continue

Le dépôt est branché sur GitHub Actions via .github/workflows/ci.yml. Il se déclenche à chaque push sur la branche main et à chaque *pull request*. Deux jobs indépendants s'exécutent en parallèle sur une machine Ubuntu (tableau 4.10).

Tableau 4.10 – Jobs du workflow d'intégration continue

Job	Environnement	Étapes
Tests Laravel	PHP 8.5, extensions mbstring, pdo_sqlite et pdo_mysql	composer install, copie de .env.example et php artisan key:generate, puis php artisan test
Tests Flutter	Flutter 3.47.1 (canal stable), dossier mobile/	flutter pub get, flutter analyze, puis flutter test

Le job Laravel exécute la suite PHPUnit des tableaux 4.4 à 4.8 sur une base SQLite en mémoire, comme en local, grâce à la configuration de phpunit.xml. Le job Flutter lance l'analyse statique du code Dart, puis les tests auth_gate_test.dart et

progress_validation_test.dart. Une modification qui casse une règle d'accès ou de validation fait échouer le workflow, et l'échec apparaît dans la *pull request* concernée.

Conclusion

Les six sprints du tableau 2.1 ont été réalisés : socle web (authentification, membre, chef d'équipe, administration), consolidation PHPUnit et sécurité (§4.3), intégration continue (§4.4), puis client Flutter et mise en ligne (chapitre 5). Les tests confirment les refus 401, 403 et 422 et les règles d'avancement ; les figures 4.1 à 4.42 illustrent code et écrans. La limitation des tentatives de connexion et l'expiration des jetons Sanctum restent ouvertes (§4.3.6).

Chapitre 5 : Déploiement

Introduction

Les chapitres 3 et 4 décrivent Briefly sur le poste de développement. Nous avons ensuite mis le serveur en ligne et branché l'application mobile sur cette adresse. Le navigateur et le téléphone joignent ce même serveur. Ce chapitre présente l'hébergement retenu, les grandes étapes de mise en production et les contrôles réalisés sur le web et sur iPhone.

5.1 Choix d'hébergement

Briefly est une application PHP avec une base relationnelle. Le site est servi depuis le dossier public/, en HTTPS, et les mises à jour passent par SSH. L'application n'a pas de worker de file d'attente et ne compile pas de ressources front. L'hébergement doit couvrir ces besoins.

Nous avons retenu alwaysdata, compte lamraoui, à l'adresse https://lamraoui.alwaysdata.net [26]. Ce compte fournit déjà PHP 8.5, MariaDB 11.4, un accès SSH et un certificat sur le sous-domaine, sans frais supplémentaires [27].

Les réglages suivent ce choix. PHP 8.5 et MariaDB 11.4 portent le serveur et la base lamraoui_briefly. La racine web pointe sur www/briefly/public/. APP_ENV vaut production et APP_DEBUG vaut false. Les sessions et le cache restent en base (SESSION_DRIVER=database, CACHE_STORE=database). La file d'attente est synchrone (QUEUE_CONNECTION=sync). L'interface web charge Bootstrap et Chart.js depuis un CDN, donc le déploiement n'exécute pas npm.

Tableau 5.1 – Environnement de production

Élément	Valeur
URL publique	https://lamraoui.alwaysdata.net
PHP	8.5
SGBD	MariaDB 11.4, base lamraoui_briefly
Racine web	www/briefly/public/
APP_ENV / APP_DEBUG	production / false

5.2 Installation sur le serveur

Dans l'administration alwaysdata, nous avons créé une base MariaDB et un site PHP. La racine documentaire pointe sur public/ de Laravel (figures 5.1 et 5.2). Le code a été copié

depuis le Mac vers ~/www/briefly/ par SSH, sans le dossier mobile/ ni docs/. Le fichier .env a été rédigé sur le serveur à partir de .env.example, avec les paramètres de connexion à la base et APP_URL en HTTPS.

Sur le serveur, nous avons enchaîné composer install --no-dev --optimize-autoloader, php artisan key:generate --force, php artisan migrate --force et php artisan optimize. La migration crée les tables métier et Sanctum. Le seeder du dépôt ne tourne pas lorsque APP_ENV vaut production : les comptes de démonstration du poste local ne sont pas recréés en ligne. Un administrateur a été créé manuellement pour les essais. Les dossiers storage/ et bootstrap/cache/ doivent rester inscriptibles pour le compte SSH.

Tableau 5.2 – Variables d'environnement principales en production

Variable	Rôle
APP_URL	URL HTTPS du site
DB_*	Connexion MariaDB alwaysdata
SESSION_DRIVER / CACHE_STORE	database
QUEUE_CONNECTION	sync
MAIL_MAILER	log (courriel non envoyé, voir §5.5)

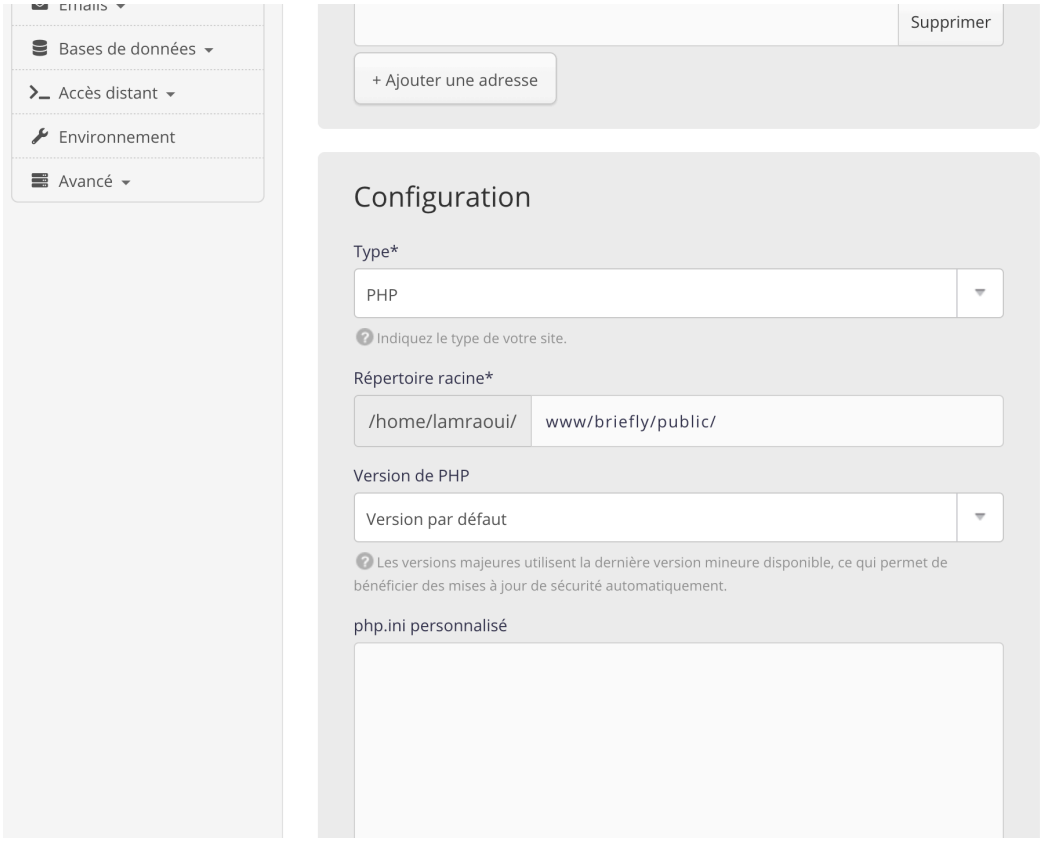


Figure 5.1 – Site PHP : racine www/briefly/public/

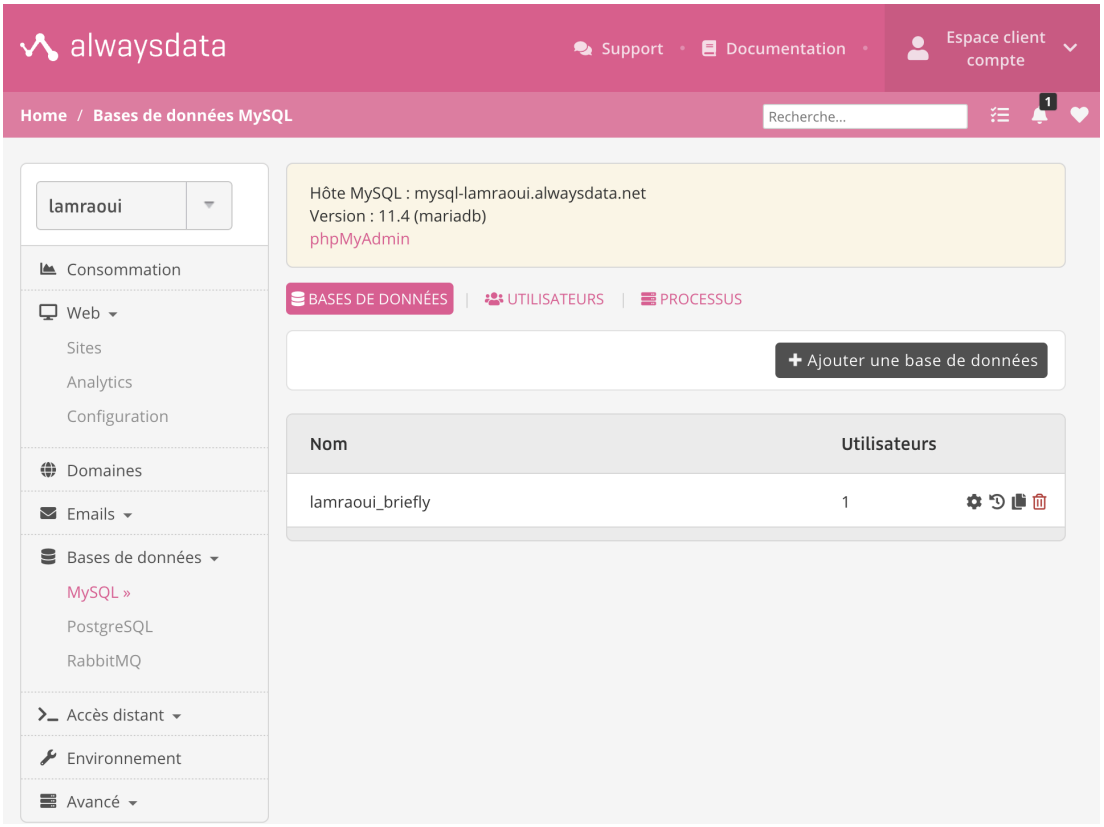


Figure 5.2 – Base lamraoui_briefly

5.3 Vérification côté web

Avant usage, nous avons contrôlé que l'application répondait et qu'un parcours administrateur fonctionnait en HTTPS. La route Laravel GET /up renvoie 200 lorsque l'application démarre correctement (figure 5.3). La page de connexion s'affiche avec un certificat valide. Après authentification, le tableau de bord administrateur reprend les mêmes règles qu'en local (§4.3) : statistiques, projets et tâches issus de la base en ligne (figures 5.4 et 5.5).

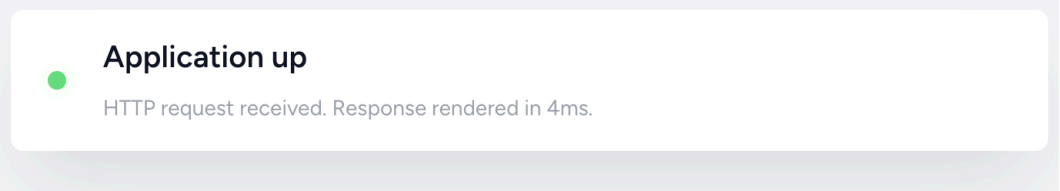


Figure 5.3 – Réponse de GET /up

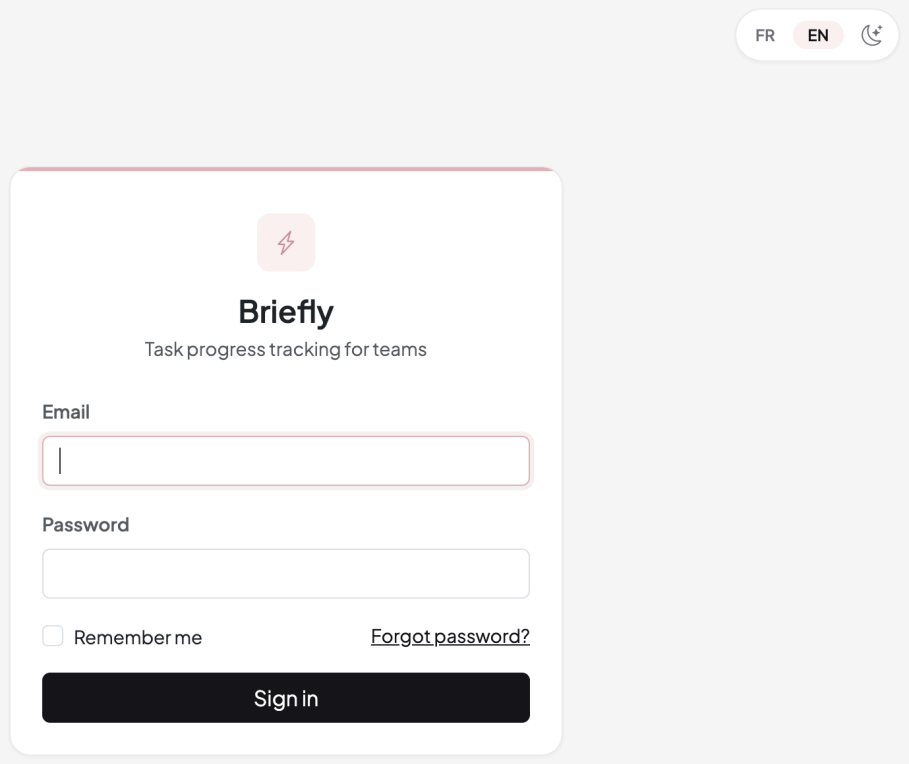


Figure 5.4 – Page de connexion

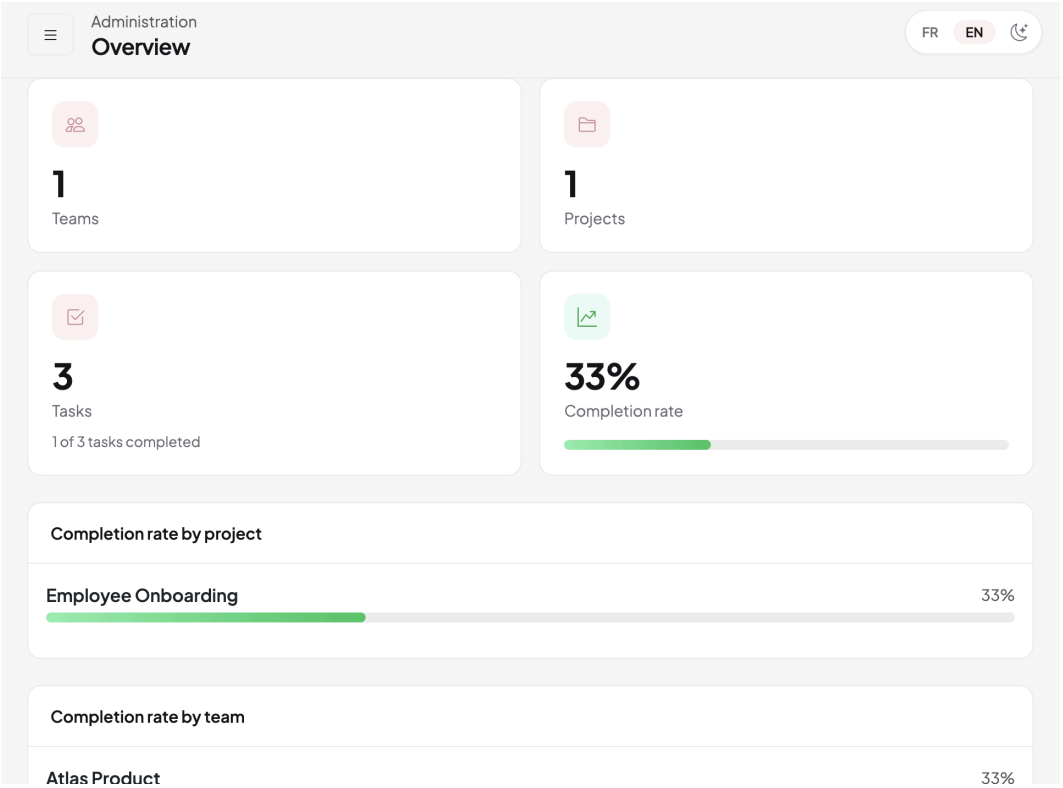


Figure 5.5 – Tableau de bord administrateur

Les tests JSON décrits au sprint 6 (tableau 4.8) ont aussi été rejoués contre l'URL publique : connexion avec Accept: application/json, émission d'un jeton Sanctum, refus 401 sans jeton. Le contrat reste celui validé au chapitre 4.

5.4 Application mobile sur iPhone

Cette mise en ligne clôt le sprint 6 du tableau 2.1 (septembre–début octobre 2026). Une fois le serveur en HTTPS, le mobile n'a plus besoin du réseau local du Mac ni de php artisan serve. La classe ApiConfig fixe l'URL de production (https://lamraoui.alwaysdata.net) ; un --dart-define=API_BASE_URL=... reste possible pour les essais en LAN (§4.1). L'application a été installée sur un iPhone physique en mode release, avec la signature Xcode du projet mobile/ios. Le jeton Sanctum est stocké comme en développement (§4.3.2).

Nous avons ouvert les écrans de connexion et de tableau de bord contre le serveur en ligne (figures 5.6 et 5.7). Les listes et pourcentages proviennent des mêmes routes JSON que sur le poste local.

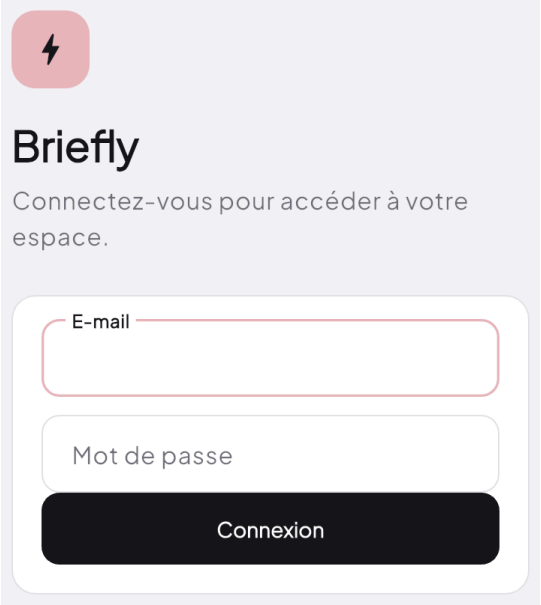


Figure 5.6 – Connexion mobile (LoginScreen)

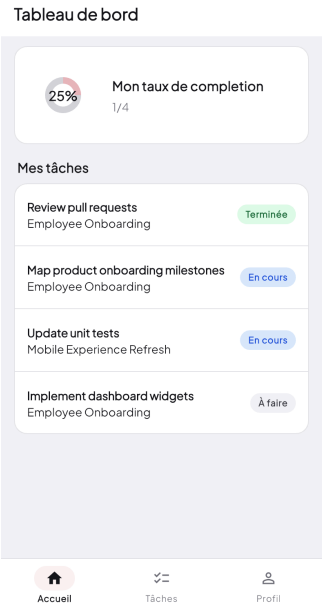


Figure 5.7 – Tableau de bord mobile (DashboardScreen)

5.5 Exploitation et limites

Les mises à jour passent par un nouveau transfert SSH et, si le schéma change, php artisan migrate --force puis php artisan optimize. Le workflow GitHub Actions (§4.4) exécute PHPUnit et les tests Flutter à chaque push ; il ne déploie pas encore automatiquement vers alwaysdata (piste en conclusion générale). Les liens de réinitialisation de mot de passe ne partent pas par courriel : MAIL_MAILER vaut log. Un envoi réel pourrait utiliser le SMTP du compte alwaysdata. Avec QUEUE_CONNECTION=sync, aucune tâche asynchrone longue n'est prévue dans cette version.

Conclusion du chapitre

Briefly est accessible sur https://lamraoui.alwaysdata.net pour le navigateur et depuis l'application Flutter installée sur iPhone. L'hébergement alwaysdata couvre PHP, MariaDB et HTTPS ; le déploiement Laravel repose sur une racine public/, un .env de production et les migrations Artisan. Le mobile rejoint le même serveur que le navigateur et s'authentifie par un jeton d'accès. Les captures de ce chapitre montrent un parcours web et mobile cohérent sur ce serveur. Les mises à jour restent manuelles (SSH, migrate, optimize) ; le courriel de réinitialisation est journalisé et la CI ne déploie pas encore automatiquement (chapitre 4).

Conclusion générale

Ce mémoire a présenté la conception et la réalisation de Briefly, une application interne de suivi d'avancement des équipes, disponible en version web (Laravel) et mobile (Flutter). Le point de départ était un besoin simple : outiller le *stand-up* quotidien, que les équipes tiennent le plus souvent dans une messagerie, sans structure ni lien avec les tâches.

Briefly répond à ce besoin pour des équipes déjà constituées, sans inscription publique. L'administrateur crée les comptes, les équipes et les projets. Le chef d'équipe crée et suit les tâches de ses équipes. Le membre rend compte de ses tâches en trois champs : ce qui a été réalisé, ce qui vient ensuite, et ce qui bloque. Une tâche bloquée exige une note. Les tableaux de bord de chaque rôle calculent ensuite un taux d'avancement par membre, par équipe et par projet.

Le travail a suivi l'étude préalable, l'analyse des besoins, la conception, puis six sprints (tableau 2.1). Un même serveur sert le web en HTML et le mobile en JSON. PHPUnit et GitHub Actions en vérifient les règles avant chaque évolution du dépôt.

Ce projet nous a permis de pratiquer l'ensemble du cycle d'une application : analyse UML, architecture en couches, conception d'une base relationnelle, développement web avec Laravel, développement mobile avec Flutter, authentification par jeton avec Sanctum, tests automatisés, intégration continue et mise en ligne du serveur (chapitre 5).

Perspectives

Cette version n'inclut pas encore certaines fonctions. Plusieurs évolutions sont envisageables pour les prochaines versions de Briefly :

- **Notifications push.** Prévenir le chef d'équipe sur mobile lorsqu'une tâche passe au statut bloqué, et le membre lorsqu'une tâche lui est assignée.
- **Brief quotidien d'équipe.** Rassembler chaque jour les points d'avancement de tous les membres, à partir de l'historique déjà conservé dans task_updates.
- **Export PDF.** Générer un état d'avancement à partir des tableaux de bord, à joindre à un compte rendu de réunion.
- **Calendrier partagé.** Afficher sur une même vue les échéances des tâches et des projets.

- **Sécurité de l'authentification.** Limiter le nombre de tentatives sur POST /login et donner une durée de validité aux jetons Sanctum (§4.3.6).
- **Déploiement automatisé.** Prolonger le workflow d'intégration continue par une mise en production de l'application après le succès des tests.
- **Tests mobiles.** Étendre les tests Flutter, qui portent aujourd'hui sur l'accès authentifié et la validation de l'avancement, aux principaux écrans.

Les règles métier restant sur le serveur Laravel, chacune de ces évolutions profiterait aux deux clients sans dupliquer la logique.

Netographie

[1] https://www.planzone.fr/blog/quest-ce-que-la-methodologie-agile, consulté le 08/09/2026.

[2] https://agiliste.fr/introduction-methodes-agiles/, consulté le 08/09/2026.

[3] https://scrumguides.org/scrum-guide.html, consulté le 08/09/2026.

[4] https://www.atlassian.com/fr/agile/scrum, consulté le 08/09/2026.

[5] https://www.atlassian.com/software/jira, consulté le 10/09/2026.

[6] https://trello.com/, consulté le 10/09/2026.

[7] https://monday.com/, consulté le 10/09/2026.

[8] https://linear.app/, consulté le 10/09/2026.

[9] https://slack.com/, consulté le 10/09/2026.

[10] https://laravel.com/docs/13.x, consulté le 12/09/2026.

[11] https://laravel.com/docs/13.x/sanctum, consulté le 12/09/2026.

[12] https://laravel.com/docs/13.x/blade, consulté le 12/09/2026.

[13] https://laravel.com/docs/13.x/eloquent, consulté le 12/09/2026.

[14] https://getbootstrap.com/docs/5.3/getting-started/introduction/, consulté le 12/09/2026.

[15] https://www.chartjs.org/docs/latest/, consulté le 12/09/2026.

[16] https://docs.flutter.dev/, consulté le 14/09/2026.

[17] https://riverpod.dev/docs/introduction/getting_started, consulté le 14/09/2026.

[18] https://pub.dev/packages/dio, consulté le 14/09/2026.

[19] https://pub.dev/packages/go_router, consulté le 14/09/2026.

[20] https://www.ibm.com/docs/fr/rsm/7.5.0?topic=uml-sequence-diagrams, consulté le 14/09/2026.

[21] https://www.drawio.com/doc/, consulté le 15/09/2026.

[22] https://dev.mysql.com/doc/refman/8.4/en/, consulté le 15/09/2026.

[23] https://docs.phpunit.de/, consulté le 15/09/2026.

[24] https://laravel.com/docs/13.x/hashing, consulté le 01/10/2026.

[25] https://docs.github.com/fr/actions, consulté le 01/10/2026.

[26] https://www.alwaysdata.com/fr/offers/, consulté le 03/10/2026.

[27] https://help.alwaysdata.com/fr/docs/hebergement-web/languages/php/, consulté le 03/10/2026.